

Agent Harness：实践者教材

这是一份基于事实的 agent harness 工程综述，主要整理自 [Awesome Harness Engineering](#) 阅读列表和 [OpenReview](#) 综述论文 [Agent Harness Engineering: A Survey](#)。正文中的每个关键论断都在相应位置给出行内引用。

21 sections · generated from Markdown sources

前言

这本书讨论的是语言模型真正被拿来做事时，围绕模型构建的那套系统。这套系统现在常被称为 *harness*。下面的章节会把相关文献串成一条连续叙事。读者如果想把某条线索追溯到原始来源，可以在对应论断旁找到引用。

这个领域的基本前提很简单。LangChain 的 Vivek Trivedy 将其概括为：“Agent = Model + Harness。如果你不是模型，那你就是 *harness*。”(LangChain - The Anatomy of an Agent Harness)。除此之外，系统提示、工具、沙箱、记忆、子代理、控制流、评估基础设施，都属于 *harness*。如何把这些东西设计好，就是本书要研究的内容。

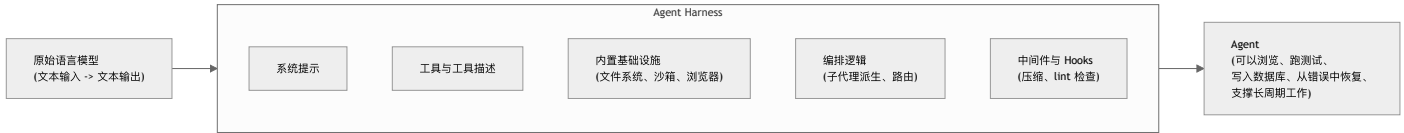
贯穿全书，可以先把模型理解为一个接收 token、输出 token 的组件。它可能输出给用户看的文本，也可能输出请求某个动作的结构化文本，但真正执行动作的始终是模型周围的软件。区分“模型输出”和“真实世界效果”，正是后续章节反复讨论上下文、工具、状态、测试、沙箱和评估的原因。

本书所依赖的模型内部机制——token、attention 与 KV-cache、context window、采样、检索，以及 tool-call 协议——都在配套的 stage-1 卷中讲解；在这里我们用一句话点出每个概念，并在其之上构建 *harness engineering* 这一层。真正属于本书的新内容主要是软件工程：context、工具、状态、sandbox 和评估。

开始之前

这是一个两卷序列中的第二本。它假定你已经读过 *LLM Foundations for Harness Engineering*，并且熟悉 token、attention 与 KV-cache、context window、采样、post-training、检索、agent loop、tool-call 协议、prompt injection，以及 pass@k/pass^k。凡是本书提到某个 Foundations 概念之处，都会向回指（例如“Foundations 第 9 章”），而不是重新推导。没有 stage-1 基础的读者仍然可以跟上叙事，但应把这些一行回顾当作指引，而不是完整讲解。

核心公式



要点

- 语言模型本身不能维护状态、执行代码或访问实时知识；这些都是 *harness* 层面的能力。
- *Harness engineering* 不等同于 *prompt engineering*：它迭代的是整个系统，而不是单个提示词。
- 这个领域仍然年轻，许多关键文章发表于 2025 和 2026 年，但实践正在快速成熟。
- 本书为每个论断都附上引用，方便读者回到原文。

延伸阅读

- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- *Awesome Harness Engineering* reading list: <https://github.com/walkinglabs/awesome-harness-engineering>

第 1 章：什么是 Agent Harness?

1.1 Model + Harness 公式

原始语言模型接收文本并输出文本——它的原生能力与边界正是配套前置卷的主题（见《LLM Foundations》第 1 章）。要把它变成 agent，也就是一个能浏览代码库、运行测试、写入数据库、与用户对话、从错误中恢复，并在数小时任务中持续推进的系统，额外能力都必须围绕模型构建出来。LangChain 将 harness 枚举为：系统提示、工具及其描述、内置基础设施（文件系统、沙箱、浏览器）、子代理派生和模型路由等编排逻辑，以及压缩、续跑、lint 检查等确定性执行的 hooks 或中间件 (LangChain - The Anatomy of an Agent Harness)。

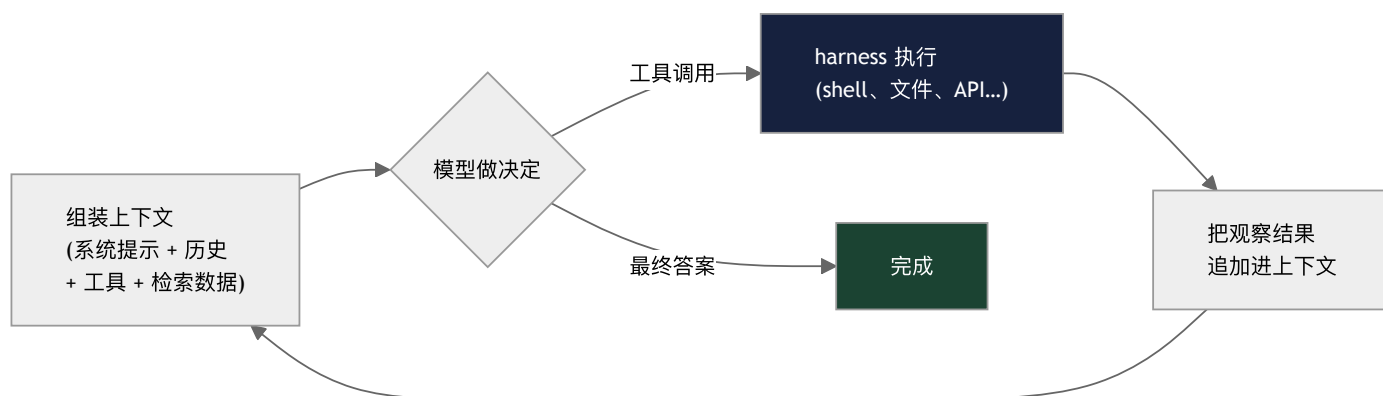
这个框架之所以重要，是因为它把设计问题摆到了明处。模型开箱即用时，无法跨交互维护持久状态、执行代码、访问实时知识，也不能为了完成任务自行配置环境和安装包；这些都是 harness 层能力 (LangChain - The Anatomy of an Agent Harness)。即使是最基本的聊天，也就是模型看起来“记得”刚才说过什么，本质上也是一个 harness 模式：一个 while-loop 记录历史消息，并把新消息追加进上下文。

HumanLayer 从配置 coding agent 的视角给出的工作定义基本相同：“coding agent = AI model(s) + harness”。这里的 harness 是 agent 的运行时，或者说是模型用来与环境交互的外围设备 (HumanLayer - Skill Issue: Harness Engineering for Coding Agents)。

1.2 Agent 循环

agent 循环——以及“工具调用是 harness 执行的结构化输出，而非模型真的直接行动”这一原则——已在配套的前置卷中确立（见《LLM Foundations》第 1 章与第 12 章）。这里快速回顾：一次原始模型调用是一次性的（文本进，文本出）；agent 把它包进一个循环，循环组装上下文，让模型发出最终答案或一个 工具调用（通常是 JSON 的结构化输出，指明工具名和参数），由确定性的 harness 代码执行该调用，追加观察结果，再带着更长的上下文重复。HumanLayer 把同一点表述为“tools are just structured outputs” (HumanLayer - 12-Factor Agents)：文件编辑、shell 命令、浏览器点击、数据库写入，都只有在 harness 接受模型请求并执行之后才会变成真实效果。这个循环中心的模型，就是 Anthropic 所说的 augmented LLM、研究文献所称的 ReAct (reason + act)（机制见《LLM Foundations》第 12 章）。

这个循环的两个后果贯穿全书。第一，上下文单调增长：每一轮都追加一个工具调用及其观察结果，因此 N 步任务会累积 N 轮历史。这就是为什么第 2 章把上下文当作有限资源，也是压缩、子代理和记忆之所以存在的原因。第二，模型本身从不执行任何东西——它只发出请求，由确定性的 harness 代码决定满足哪些请求。请求与执行之间的这个间隙，正是后续章节中每一道护栏、沙箱、hook 和审批关卡的插入点：harness 就处在循环里，夹在模型所要求的与实际发生的之间。



1.3 Harness 的边界：内层与外层

“Harness”这个词的用法并不总是严格。Thoughtworks 的作者指出，不同人说 harness 时可能指不同层。Birgitta Böckeler 建议把它理解成三层同心圆：核心是模型，中间是 coding agent 的 builder harness (Anthropic、OpenAI 等提供的系统提示和工具)，外层是 user harness (团队为了适配自己代码库而添加的 AGENTS.md、hooks、skills、review agents) (Thoughtworks / Martin Fowler - Harness Engineering)。多数工程师的日常工作主要发生在外层。

OpenReview 综述论文 Agent Harness Engineering: A Survey 给了同一边界一个更正式的定义：harness 是一套软件与接口基底，负责管理 foundation model 如何感知上下文、调用工具、跨时间行动，并在部署环境中保持可审计 (OpenReview - Agent Harness Engineering: A Survey)。它的 binding-constraint thesis 比“harness 有用”更强：对长周期 agent 来说，可靠性常常既取决于模型本身的质量，也同等地取决于包裹模型的运行基底。

1.4 ETCLOVG：七层系统地图

这篇综述用 **ETCLOVG** 组织 harness 设计空间：Execution environment、Tool interface、Context、Lifecycle、Observability、Verification、Governance ([OpenReview - Agent Harness Engineering: A Survey](#))。这个分类的价值在于，它防止“harness”被缩减成只有 prompt 或只有 tool。

- **Execution environment**：动作变成真实效果的沙箱、浏览器、操作系统、代码执行器或托管云环境。
- **Tool interface**：协议、schema、registry、function calling、MCP/A2A 式边界，以及工具选择策略。
- **Context**：prompt 组装、检索、记忆、压缩、状态摘要，以及模型被允许看到什么。
- **Lifecycle**：任务启动、计划、checkpoint/resume、失败恢复、handoff、session 终止和长周期状态。
- **Observability**：trace、telemetry、成本归因、延迟、token 记账和失败取证。
- **Verification**：eval harness、grader、任务集、outcome check 和 readiness gate。
- **Governance**：权限、策略语言、审计轨迹、人类审批、constitutional/rule-based 控制和跨层安全。

本书多数章节都可以看作对这七层的展开。第 2-3 章主要讨论 context 和 memory；第 4-5 章覆盖 tools 和 execution；第 7-9 章进入 lifecycle；第 10-12 章展开 verification、observability 和 governance，并在展望中再次回到。

1.5 Harness 为什么存在：从模型缺陷倒推

LangChain 给出了一种有用推导：先列出你希望 agent 具备的行为，再列出模型原生做不到什么，harness 组件就会自然浮现出来 ([LangChain - The Anatomy of an Agent Harness](#))。文件系统之所以存在，是因为模型只能操作上下文窗口中的内容，而文件系统提供持久存储、卸载工作内容的空间，以及多 agent 和人类协作的界面。Bash 和代码执行之所以存在，是因为预先定义 agent 可能需要的所有工具并不现实；给模型一个通用执行通道，可以让它按需现场设计工具。沙箱之所以存在，是因为执行必须发生在安全边界内。记忆和搜索之所以存在，是因为模型除了权重和当前上下文之外没有新知识，任何新信息都必须注入。压缩、工具结果卸载和 skills 之所以存在，是因为上下文窗口有限，并且会随着填充而退化。

每个组件都是对某个具体限制的回应。Harness 整体就是这些回应的总和。

1.6 历史脉络：从 Prompt Engineering 到 Harness Engineering

Anthropic 将最近的转变描述为自然演进。早期 LLM 应用的主导工作是 *prompt engineering*：为一次性任务编写和组织指令。随着应用发展成多轮、长时间运行的 agent，相关工作转向 *context engineering*：在 LLM 推理时策划和维护最优 token 集合，包括提示词之外进入上下文的一切信息 ([Anthropic - Effective Context Engineering for AI Agents](#))。

Harness engineering 位于 context engineering 之上。Mitchell Hashimoto 的说法是，每当 agent 犯错，就花时间把系统工程化到它以后不再犯同一个错 ([HumanLayer - Skill Issue: Harness Engineering for Coding Agents](#) quoting Hashimoto)。Prompt engineering 调的是一个提示；harness engineering 迭代的是承载这个提示运行的整个系统。

2026 年又出现了一次重构，名字叫 *loop engineering* (循环工程)。随着 agent 开始在长周期上无人值守地运行，工作的核心单元再次转移——从 prompt，到 context，到那个决定“prompt 什么、何时 prompt、结果是否够好”的 loop ([Addy Osmani - Loop Engineering](#))。与其说它是 harness engineering 的对手，不如说它是后者面向运维的外层控制循环视角——环绕 agent 的 trigger、verifier 和 stop rule——第 8 章会完整展开它。

1.7 Framework、Runtime 与 Harness

这三个词有时会被混用。LangChain 的 Harrison Chase 给出了如下区分 ([LangChain - Agent Frameworks, Runtimes, and Harnesses, Oh My!](#))：

Framework，例如 LangChain、Vercel AI SDK、CrewAI、OpenAI Agents SDK、Google ADK，提供抽象，帮助快速开始并标准化应用构建方式。*Runtime*，例如 LangGraph、Temporal、Inngest，负责基础设施层问题：持久执行、流式输出、human-in-the-loop、线程级和跨线程持久化。*Harness*，例如 LangChain 的 DeepAgents 或 Anthropic 的 Claude Agent SDK，则位于更高一层：它带有默认提示、带立场的工具处理、规划工具、文件系统访问，以及其他“开箱即用”的能力。边界会模糊 (LangGraph 可以合理地既被叫 runtime 也被叫 framework)，但这个区分有助于判断应该采用什么。

1.8 怀疑论观点

Harness engineering 这个视角并非没有反对意见。HumanLayer 在“Skill Issue”中的论点是，很多团队把问题归因于模型：“GPT-6 会解决”“我们只需要更好的指令遵循”，但真正的问题往往是 harness 配置 ([HumanLayer - Skill Issue: Harness Engineering for Coding Agents](#))。随着模型进步，现有失败模式会消失，但更聪明的模型会被交给更难的问题，并继续以意想不到的方式失败，因为意外失败是非确定性系统的基本属性。结论是：harness engineering 是长期工作，不是模型足够好后就能丢掉的脚手架。

LangChain 得出类似结论：随着模型更强大，今天 harness 中的一些能力会被吸收到模型中，但 harness engineering 仍然有用，既用于弥补缺陷，也用于围绕模型智能构建更有效的系统 ([LangChain - The Anatomy of an Agent Harness](#))。

图：模型到 Harness 层



要点

- **Agent = Model + Harness**: 任何超出原始文本输入输出的能力，都必须由周边系统工程化出来。
- **Agent 循环是基础**: 组装上下文、模型发出工具调用、harness 执行、结果被追加——循环往复，每一轮都让上下文增长。
- **工具调用是结构化请求**: 模型用文本提出动作，harness 决定哪些请求会变成真实效果。
- **三层同心圆**: LLM 核心、AI 实验室提供的 builder harness、团队自行构建的 user harness。
- **ETCLOVG 提供系统地图**: execution、tools、context、lifecycle、observability、verification、governance 都是 harness 层。
- **Harness 组件来自模型缺陷**: 文件系统、沙箱、记忆、压缩分别回应具体限制。
- **Harness engineering 是长期工作**: 模型越强，任务越难，新的失败模式也会出现。
- **Framework 不等于 Runtime, 不等于 Harness**: 理解区别有助于做采用决策。

延伸阅读

- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- Harrison Chase, *Agent Frameworks, Runtimes, and Harnesses, Oh My!*, LangChain, Oct 2025. <https://blog.langchain.com/agent-frameworks-runtimes-and-harnesses-oh-my/>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- Birgitta Böckeler, *Harness Engineering for Coding Agent Users*, Thoughtworks / martinowler.com, Apr 2026. <https://martinowler.com/articles/exploring-gen-ai/harness-engineering.html>
- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>

第 2 章：上下文是一种有限资源

2.1 Context Rot 与注意力预算

对 agent harness 来说，最重要的运行约束之一是：有用上下文不等于最大上下文长度。底层失败模式——context rot，即模型输出随输入增长而退化——在配套卷中已经介绍（见《LLM Foundations》第 9 章），并给出了它的两层模型（长度诱发，加上无关材料累积）。这里我们把它当作本章的主导设计约束。Anthropic 把同一失败模式与 needle-in-a-haystack benchmark 联系起来：随着 token 数增加，模型从上下文中准确回忆信息的能力可能下降（Anthropic - Effective Context Engineering for AI Agents）。这个效应取决于模型、任务和相关信息在上下文中的位置，所以应当把它理解成概率性工程约束，而不是每个 prompt 都严格成立的硬规律。

Anthropic 的机制解释并不是说 transformer 字面意义上“耗尽”了注意力，而是长上下文为模型带来了更多需要表示的 token 关系，而训练数据和位置机制通常更擅长较短、更局部的依赖。位置编码插值和其他长上下文技术可以让模型处理比原始训练更长的序列，但仍可能降低位置分辨率或检索可靠性（Anthropic - Effective Context Engineering for AI Agents）。

实践结论是：上下文是一种边际收益递减的有限资源。Anthropic 把它称作“注意力预算”。HumanLayer 说得更直白：即便模型支持更长上下文，生产系统通常也应偏向小而聚焦的提示与上下文；在他们的经验中，开放式“tool-calling loop”在大约 10-20 轮之后往往会变得难以恢复（HumanLayer - 12-Factor Agents）。

更大的上下文窗口在缺失信息确实需要出现时有帮助，但它不会自动带来更好的注意力或指令遵循。HumanLayer 指出，扩展上下文版本通常用 YaRN 等技术延长序列长度，而不是增加模型的有效“指令预算”；对 needle-in-a-haystack 式任务来说，更大的窗口有时只是把草堆变得更大（HumanLayer - Skill Issue）。

2.2 有效上下文的结构

在预算约束下，目标是 Anthropic 所说的：“最小的一组高信号 token，使期望结果的概率最大化”（Anthropic - Effective Context Engineering for AI Agents）。具体来说：

系统提示应当处在合适的“高度”：既不要为每个边界情况写硬编码 if-else，也不要只有模糊高层原则；它应该具体到足以引导行为，同时保留模型的强启发能力。Anthropic 建议用 XML 或 Markdown 分隔符组织提示，例如

`<background_information>`、`<instructions>`、`## Tool guidance` 等，但随着模型改进，格式本身的重要性会下降。

工具定义了 agent 与环境之间的契约。它们应当自包含、描述清晰、重叠最少。Anthropic 看到的常见失败是工具集臃肿，覆盖过多功能且决策点模糊；如果人类工程师都无法明确判断某个场景该用哪个工具，就不能期待 AI agent 做得更好。

Few-shot 示例应该多样且典型，而不是把每个边界情况都列成清单。Anthropic 的类比是：对 LLM 来说，示例是“胜过千言万语的图片”。

2.3 KV-Cache：为什么稳定前缀有价值

KV-cache 与前缀稳定规则——包括“改动一个早期 token（例如系统提示里的时间戳），其后每个 token 都要重新计算”这一机制——在配套卷中已有覆盖（见《LLM Foundations》第 9 章）；这里我们把 cache 当作一个生产成本杠杆。有一个实用杠杆，它在上下文工程的学术文献中几乎不被提及，却是生产级 agent 设计的核心，那就是 KV-cache。Manus 认为其命中率是“生产阶段 AI agent 最重要的指标”（Manus - Context Engineering for AI Agents）。

它为什么主导成本模型：典型 agent 接收输入，从工具空间中选择动作，执行动作，并把动作和观察结果追加进上下文供下一轮使用。上下文随每一步增长，而输出通常较短，使 prefilling 与 decoding 的比例极度倾斜。Manus 报告其平均输入/输出 token 比约为 100:1。相同前缀可以由 KV-cache 服务，从而降低首 token 延迟和推理成本。Manus 引用 Claude Sonnet 价格：缓存输入 token 每百万 \$0.30，未缓存 \$3，是 10 倍差距。

Manus 保持 cache 命中的三条规则是：保持 prompt 前缀稳定（一个 token 的差异都会从该点起使 cache 失效，所以把秒级时间戳放进系统提示代价很高）；上下文以追加方式增长，并使用确定性 JSON 序列化（某些库不保证 key 顺序，会悄悄破坏 cache）；当推理框架需要时，显式标记 cache 断点。

2.4 Mask，而不是删除

Manus 的第二条原则关于动作空间。随着工具数量增长，MCP（Model Context Protocol，模型上下文协议——见第 4 章）让用户轻易接入数百个工具，常见冲动是运行中动态加载和卸载工具。Manus 的实验给出明确规则：避免这样做。工具定义位于上下文前部，任何变化都会使后续 cache 失效；前面轮次还可能引用已不存在的工具，导致 schema violation 或幻觉调用（Manus - Context Engineering for AI Agents）。

替代方案是 action masking：在上下文中保持稳定的工具界面，然后根据当前状态约束可选动作。不同 provider 和 harness 可能用 logit constraint、tool-choice 控制、response prefill，或运行时 validator 拒绝不允许的动作来实现 (Manus - Context Engineering for AI Agents)。Manus 使用一致的动作名前缀，例如浏览器工具 `browser_*`、shell 工具 `shell_*`，这样就能用简单约束启用或排除整个工具组。

2.5 文件系统作为工作记忆

即使有 128K token 窗口（这是 Manus 当时引用的数字，如今的窗口更大，但下面描述的动态不变），真实 agentic 工作也经常超出上下文。网页和 PDF 的观察结果很大；性能在达到技术上限前就会退化；即使用 cache，长输入仍然昂贵 (Manus - Context Engineering for AI Agents)。

Manus 的解决方案，也是 Anthropic 和 LangChain 都趋同的方向，是把文件系统当作 agent 的工作记忆：它远大于上下文、跨轮次持久、可由 agent 直接操作，并用路径作为引用索引。它不是人类意义上的“记忆”；如果没有良好文件名、摘要、索引或检索习惯，agent 仍可能找不到自己写过的东西。Manus 的压缩策略刻意保持可恢复：只要保留 URL，就可以从上下文中移除网页内容；只要保留路径，就可以省略文档内容。

LangChain 称文件系统是“可能最基础的 harness primitive”，因为它提供读取数据、代码和文档的工作区；让 agent 可以逐步卸载工作，而不是全放在上下文里；还天然成为多 agent 和人机协作界面 (LangChain - The Anatomy of an Agent Harness)。叠加 git 后，还能得到版本、回滚和分支。

对 coding agent 来说，这个原则还要再推进一步：仓库应当成为项目的事实来源。OpenAI 的 Codex harness 指南强调，agent-first 仓库会把 agent 需要的上下文放在代码附近：`AGENTS.md` 负责指路，短设计文档解释局部架构，结构检查把关键规则编码下来 (OpenAI - Harness Engineering)。如果一个决策只存在于 Slack、ticket 或资深工程师脑子里，它就不是新 agent session 可以稳定依赖的上下文。

这不是说要把更多文本塞进 prompt，而是要让仓库事实可发现。一个实用的冷启动测试是：开一个没有口头背景的新 agent session，只让它看仓库，然后问五个问题：这是什么系统、怎么组织、怎么运行、怎么验证、当前状态是什么？如果答案不能从 repo 文件和标准命令中找到，harness 就存在知识可见性缺口。修这个缺口通常靠贴近代码的小文档、稳定入口文件、进度文件和验证命令，而不是一个巨大的根指令文件。

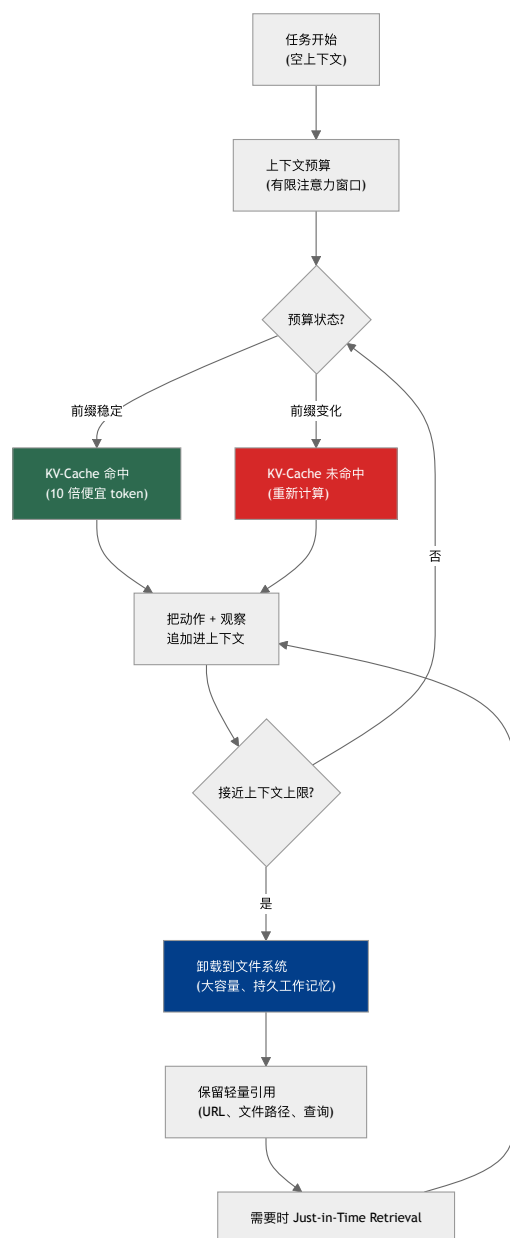
2.6 Just-in-Time Retrieval

传统模式是：预先 embed 一切，检索 top-k chunk，再 prepend 到上下文（即 RAG 流水线，见《LLM Foundations》第 11 章）。现在它正被 just-in-time 方法补充：与其预处理全部内容，agent 保留轻量标识符（文件路径、查询、链接），在需要时动态加载数据进上下文 (Anthropic - Effective Context Engineering for AI Agents)。

Anthropic 的 Claude Code 在大型代码库工作中使用这个模式：模型编写目标查询、存储结果，并使用 `head`、`tail` 等工具分析大数据，而不是全部加载。文件路径元数据本身也有信息量：`tests/` 里的 `test_utils.py` 与 `src/core_logic/` 里的同名文件角色不同。目录层级、命名和时间戳都会成为 agent 导航信号。

代价是：运行时探索比检索预计算数据更慢；缺少工具指导的 agent 会浪费上下文追逐死路。常见混合模式是：先提供少量高价值上下文，例如 `CLAUDE.md` 或 `AGENTS.md` 中的项目说明，然后让 agent 按需探索其余部分。

图：上下文预算 -> KV-Cache -> 文件系统记忆



要点

- **Context rot** 足以影响设计：更大上下文有帮助，但不能替代上下文策划。
- **KV-cache** 是重要生产指标：稳定前缀对延迟和成本的影响可能与模型质量同样关键。
- 能 **mask** 就不要频繁更换工具界面：运行中动态增删工具会破坏 cache locality，并可能导致 schema violation。
- 文件系统是工作记忆，不是魔法记忆：它更大、更持久，但仍需要路径、摘要和检索纪律。
- 仓库是 **agent** 的事实来源：关键项目事实需要能在冷启动时从 repo 文件和命令中发现。
- **Just-in-time retrieval** 往往优于预加载：保留轻量标识符，只在需要时加载数据。

延伸阅读

- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- Yichao 'Peak' Ji, *Context Engineering for AI Agents: Lessons from Building Manus*, Manus, Jul 2025. <https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>

- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- OpenAI, *Harness Engineering: Leveraging Codex in an Agent-First World*, Feb 2026. <https://openai.com/index/harness-engineering/>

第 3 章：压缩、记忆与子代理模式

即便上下文工程做得很好，长周期任务也经常会超过单个上下文窗口。相关文献通常汇聚到三类技术。

3.1 压缩

压缩 (compaction) 会在对话接近上下文窗口上限时，总结当前对话，并用该总结重新开启一个新的上下文窗口 ([Anthropic - Effective Context Engineering for AI Agents](#))。在 Claude Code 中，消息历史会被交给模型，并要求保留架构决策、未解决 bug 和实现细节，同时丢弃冗余工具输出。随后 agent 以压缩后的上下文和最近访问的文件继续工作。触发时机——通常是上下文用量的某个 token 或百分比水位线——本身就是一个需要调优的参数：触发太晚，可能在一轮中途就溢出；触发太早，又会丢掉仍然有用的细节。

Anthropic 对压缩提示的建议是：在真实复杂 trace 上调优；先最大化 recall，确保相关信息被捕获；再迭代 precision，移除多余内容。最轻量的做法是清理工具结果：工具被调用且结果已被后续行动吸收后，原始结果通常可以丢弃。

关键限制是：压缩是有损的。它适合保留决策、目标、约束和产物指针，而不是保留长调试 trace 的每个细节。好的 harness 会把压缩与可恢复引用结合起来：commit hash、文件路径、issue ID、URL、短笔记，让后续 agent 在摘要不够时能重新加载一手证据。

3.2 结构化笔记

互补模式是 *agentic memory*：让 agent 定期把笔记写到磁盘，之后再加载。Anthropic 用 Claude 玩宝可梦作为一个清晰的例子：在数千个游戏步骤中，agent 会维护各类计数（“在过去 1,234 步里，我一直在 1 号道路训练宝可梦，皮卡丘已升 8 级，目标是 10 级”）、绘制地区地图、记录战斗策略，使它能在上下文重置后继续多小时的训练序列 ([Anthropic - Effective Context Engineering for AI Agents](#))。

Manus 的 `todo.md` 技巧是这一模式的专门形式，但它还有下一节中的额外作用。

3.3 反复复述：把注意力拉回上下文尾部

Manus 报告称，在处理复杂任务时，agent 会创建 `todo.md`，并随着任务推进逐步重写它，勾掉已完成项目。这不仅是为了组织工作。典型 Manus 任务平均约 50 次工具调用；在长上下文中，模型容易偏离主题或忘记早期目标。通过反复重写 todo list，agent 把目标“复述”到上下文尾部，将全局计划推入模型最近的注意范围，缓解“lost-in-the-middle”问题（其底层效应见配套卷《LLM Foundations》第 9 章）([Manus - Context Engineering for AI Agents](#))。

3.4 子代理与上下文防火墙

第三种模式，也是架构影响最大的一种，是子代理分解。一个专门的 sub-agent 在自己的上下文窗口内处理聚焦任务，内部可能使用数万 token，最后只向父 agent 返回 1,000-2,000 token 的压缩摘要 ([Anthropic - Effective Context Engineering for AI Agents](#))。HumanLayer 称其为 *context firewall*：负责编排的父线程永远看不到子代理工作的中间噪声，只接收浓缩结果，因此更久地避免进入“dumb zone” ([HumanLayer - Skill Issue](#))。

HumanLayer 对这里什么有效、什么无效说得很明确。把子代理设成“前端工程师”“后端工程师”这种 persona 通常不有效；把子代理用于上下文控制才有效。适合子代理的任务是：最终答案简单，但中间工具调用很多，例如定位代码定义、追踪跨服务的信息流、做大范围研究。

子代理也有助于成本控制：HumanLayer 用昂贵模型 (Opus) 做 orchestrator，用更便宜的模型 (Sonnet 或 Haiku) 做子代理。没有必要用 Opus token 做 `grep`。

Anthropic 的多代理研究系统是这一模式规模化应用的典型例子。lead agent 分析查询并并行派生专门 sub-agents，各自探索一个方面；每个 sub-agent 有自己的上下文窗口；结果被压缩回 lead，由 lead 综合成最终报告。lead-agent-as-Opus、sub-agents-as-Sonnet 的配置在 Anthropic 内部研究评估中比单 agent Opus 相对高出 90.2% ([Anthropic - How We Built Our Multi-Agent Research System](#))。机制很大程度是 token economics：他们的分析中，三个因素解释了 BrowseComp benchmark 上 95% 的性能方差，其中 token 使用量单独解释了 80%。

代价是成本。在 Anthropic 的数据中，single-agent run 使用的 token 约为 chat 的 4 倍，多 agent 系统约为 15 倍——因此多 agent 系统的成本约为单个 agent 的 4 倍（这是推导出的比值，而非该来源直接给出的数字）。所以它们只有在高价值且并行化确实有帮助的任务上才经济。它不适合共享可变状态的强耦合子任务；许多重新实现类的 coding task 属于这一类。但在 coding workflow 中，如果委托工作是只读调查，或能按 ownership 边界清晰拆开，它仍有价值。当前模型也不擅长 agent 间实时协调，所以 coordinator 必须明确任务边界。

3.5 不要把 Few-Shot 做成惯性

Manus 提供了一个反直觉原则：上下文里过度一致可能有害。模型很会模仿，会照着上下文中的模式继续。如果 trace 充满相似的 action-observation 对，模型会在不再合适时继续沿用模式，导致漂移、过度泛化和幻觉。Manus 的例子是批量审阅 20 份简历，agent 会进入节奏，开始为重复而重复 ([Manus - Context Engineering for AI Agents](#))。

他们的修复方式是引入小的结构化变化：不同序列化模板、替代说法、轻微重排、受控噪声。trace 多样性可以让注意力分布更均衡。

3.6 保留有用的错误

互补原则是：不要抹掉有用错误。自然冲动是重试失败动作，并隐藏失败 trace；但 Manus 认为这会删除模型需要用来远离类似错误的证据 ([Manus - Context Engineering for AI Agents](#))。在上下文中保留最近的失败动作和相关 stack trace，让模型可以隐式学习。这不等于永远保留无限日志；重复的相同失败应压缩成简短诊断和 retry counter。Manus 称错误恢复是“真正 agentic 行为最清晰的指标之一”，并指出它在学术 benchmark 中代表性不足。

HumanLayer 将其形式化为 Factor 9：把错误压缩进上下文。Agent 的 *self-healing* 能力，即读取错误并调整下一次调用，是 LLM agent 的真实优势之一，而它只有在错误可见时才有效 ([HumanLayer - 12-Factor Agents](#))。配合连续相同错误计数器，这个模式更稳健。

3.7 成本、延迟与模型路由

上面的子代理模式已经把成本当作一个设计变量——用昂贵模型做编排，用更便宜的模型做跑腿。值得把这个一般原则讲明：成本和延迟是 harness 的一等关切，不是事后才考虑的东西。

文献中反复出现三个杠杆：

- **模型路由**。并非每一步都需要最强的模型。harness 可以把便宜、高频的工作——一次 grep、一次分类、一段简短摘要——路由给小而快的模型，把 frontier 模型留给推理密集的步骤。HumanLayer 用 Opus 做 orchestrator，用 Sonnet 或 Haiku 做子代理 ([HumanLayer - Skill Issue](#))；Anthropic 的研究系统采用同样的 lead agent / sub-agent 拆分（见第 7 章）。
- **KV-cache**。稳定的上下文前缀由缓存服务，价格约为未缓存 token 的十分之一，延迟也只是其一小部分（其机制见配套卷《LLM Foundations》第 9 章；亦见第 2 章）。在生产 agent 中，缓存纪律往往是单个最大的成本杠杆。
- **Token 核算**。Agentic 工作负载偏重 prefill——Manus 报告输入输出比约 100:1——而成本随上下文长度增长。多 agent 系统可能烧掉单次 chat 约 15 倍的 token，这正是它们只在高价值任务上才划算的原因。

延迟有它自己的结构。首 token 延迟主要由 prefill 决定，因而由缓存命中决定；端到端延迟则主要由顺序模型往返的次数决定。并行工具调用和并行子代理能大幅削减墙钟时间——在 Anthropic 的研究工作负载上最长达 90%（见第 7 章）——却不减少总 token 成本。一般规则是：把 token、金钱和秒数都当作显式预算，并清楚哪个杠杆影响哪一个。

3.8 具名记忆架构

第 3.1-3.3 节把压缩、笔记和复述当作 harness 技术来讲。研究文献则进一步把这些想法打包成了几个值得记住的具名记忆系统。

- **MemGPT** 明确类比操作系统：它把上下文窗口当作快速的“主存”，把外部存储当作“磁盘”，让模型通过函数调用，在固定大小的上下文里换入换出信息——即虚拟上下文管理 (*virtual context management*)。有界窗口由此得以呈现出远大于自身容量的表象 ([Packer et al. - MemGPT: Towards LLMs as Operating Systems](#))。该系统现已产品化为 Letta。
- **Mem0** 是一个记忆层：它从对话中动态抽取显著事实，与已有存储整合，并在后续轮次检索；其图变体还能捕获实体间关系。它报告的收益是运营层面的——在跨会话的长对话中，token 成本和延迟都远低于回放完整历史 ([Chhikara et al. - Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory](#))。
- **Sleep-time compute**（睡眠期计算）利用的是空闲时间：agent 不在请求之间闲着，而是离线处理其上下文——预判可能的后续问题并预先计算推断——使后续查询所需的 test-time compute 更少。在部分 benchmark 上，这把达到给定准确率所需的推理预算削减了约 5 倍 ([Lin et al. - Sleep-time Compute: Beyond Inference Scaling at Test-time](#))。

harness 视角与第 3.1 节一致：受管记忆很强大，但同样是有损的，并引入了它自己的失败面——检索到错误记忆、整合出错、自信地断言过时事实。当会话很长、跨会话回忆确实重要时，这些系统才物有所值；在短任务上，它们只是在第 3.2 节已有的笔记之上徒增开销。

3.9 多代理拓扑及其失败原因

第 3.4 节把子代理作为上下文防火墙引入，并介绍了 orchestrator-worker 配置；第 6、7 章进一步展开编排。除 orchestrator-worker 外，实践者还会用到一小套多代理*拓扑 (topology) *词汇：

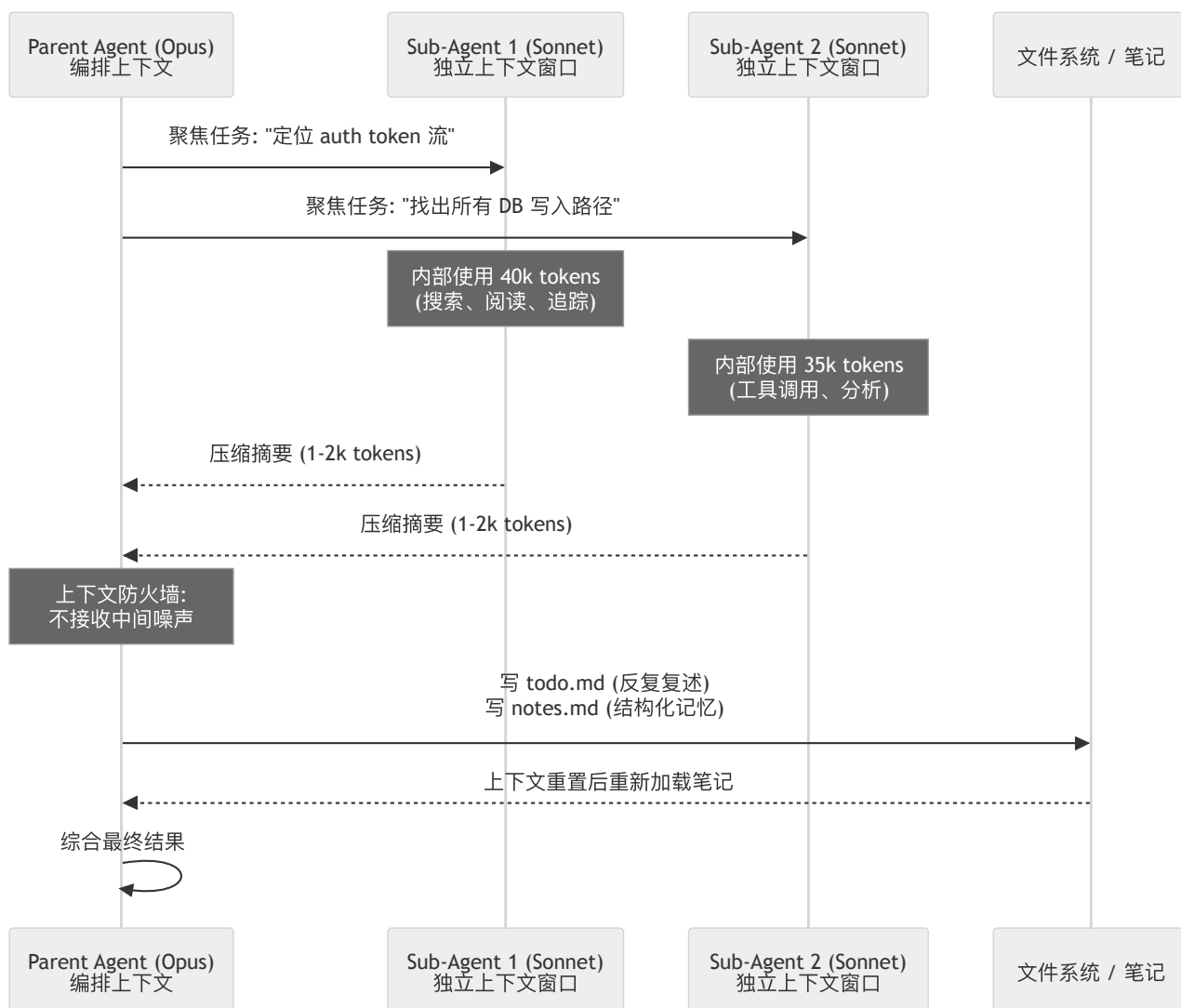
- **Orchestrator-worker (supervisor, 主管)**：一个 lead agent 分解任务、委派给 worker，再综合结果（第 6、7 章）。
- **Hierarchical (分层)**：主管之上还有主管，适用于分解太深、单个 lead 容纳不下的任务。

- **Blackboard / shared memory (黑板 / 共享记忆)**：agent 通过读写一块公共工作区来协调，而非彼此直接发消息——当许多 agent 共同构建同一份不断演进的产物时尤其有用。
- **Debate / voting (辩论 / 投票)**：多个 agent 争论或投票以提升可靠性，是第 6.3 节 parallelization-voting 模式的多代理形式。

诱惑在于把更多 agent 读成更多能力，而经验记录要清醒得多。MAST 研究在七个流行的多代理框架上人工标注了 200 多个任务，归纳出 14 种失败模式，分为三大类：**规格问题 (specification issues)** (角色和提示欠定义)、**代理间错位 (inter-agent misalignment)** (agent 各说各话、丢失信息或偏离共享目标)、以及**任务验证 (task verification)** (对最终结果的检查薄弱或缺失) (Cemri et al. - [Why Do Multi-Agent LLM Systems Fail?](#))。要点是：很大一部分失败并非模型本身能力不足，而是**协调与验证的崩溃**——恰恰是 harness 掌管的那些面。

实践指引由此得出：优先选择能奏效的最简拓扑 (第 6.1 节)；明确任务边界，使 worker 不重叠、不漏活 (第 3.4 节、第 7 章)；把验证当作一等 agent 而非事后补丁 (第 7 章的 generator-evaluator 拆分)，因为 MAST 把薄弱验证列为三大失败族之一。

图：父 Agent -> 子 Agent -> 压缩结果 (上下文防火墙)



要点

- 压缩延长任务视野，但会丢细节：保留关键决策和可恢复引用，而不是每个原始观察。
- 结构化笔记支持多会话连续性：把进度写到磁盘的 agent，可以在上下文重置后恢复工作。
- 反复复述缓解 lost-in-the-middle：反复重写 todo list，把目标推入最近注意范围。
- 上下文防火墙是子代理模式的关键价值：父 agent 不看中间噪声，只接收浓缩结果。
- 保留有用错误：self-healing 需要相关错误 trace 可见，但重复失败应被压缩。
- 成本与延迟是设计变量：把便宜的工作路由给小模型，保持 KV-cache 命中，并用并行换取墙钟速度。

- 具名记忆系统把记忆模式打包：MemGPT（OS 式虚拟上下文）、Mem0（抽取-整合-检索）和 sleep-time compute（离线预处理）值得了解——但每个都会引入自己的失败面：检索出错和信息陈旧。
- 更多 agent 放大的是协调失败，不只是成本：MAST 分类法发现规格缺口、代理间错位与薄弱验证——都是 harness 掌管的面——主导了多代理失败；优先选最简拓扑，并让验证成为一等公民。

延伸阅读

- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- Yichao 'Peak' Ji, *Context Engineering for AI Agents: Lessons from Building Manus*, Manus, Jul 2025. <https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- Jeremy Hadfield et al., *How We Built Our Multi-Agent Research System*, Anthropic, Jun 2025. <https://www.anthropic.com/engineering/multi-agent-research-system>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Charles Packer et al., *MemGPT: Towards LLMs as Operating Systems*, arXiv, Oct 2023. <https://arxiv.org/abs/2310.08560>
- Prateek Chhikara et al., *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*, arXiv, Apr 2025. <https://arxiv.org/abs/2504.19413>
- Kevin Lin et al., *Sleep-time Compute: Beyond Inference Scaling at Test-time*, arXiv, Apr 2025. <https://arxiv.org/abs/2504.13171>
- Mert Cemri et al., *Why Do Multi-Agent LLM Systems Fail?*, arXiv, Mar 2025. <https://arxiv.org/abs/2503.13657>

第 4 章：工具与 Agent-Computer Interface

4.1 为什么工具设计不同

Anthropic 借用 HCI 的类比提出 *agent-computer interface* (ACI)：agent 如何使用工具，值得像人类如何使用界面一样认真设计 ([Anthropic - Building Effective Agents](#))。工具格式有三条具体建议：

- 给模型足够 token 在提交语法前“思考”，因为语法一旦写出很难撤回。
- 保持格式接近模型训练数据中常见形式。
- 避免过高的格式开销，例如 diff header 中的精确行号，或 JSON 嵌套代码时的过度转义。

Anthropic 构建 SWE-bench agent 时，在工具 schema 优化上花的时间比 prompt 本身更多。一个具体改进是：把工具路径从相对路径改为绝对路径后，agent 离开根目录后的路径错误几乎全部消失 ([Anthropic - Building Effective Agents](#))。

4.2 选择正确工具与正确数量

Anthropic 后续的 “Writing Effective Tools for Agents” 进一步说明核心陷阱：更多工具并不带来更好结果 ([Anthropic - Writing Effective Tools for Agents](#))。常见错误是把每个 API endpoint 都包装成工具，不管它是否适合 agent。Agent 的 *affordance* 与传统软件不同：如果用 `list_contacts` 查通讯录，agent 必须逐 token 读每个联系人，等于暴力搜索；更合适的是 `search_contacts` 或 `message_contact`。

工具应合并经常串联的操作。与其给 `list_users`、`list_events`、`create_event`，不如给 `schedule_event`。与其给 `read_logs`，不如给 `search_logs`。与其给 `get_customer_by_id` + `list_transactions` + `list_notes`，不如给 `get_customer_context`。

4.3 工具从哪里来：Model Context Protocol

本章前后几节都假设 agent 已经有了一组设计良好的工具。实践中，这些工具很多是通过一个标准接口送达的：*Model Context Protocol* (MCP，模型上下文协议)。简要回顾一下，MCP 是一个开放的客户端-服务器标准：一个 *MCP server* 通过统一协议暴露一组工具，以及可选的资源 and 可复用提示，任何兼容 MCP 的客户端 (Claude Code、IDE、自定义 agent) 都能发现并调用它们，无需定制集成。传输机制以及结果不可信这一点已在《LLM Foundations》第 12 章讲过；本章把 MCP 当作一个 harness 设计面来处理 ([Anthropic - Code Execution with MCP](#))。

它的价值在于可组合性。一个团队可以把 Google Drive server、Salesforce server 和内部数据库 server 接到同一个 agent 上，每个 server 独立构建和维护，而 agent 看到的是一个合并后的工具界面。这就是 MCP 在本书中反复出现的原因：它是本章其余部分所讲的命名空间、masking 和代码执行模式的底座。

代价是 MCP 让工具供给过剩变得轻而易举。正如第 2 章指出的，MCP 让用户能轻易接入数百个工具，而一个装满数百条工具定义的上下文，正是第 2 章“mask，而不是删除”原则和上面的合并建议所要对抗的膨胀。MCP 是管道，不是工具设计的替代品：一个设计糟糕的 MCP server 只会把设计糟糕的工具规模化地送来。本章的纪律——合并经常串联的操作、用命名空间、限制响应大小、像写入职文档一样写描述——无论工具是手写的还是经 MCP 送达，都同样适用。

4.4 四类集成边界

OpenReview 综述认为，比较工具与协议标准时，按它们跨越的边界来分，比按厂商或发布时间更有用 ([OpenReview - Agent Harness Engineering: A Survey](#))：

- **Model -> Function**：function calling 这类结构化调用，模型输出机器可读请求，由确定性代码执行。
- **Agent -> External capability**：MCP 式解耦，agent runtime 发现外部 server 暴露的 tools、resources 和 prompts。
- **Agent -> Agent**：A2A 式委托，一个 agentic application 把工作交给另一个有自己状态和工具的不透明 agent。
- **Agent -> Repo/environment**：版本控制下的 policy 与 affordance，例如 AGENTS.md、本地 skills、仓库命令和环境特定工具规则。

这个边界视角说明了为什么 MCP、A2A、OpenAPI、function calling 和 AGENTS.md 不应被当成直接替代品。它们解决的是不同集成问题。Harness 设计者要先判断当前跨的是哪条边界，再选择能在边界上保留 provenance、permission、cost 和 failure evidence 的协议与治理模型。

4.5 命名空间

当 agent 能访问几十个 MCP server 和数百个工具时，命名冲突和目的模糊会成为关键失败模式。Anthropic 建议把相关工具放在共同前缀下，例如服务前缀 `asana_*`、`jira_*`，以及服务内部资源前缀 `asana_projects_*`、`asana_users_*`。他们发现前缀与后缀命名方案会对工具使用评估产生非平凡影响，且最佳方案依赖 workload ([Anthropic - Writing Effective Tools for Agents](#))。

Manus 也用相同模式控制动作空间：所有浏览器工具用 `browser_` 前缀，shell 工具用 `shell_` 前缀，从而用简单 `logit` 约束 mask 整组工具 (Manus - Context Engineering for AI Agents)。

4.6 返回有意义的上下文

工具响应应优先考虑相关性，而不是最大灵活性；应优先使用自然语言标识符，而不是技术 ID。Anthropic 发现，把字母数字 UUID 解析成语义标签，甚至 0-indexed ID，可以显著提升 Claude 的精度并减少幻觉 (Anthropic - Writing Effective Tools for Agents)。如果两者都需要，可以让自然名称供 agent 使用、技术 ID 供下游调用，也可以用 `response_format` enum 提供 `concise` 与 `detailed` 两种模式；他们的 Slack 示例中，`concise` 响应体积可以只有 `detailed` 的三分之一。

4.7 Token 高效响应

工具响应是上下文膨胀的主要来源。Anthropic 默认将 Claude Code 的工具响应限制为 25,000 token，并建议结合分页、范围选择、过滤和带合理默认值的截断 (Anthropic - Writing Effective Tools for Agents)。被截断的响应应包含引导，建议 agent 采取更高效策略，例如小而精准的搜索，而不是一次宽泛搜索；错误响应也应有帮助，而不是不透明 `traceback`。

HumanLayer 在自己代码库中的“back-pressure”实践就是直接应用：build 和 test hook 在成功时吞掉输出，只暴露错误。早期他们让 agent 每次改动后跑完整测试套件，4,000 行通过测试输出会灌满上下文，导致 agent 忘记真实任务并开始对测试文件产生幻觉 (HumanLayer - Skill Issue)。

4.8 对工具描述做 Prompt Engineering

Anthropic 认为这是最有效的杠杆之一，并报告称对工具描述的精细改写是 Claude Sonnet 3.5 在 SWE-bench Verified 达到 SOTA 的关键杠杆之一 (Anthropic - Writing Effective Tools for Agents)。建议是：像给刚入职的初级工程师写说明一样写工具描述。把隐含上下文显式化，例如专门查询格式、领域术语、资源之间关系。参数名要明确，如 `user_id` 而不是 `user`。在 workbench 中跑大量例子，观察错误并迭代。

一个具体调试例子：Claude 的 web search 工具刚推出时，trace 显示 Claude 会不必要地把 2025 附加到 `query` 参数中，偏置搜索结果。修复无需重新训练模型，只需要更清楚的工具描述。

4.9 代码执行作为元工具

近期一个转变是：不要把 MCP 工具直接呈现为调用，而是呈现为 agent 通过写代码调用的代码 API。Anthropic 的“Code Execution with MCP”认为，当 agent 面对几十个 MCP server、数百个工具时，预先把每个工具定义加载进上下文，并让每个中间结果都经过模型，非常浪费 (Anthropic - Code Execution with MCP)。

替代方案是：把 MCP server 暴露为 TypeScript 文件系统，每个工具一个文件，文件中是 `callMCPTool` 的类型化 wrapper。Agent 通过列目录和读取所需工具文件来发现工具。在 Anthropic 的 Google Drive -> Salesforce 示例中，这将 token 使用量从 150,000 降到 2,000，节省 98.7%。

收益会叠加：

- **渐进披露**：只在需要时加载工具，减少前置上下文成本。
- **上下文高效结果**：agent 可以在执行环境中把 10,000 行 spreadsheet 过滤到 5 行，再把结果带进模型上下文。
- **更好的控制流**：循环、条件、错误处理用熟悉的代码模式，由 runtime 评估条件，而不是模型用 token 推演。
- **隐私保护操作**：中间结果默认留在执行环境中；只有 agent 显式 log 的内容进入模型。设计正确的 proxy 可以在 MCP client 边界 token 化 PII，使原始值无需进入模型。
- **状态持久化与 skills**：agent 可以把工作代码保存成由 `SKILL.md` 支持的可复用函数，逐步积累工具箱。

Cloudflare 以“Code Mode”为名报告了相似发现，强化了同一个结论：LLM 擅长写代码，开发者应当让它们这么做 (Anthropic - Code Execution with MCP)。

代价是：代码执行需要沙箱基础设施，带来运营和安全成本。

4.10 用 Eval 迭代工具

Anthropic 推荐的工具开发流程有四阶段 (Anthropic - Writing Effective Tools for Agents)：

1. **Prototype**：在本地 MCP server 中原型化工具，手动测试，建立直觉。
2. **Build an evaluation**：使用真实任务、真实数据和可验证成功标准，而不是玩具 sandbox。
3. **Run the evaluation**：程序化运行，并捕获包含 planning summary、工具调用、工具结果、runtime、token count、工具错误的 trace。如果模型暴露 visible thinking mode，它可以帮助调试，但 eval 不应依赖隐藏 chain-of-thought。
4. **Analyze results**：阅读 transcript，注意 agent 没说什么 (LLM 不总是说出真实意图)，并据此重构工具。

Anthropic 在内部 Slack 和 Asana 工具上跑这个循环，发现 Claude 优化版工具在 held-out 测试集上超过专家手写实现。这验证了这个循环，也提供了 agent 改进自身工具的早期实例。

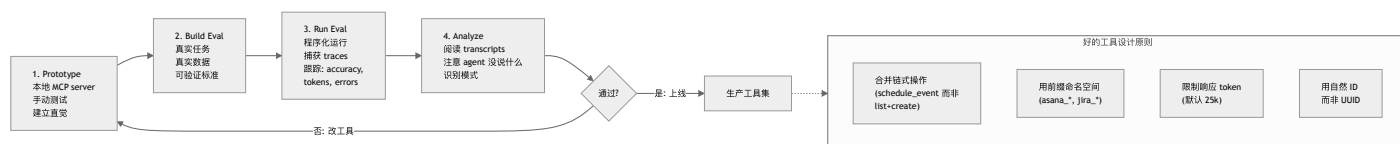
4.11 Agent 到 Agent 的边界：A2A

第 4.4 节的四个集成边界里，model-to-function、agent-to-external-capability (MCP)、agent-to-repo/environment 三个已在上文展开。第四个——agent 到 agent——则有一套新兴的专属标准。**Agent2Agent (A2A)** 协议由 Google 于 2025 年 4 月推出，并于 2025 年 6 月捐赠给 Linux 基金会，是一个开放标准，让一个 agentic 应用把工作委派给另一个*不透明 (opaque)* 的 agent——一个拥有自己模型、工具、记忆和内部状态、且不对外暴露这些的对等体 (Agent2Agent (A2A) Protocol; Linux Foundation - Agent2Agent Protocol Project)。

其机制刻意采用常规做法，以便复用现有 Web 基础设施：通信是 HTTP(S) 上的 JSON-RPC 2.0；每个 agent 发布一张描述其能力的 *Agent Card* 供他人发现；任务生命周期涵盖提交、交互模态（文本、文件、结构化数据）协商，以及把结果流式返回给调用方。

真正有启发的是它与 MCP 的对比。MCP 把一个 agent 向下连接到它掌控、可检视的工具与资源；A2A 把一个 agent 横向连接到一个它既不掌控、也无法窥其内部的对等体。这就把首要工程关切从工具 schema 设计，翻转为跨组织边界的信任与溯源。因为你无法检视对方 agent 的上下文或 sandbox，来自 A2A 对等体的响应正是第 5 章意义上的不可信内容——lethal trifecta 纪律对对等 agent 与对网页同样适用——治理（受限身份、委托授权、审计）必须横跨这次 A2A 调用，而非止于你自己的进程边界（第 5 章、第 17 章）。A2A 并不消除信任问题；它只是把解决这个问题位置标准化了。

图：工具设计流水线



要点

- 工具设计与提示设计同样重要：ACI 类比 HCI 是恰当的。
- 更多工具会伤害而非帮助：把常串联操作合并成专用工具。
- **MCP 标准化了工具的来源**：它让工具能跨独立 server 组合，但也让工具供给过剩变得容易——无论如何，工具设计纪律都适用。
- 工具协议跨越不同边界：function calling、MCP、A2A 和 repo-local policy 互补，不是互相替代。
- 命名空间不是表面美化：它支持工具组 masking，也减少大型 MCP 环境中的冲突。
- 工具响应是上下文膨胀主因：默认 cap、分页、过滤和截断。
- 代码执行作为元工具是阶段性跃迁：Anthropic 示例中，把 MCP 暴露成类型化代码 API 节省 98.7% token。
- 四阶段 **eval loop** 是推荐 workflow：prototype -> build eval -> run eval -> analyze transcripts -> iterate。
- **A2A 标准化了 agent 到 agent 的边界**：它让一个 agent 借助 JSON-RPC + Agent Card 把工作委派给不透明的对等体——把关切从工具 schema 设计转向跨组织边界的信任与溯源，第 5 章的 lethal trifecta 与治理规则在此适用。

延伸阅读

- Ken Aizawa, *Writing Effective Tools for Agents - with Agents*, Anthropic, Sep 2025.
<https://www.anthropic.com/engineering/writing-tools-for-agents>
- Adam Jones and Conor Kelly, *Code Execution with MCP: Building More Efficient Agents*, Anthropic, Nov 2025.
<https://www.anthropic.com/engineering/code-execution-with-mcp>
- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024.
<https://www.anthropic.com/engineering/building-effective-agents>
- Yichao 'Peak' Ji, *Context Engineering for AI Agents: Lessons from Building Manus*, Manus, Jul 2025.
<https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026.
<https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>

- *Agent2Agent (A2A) Protocol*, Google / Linux Foundation, 2025. <https://github.com/a2aproject/A2A>
- Linux Foundation, *Linux Foundation Launches the Agent2Agent Protocol Project*, Jun 2025. <https://www.linuxfoundation.org/press/linux-foundation-launches-the-agent2agent-protocol-project-to-enable-secure-intelligent-communication-between-ai-agents>

第 5 章：沙箱、护栏与安全自治

5.1 Agent 安全威胁模型

本章大部分内容讲的是缓解手段——沙箱、hooks、审批关卡。先要把它们要缓解的东西说清楚是值得的。一个会读取不可信内容、又能对世界采取行动的 agent，有一份特定的风险画像 ([Anthropic - Beyond Permission Prompts](#)):

- **Prompt injection (提示注入)** (见《LLM Foundations》第 8 章 “Prompt Injection as Context Confusion” 与第 12 章) —— 模型无法可靠地把数据和指令分开，因此 agent 所读的不可信内容 (网页、issue 评论、源文件、工具结果) 可以像命令一样操纵它。
- **数据外泄** — 被操纵且有网络访问权的 agent，可以把密钥 (SSH key、API token、专有源码) 发送到攻击者控制的目的地。
- **破坏性操作** — 被操纵且有文件系统或 shell 访问权的 agent，可以删除或损坏文件，或提交并推送有问题的代码。
- **工具与供应链风险** — 恶意或被攻陷的 MCP server、软件包或依赖，可以引入 agent 随后会信任的敌对工具或指令。

实践者最担心的组合，有时被称为 *lethal trifecta* (致命三要素)：访问私有数据、接触不可信内容、向外通信的能力 ([Simon Willison - The lethal trifecta for AI agents](#))。单独任何一项都还能承受；三者集于一个 agent，就意味着一条注入的指令能读到密钥并把它发出去。本章的多数控制手段，都是通过打断三要素中的一条腿来工作的——网络隔离移除向外通信，文件系统隔离移除私有数据访问，审批关卡把人放进有后果操作的路径上。

要带进本章其余部分的视角是：模型不是可信组件。它是一个能力强但可被操纵的核心，而 harness 就是夹在敌对指令与真实后果之间的那一层。

5.2 权限疲劳问题

完全无监督运行的 coding agent 很危险；每一步都请求批准的 coding agent 又不可用。Anthropic 将其称作 approval fatigue：不断点击 approve 会拖慢开发循环，并让用户不再认真看自己批准了什么，反而降低安全性 ([Anthropic - Beyond Permission Prompts](#))。解决方案是结构性的：定义 agent 可以自由行动的边界，只有越界时才请求权限。

在 Anthropic 内部使用中，沙箱安全地减少了 84% 的权限提示 ([Anthropic - Beyond Permission Prompts](#))。

5.3 沙箱同时是笼子、重置按钮和许可证

OpenReview 综述把 sandbox 的作用扩展到安全之外。在 agent 系统中，沙箱同时有三种目的 ([OpenReview - Agent Harness Engineering: A Survey](#)):

- **Security**：限制不可预测的模型生成动作和 prompt-injection 行为的 blast radius。
- **Reproducibility**：给 eval、训练轨迹和长运行 session 一个可重置 baseline。容器或 microVM 可以销毁重建，开发者工作站不行。
- **Liveness**：定义一个 agent 可以自由行动的区域，不必每次文件写入、包安装或网络调用都问人类。

第三点是 agent 时代特有的。沙箱不只是笼子，也是许可证。它把权限从逐动作问题转成 session 配置，让长周期自治可用，而不是退化成权限疲劳。

5.4 文件系统隔离必须与网络隔离配对

Claude Code 的沙箱同时执行两类边界，Anthropic 认为二者都必要。文件系统隔离防止被 prompt injection 的 agent 修改敏感文件；网络隔离防止它泄露数据或下载恶意软件。没有网络隔离，被攻陷的 agent 可能外传 SSH key；没有文件系统隔离，被攻陷的 agent 可能逃出沙箱并访问网络 ([Anthropic - Beyond Permission Prompts](#))。

实现基于 OS 级 primitive：Linux bubblewrap 和 macOS seatbelt，并覆盖 Claude Code 的直接交互以及任何 subprocess。网络访问通过 Unix domain socket 进入 proxy，由 proxy 强制执行域名限制，并在请求新域名时处理用户确认。该 runtime 已开源。

Claude Code on the web 将其扩展为云沙箱，敏感凭据 (git credentials、signing keys) 从不与 agent 同处沙箱中。自定义 proxy 处理 git 交互，只在确认操作被允许后附加 scoped credentials，例如只允许 push 到配置分支。

5.5 Governance：身份、策略与审计

沙箱边界必要但不充分。Governance 层要回答：agent 代表谁行动、拥有什么权限、权限如何随任务上下文变化、行动后留下什么证据 ([OpenReview - Agent Harness Engineering: A Survey](#))。

生产中有三个关键设计动作：

- **身份与 delegated auth**：agent 应使用 scoped credentials 或 delegated identity 行动，而不是以用户的完整 ambient authority (默认隐式权限) 行动。Credential vault 和 proxy 应只在操作被授权的边界处附加 secret。

- **上下文相关权限策略**：静态 allow/deny list 可检查但粗糙。任务感知策略可以在每次调用前评估工具名、参数、session state、目标 repo、网络域名和用户角色，再由确定性 checker 执行结果。
- **可审计 trace**：日志不仅要记录 tool call，还要记录身份、权限决策、policy 版本、参数、输出摘要，以及是否有人类批准升级。

这也是供应链攻击进入 harness 范围的地方。MCP tool poisoning、tool squatting、rug-pull update、幻觉包名、retrieval-source poisoning 都跨越了 tool interface 与 governance 的边界。安全 harness 需要对工具、包、数据集和检索来源做 provenance 与 integrity 检查，而不只是写一句“请小心”。

5.6 Hooks 与 Middleware 作为程序化执行

沙箱是一种程序化护栏；hooks 和 middleware 是另一种更细粒度的护栏。Claude Code 支持用户定义的命令或脚本，在生命周期事件上自动运行，例如 agent 启动、工具调用后、停止时等 ([HumanLayer - Skill Issue](#))。LangChain 的 middleware 概念结构上相似。有些 hook 是完全确定性的脚本；有些是把上下文重新注入模型的过程性检查点。可靠性来自 harness 自动执行它们，而不是依赖模型记住某条规则。

常见用途包括通知（agent 完成时播放声音）、自动批准或拒绝（拒绝 migration 命令，让用户手动运行）、集成（发 Slack 消息、开 PR）、验证（停止时运行 typecheck 和 build，把错误暴露给 agent，迫使其修复后再结束）。HumanLayer 的示例 hook 会在每次 Claude stop 时并行运行 Biome 和 TypeScript；成功时静默退出，失败时只暴露错误并以 exit code 2 返回，告诉 harness 重新拉起 agent。

LangChain 报告称，这类 middleware 是 deepagents-cli 从 Terminal-Bench 2.0 Top 30 提升到 Top 5 的关键。他们的 `PreCompletionChecklistMiddleware` 在 agent 退出前拦截并提醒它对任务 spec 做验证；`LocalContextMiddleware` 启动时映射工作目录和可用工具；`LoopDetectionMiddleware` 跟踪每个文件编辑次数，并在同一文件被编辑 N 次后提示 agent 重新考虑，从而打断在一个已坏方法上做小幅变体的“doom loop” ([LangChain - Improving Deep Agents](#))。

综述中的 governance 分类把这些 hook 放进更大的执行管线：pre-invocation check 可以拒绝危险工具调用，post-invocation hook 可以在不可信输出进入上下文前做 taint 或 redact，stop hook 可以要求验证或审计更新，escalation hook 可以把模糊情况交给人类。行动越有后果，就越不应该依赖模型记住指令。

5.7 前馈与反馈：控制论视角

Thoughtworks 的 Birgitta Böckeler 给出更高层分类 ([Thoughtworks - Harness Engineering](#))。外层 harness 控制分为两个方向：

- **Guides（前馈）** 在 agent 行动前预判并引导行为，提高第一次产出好的概率，例如 AGENTS.md、skills、参考文档、语言服务器提示。
- **Sensors（反馈）** 在 agent 行动后观察并帮助自我修正，例如测试、linter、type checker、AI code review。

只有前馈的 harness 会不断发布规则，却不知道规则是否有效；只有反馈的 harness 会不断抓到同样错误，却无法预防。两者都需要。

每个方向还有第二条轴：

- **Computational 控制**，例如 linter、type checker、结构测试，确定性强、运行快、结果可靠。
- **Inferential 控制**，例如语义分析、AI code review、LLM-as-judge，能处理细微判断，但更慢、更贵、非确定性。

两条轴互相独立。AGENTS.md 中的编码约定是 inferential feedforward。提交时检查模块边界的 ArchUnit 测试是 computational feedback。`/code-review` skill 是 inferential feedback。预启动脚本创建项目结构是 computational feedforward。好的 harness 会混合四类。

5.8 三类调节对象

Böckeler 还按 harness 调节对象区分三类 ([Thoughtworks - Harness Engineering](#))：

- **Maintainability harness**：内部代码质量、重复、复杂度、覆盖率、风格。这是最容易的一类，因为已有几十年工具积累。
- **Architecture fitness harness**：性能、可观测性、可调试性，捕捉应用的横切“fitness functions”。
- **Behavior harness**：应用功能行为是否符合预期。这是未解决类别。今天多数团队依赖功能 spec 作为前馈，用 AI 生成测试作为反馈，有时加 mutation testing；Böckeler 坦率地说，信任 AI 生成测试“还不够好”。

这些类别的意义在于评估 harness 的覆盖面。一个 maintainability 很强、behavior 很弱的 harness 会给人虚假的安全感。

5.9 时机：把质量左移

CI 的经验是，越早发现问题，修复越便宜；harness 设计也是如此。快速 computational sensors (linter、快速测试) 应在 commit 前运行；昂贵 computational 与 inferential sensors (mutation testing、更广泛 code review) 在 pipeline 中 post-integration 运行；持续漂移 sensors (死代码检测、依赖扫描、日志异常 judge) 则独立于变更生命周期持续运行 (Thoughtworks - Harness Engineering)。

Böckeler 注意到，OpenAI Codex 团队的 harness 也类似：用自定义 linter 和结构测试强制分层架构，加上周期性“garbage collection”扫描漂移，并让 agent 建议修复。

5.10 Harnessability 与环境可供性

不是每个代码库都同样容易 harness。强类型语言天然带来 type-checking sensor；清晰模块边界让架构约束规则可写；Spring 等 opinionated framework 抽象掉了 agent 无需操心的细节 (Thoughtworks - Harness Engineering)。

Böckeler 将 *ambient affordances* 这一术语归功于 Ned Letcher，它捕捉了这一点：环境本身会带有一些属性，使 agent 更容易理解、导航和处理 (Thoughtworks - Harness Engineering)。Greenfield 团队可以从第一天就设计这些 affordance；legacy 团队面对的是相反情况：越需要 harness 的地方，越难构建 harness。

面向未来，Böckeler 提出 *harness templates*：按服务拓扑打包 guides 和 sensors，例如 JVM CRUD service、Go event processor、Node dashboard，并随现有 service template 一起分发。Böckeler 援引 Ashby 的必要变异度定律给出形式化理由——调节器必须具有至少与被调节系统同样多的变异度——因此，约束服务拓扑本身就是降低变异度的动作，使完整 harness 更可达 (Thoughtworks - Harness Engineering)。

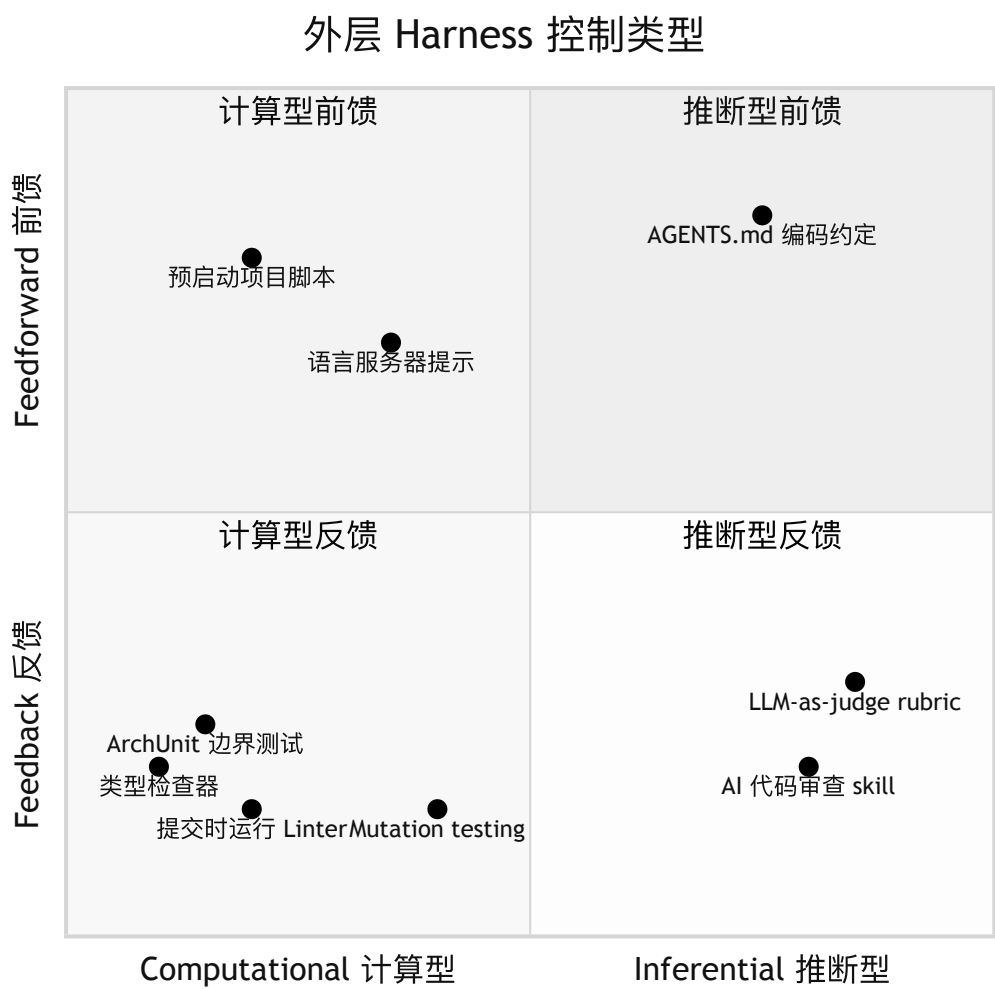
5.11 运营安全：熔断器、终止开关、预算与金丝雀

Sandbox、治理和 hooks (第 5.3-5.6 节) 约束的是 agent 可以做什么。第二类控制约束的是当 agent 或其工具在运行时行为异常时会发生什么。它们几乎原封不动地借自分布式系统的可靠性工程与安全运营，并且属于 harness——因为被操纵或陷入循环的 agent 无法指望它自行施加这些控制。

- **熔断器 (circuit breaker)**。包裹一个不稳定或昂贵的依赖——一个工具、一个下游服务、一个子代理——使其在失败达到阈值后跳闸，让后续调用快速失败，而不是挂起或重试成风暴 (Fowler - CircuitBreaker)。对 agent 而言，这限制了某个开始报错的工具、或某个卡在重试同一失败动作的 agent 的爆炸半径——它是第 3 章“保留有用错误”的对应物：后者让单个失败可见，但决不能让失败无上限地反复发生。
- **终止开关 (kill switch)**。一个由人或策略触发的停止：立即终止一个 agent 或整个 fleet，且独立于 agent 自身的控制流。因为被 prompt 注入的 agent 可能正积极违抗其指令，这个开关必须存在于 harness 中——一个 supervisor 进程、一份可撤销凭证、一次 sandbox 拆除——而不是存在于一句写着“如被要求就停下”的 prompt 里。
- **动作预算、迭代上限与成本调节器**。对工具调用、token、墙钟时间或花费的硬性限额，达到后循环停止并上报，而不是失控奔跑。这是第 8 章循环停止规则和第 17 章按任务预算的运营形式：一个无界循环既是失控的账单，也是失控的爆炸半径。
- **金丝雀令牌 (canary token)**。把假的机密——一个未使用的 API key、一个诱饵文件、一个陷阱 URL——一种在被 prompt 注入的 agent 会去读取或外泄的地方。金丝雀上的回调是一个高信号警报，表明 agent 已被操纵去触碰它不该碰的数据 (Thinkst - Canarytokens)。与 sandbox 不同，金丝雀并不阻止 lethal trifecta 的外泄一环 (第 5.1 节)；它检测它，这正是它成为“预防不完美时最后一道防线”的原因。

框架与本章其余部分一致：失败越严重，就越不该依赖模型选择去避免它。预防 (sandbox、策略) 与检测 (金丝雀、第 17 章的漂移告警) 相互组合；单靠任何一个都不够。

图：前馈/反馈 x 计算/推断



要点

- 威胁模型先行：prompt injection、数据外泄、破坏性操作、供应链风险——“致命三要素”（私有数据 + 不可信内容 + 向外通信）是每道控制手段所针对的核心危险。
- 沙箱减少 84% 权限提示：结构边界优于不断弹窗。
- 沙箱有三项工作：security、reproducibility 和 liveness。
- 文件系统与网络隔离必须配对：二者对应不同攻击向量，单独使用都不足。
- Governance 不只是审批：身份、scoped credentials、policy checks、provenance 与 audit trail 必须跨工具和 session 组合。
- Hooks 与 middleware 是程序化执行：它们不依赖模型记忆，比纯 prompt 约束可靠。
- 前馈与反馈都需要：guide 没有 sensor 就没有学习回路；sensor 没有 guide 只能事后反应。
- 三类 harness 覆盖：maintainability、architecture fitness、behavior；behavior 仍是难题。
- 环境可供性重要：强类型语言和 opinionated framework 让 harness 更容易。
- 运行时安全也需要运营控制：熔断器、终止开关、动作/成本预算与金丝雀令牌在异常发生时约束它——它们存在于 harness 中，因为被操纵的 agent 不能被指望停下自己。

延伸阅读

- David Dworken and Oliver Weller-Davies, *Beyond Permission Prompts*, Anthropic, Oct 2025.
<https://www.anthropic.com/engineering/claude-code-sandboxing>
- Birgitta Böckeler, *Harness Engineering for Coding Agent Users*, Thoughtworks / martinfowler.com, Apr 2026.
<https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026.
<https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>

- Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026. <https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>
- Martin Fowler, *CircuitBreaker*, martinfowler.com, Mar 2014. <https://martinfowler.com/bliki/CircuitBreaker.html>
- Thinkst, *Canarytokens* (free tripwire tokens). <https://canarytokens.org/>

第 6 章：Agentic 工作流程模式

6.1 Workflows 与 Agents

Anthropic 在更大的“agentic systems”类别中区分了两种架构 (Anthropic - Building Effective Agents):

- **Workflows** 通过预定义代码路径编排 LLM 和工具。
- **Agents** 动态决定自己的流程和工具使用。

他们强调的第一原则是：先找到最简单可行方案，只在需要时增加复杂度。许多用例根本不需要 agent；带检索和上下文示例的单次 LLM 调用通常就足够。Workflow 适合结构明确、需要可预测性和一致性的任务；agent 适合需要灵活性和模型驱动决策的大规模场景。

6.2 Augmented LLM

基本构件是 *augmented LLM*：带有检索、工具和记忆的模型。现代模型可以主动使用这些能力：生成自己的查询、选择工具、决定保留什么 (Anthropic - Building Effective Agents)。MCP（见第 4 章）是暴露这些增强能力的常见方式之一。

6.3 组合式工作流程模式

从最简单到最灵活：

Prompt chaining 将任务分解成顺序步骤，每个 LLM 调用处理前一步的输出，并可加程序化 gate。适合任务可清晰分解、愿意用延迟换取准确率的情况。例如：先写营销文案，再翻译。失败模式：延迟会逐步累加，早期调用的错误会向下游传播——要积极设置 gate。

Routing 对输入分类，并派发到专门后续流程。适合输入类别明确、且分类可靠的场景。例如：把客服问题路由到退款、技术支持或一般问题 pipeline。失败模式：在分类错误和模糊输入上会无声失败——要对分类器做埋点，盯住它的错误率。

Parallelization 同时运行多个 LLM 调用并聚合输出。两个变体是 *sectioning*（拆成独立子任务）和 *voting*（同一任务多次运行）。适合加速，或多视角能提升信心的任务，例如多 prompt 漏洞审查、内容审核投票。失败模式：voting 会把 token 成本放大 N 倍，而且相关性错误会让 N 个一致的投票产生虚假的安心感。

Orchestrator-workers 由中心 LLM 动态拆分任务、委托 worker LLM，并综合结果。它不同于 parallelization，因为子任务不是预先定义的。适合子任务形状依赖输入的复杂任务，例如触及多文件的 coding agent、跨多来源研究。失败模式：orchestrator 可能无界地 fan out——要限制 worker 数量和总 token 预算。

Evaluator-optimizer 由一个 LLM 生成，另一个 LLM 批评，循环改进。适合有明确评价标准，且迭代带来可测收益的任务。两个信号是：人类反馈能显著提升输出；LLM 也可能产生类似反馈。例如文学翻译的 critic、多轮研究中的相关性 evaluator。失败模式：循环可能不收敛——要限制迭代次数，因为每一轮都会增加延迟和成本。

6.4 Agent 实现的三条原则

Anthropic 最后给出三条规则 (Anthropic - Building Effective Agents):

1. 保持简单。
2. 优先透明，明确展示 agent 的规划步骤。
3. 认真设计 **agent-computer interface**，包括工具文档和测试。

Framework 能帮助快速开始，但也可能引入抽象层，遮蔽底层 prompt 和工具调用。Anthropic 建议在仍然摸索问题形状时，从直接 API 调用开始；当重复模式和运营需求变清楚后，再引入 framework。

6.5 小代理模式

HumanLayer 的实践版本表达了相同洞见 (HumanLayer - 12-Factor Agents)：“loop until done”模式大约在 10-20 轮后会撞墙，随着累积的 context 和每轮误差不断叠加，agent 失去连贯性（见第 2 章《上下文是一种有限资源》）。有效做法是在更大的确定性 DAG（由代码定义步骤组成的有向无环图）中嵌入小而聚焦的 agent。这种把 LLM 微代理嵌入确定性代码的 harness 形态，也正是 LangChain 所描述的 (LangChain - The Anatomy of an Agent Harness)。他们的 deploybot 示例中，确定性代码负责 staging deploy、e2e test 和真正的 prod deploy 命令；LLM 只负责解释人类自然语言反馈（“能先部署 backend 吗？”）并提出更新步骤。把 agent 的作用域限制在 5-10 步，错误失控发散会少很多。

原则可以推广：随着模型变强，agent 可处理步骤可能变多；但小而聚焦的 agent 方式让你今天就能交付，并随着模型能力增长逐步扩大范围。

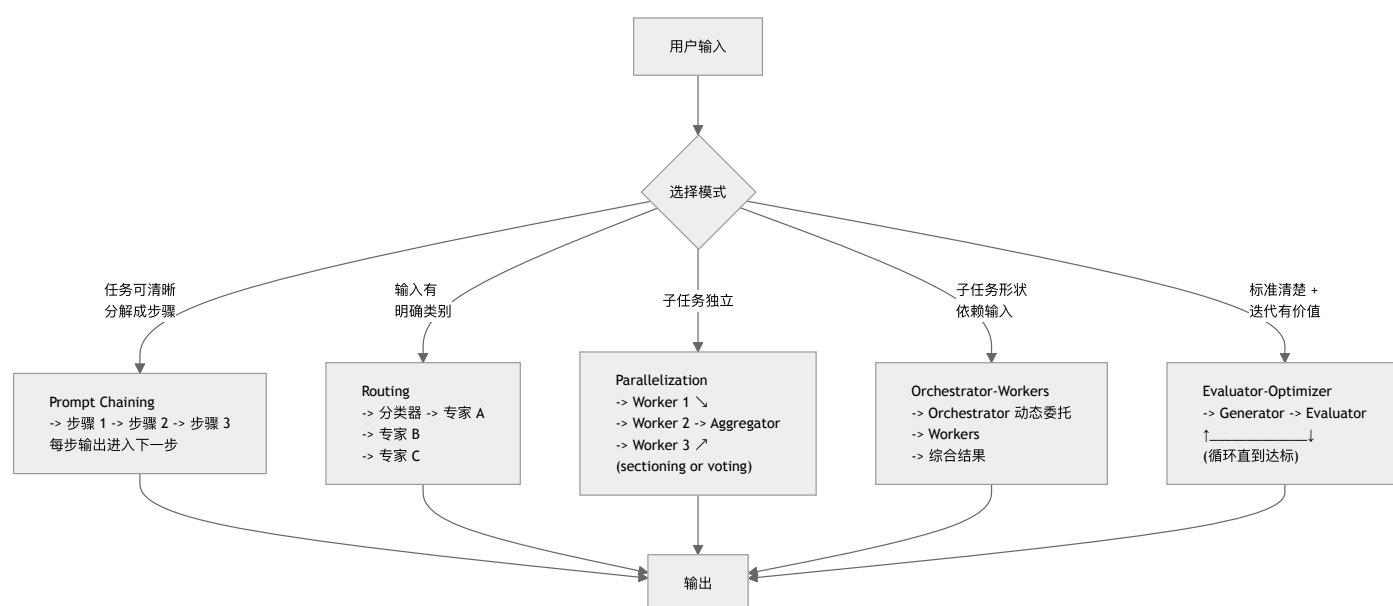
6.6 推理与自我纠错模式

第 6.3 节的五个模式是组合式的控制流模式——它们编排调用。另一条研究线贡献了推理模式：单个 agent（或一个紧密循环）如何组织自身的思考与自我纠错。它们与 workflow 模式相组合，而非取代后者，且大多是值得辨识的具名研究成果。

- **ReAct** 交错推理与行动——模型思考、调用工具、观察、重复。它是大多数 agent 循环之下的基础模式（第 1 章；《LLM Foundations》第 12 章）。
- **Reflexion** 加了一层自我批评的记忆：一次尝试失败后，agent 用自然语言写下为何失败的反思，并把这段反思放进上下文再试一次——一种无需更新权重的“言语强化学习（verbal reinforcement learning）”（Shinn et al. - Reflexion）。
- **Self-Refine** 把 evaluator-optimizer 模式（第 6.3 节）收进单个模型：它生成、批评自己的输出、再修订，反复迭代直到满意（Madaan et al. - Self-Refine）。
- **CRITIC** 把这种批评落地到外部工具——搜索、代码执行、计算器——而非仅靠内省，使纠正被世界检验，而非被模型自己的自信检验（Gou et al. - CRITIC）。
- **Tree of Thoughts (ToT)** 用对多个分支的搜索取代单条推理链，以前瞻与回溯在提交前探索多个部分解（Yao et al. - Tree of Thoughts）。
- **LATS** 通过在 agent 轨迹上运行蒙特卡洛树搜索（MCTS）来统一推理、行动与规划，由 LM 价值函数与反思引导搜索（Zhou et al. - Language Agent Tree Search）。
- **ReWOO** 把规划与执行解耦：Planner 先写出完整计划，Worker 执行工具调用，Solver 合成答案——通过不在每次观察后重新推理来削减 token（Xu et al. - ReWOO）。

harness 视角把它们串起来。每个大多是用 token 和延迟换可靠性，而且——正如第 6.1 节和第 14 章所警告——这笔交易在困难、可检验的任务上才划算，在简单任务上纯属额外开销。更可信的是那些自我纠错被落地到工具或测试的模式（这里的 CRITIC；第 14 章由验证器把关的 cascade；第 7 章的 generator-evaluator 拆分），而非靠模型自我批评，这与本书反复出现的主题一致：验证胜于内省（第 7 章、第 10 章）。

图：五种 workflow 模式



要点

- 从最简单模式开始：许多任务只需要一次 LLM 调用，过早加入 agent loop 往往浪费。
- **Workflow** 给可预测性，**Agent** 给灵活性：依据子任务结构是否预先已知来选择。
- 五种模式覆盖多数场景：chaining、routing、parallelization、orchestrator-workers、evaluator-optimizer。
- 小代理模式今天更容易落地：把 5-10 步聚焦 agent 嵌入确定性 DAG，比“loop until done”更稳。
- 抽象前先保留可见性：直接 API 调用让早期行为更容易检查；模式稳定后 framework 才更划算。
- 推理模式与 workflow 模式相组合：Reflexion、Self-Refine、CRITIC、Tree of Thoughts、LATS 和 ReWOO 组织模型自身的思考——在困难、可检验的任务上最有价值，当自我纠错落地到工具或测试而非内省时最可信。

延伸阅读

- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- Noah Shinn et al., *Reflexion: Language Agents with Verbal Reinforcement Learning*, arXiv, Mar 2023. <https://arxiv.org/abs/2303.11366>
- Aman Madaan et al., *Self-Refine: Iterative Refinement with Self-Feedback*, arXiv, Mar 2023. <https://arxiv.org/abs/2303.17651>
- Zhibin Gou et al., *CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing*, arXiv, May 2023. <https://arxiv.org/abs/2305.11738>
- Shunyu Yao et al., *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*, arXiv, May 2023. <https://arxiv.org/abs/2305.10601>
- Andy Zhou et al., *Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models*, arXiv, Oct 2023. <https://arxiv.org/abs/2310.04406>
- Binfeng Xu et al., *ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models*, arXiv, May 2023. <https://arxiv.org/abs/2305.18323>

第 7 章：长运行代理与多上下文窗口任务

7.1 交接班问题

Anthropic 的“Effective Harnesses for Long-Running Agents”用一个比喻说明核心挑战：想象一个软件项目由轮班工程师完成，每个工程师上班时都不知道前一班发生了什么 ([Anthropic - Effective Harnesses for Long-Running Agents](#))。由于上下文窗口有限，而多数项目都会超过一个窗口，agent 需要桥接 session 的机制。

单靠压缩不总是足够。在 Anthropic 的长运行应用实验中，即使 Claude Agent SDK 有自动压缩，一个简单的 Opus 4.5 loop 也无法可靠地从“build a clone of claude.ai”这样的高层次 prompt 构建生产质量应用。失败集中在两类：agent 试图一次性完成应用，却在实现中途耗尽上下文，留下问题给下一 session 清理；或者前面构建了一些功能后，后续 agent 直接宣称任务完成。

7.2 Initializer + Coding Agent 模式

Anthropic 的解决方案将工作拆成两个角色 ([Anthropic - Effective Harnesses for Long-Running Agents](#)):

Initializer agent 只运行一次，使用专门 prompt，产出：

- 启动开发服务器的 `init.sh`。
- 每个 session 更新的 `claude-progress.txt` 日志。
- 初始 git commit。
- 完整 feature-list 文件（JSON；他们先试过 Markdown，但模型更容易不当编辑它）。在 claude.ai clone 中，列表超过 200 个 feature，每个初始 `passes: false`。

Feature list 中每项是 JSON object，包含 category、description、验证步骤和 `passes` boolean。Coding agent 允许翻转 `passes`，但被强烈告知不允许删除或编辑 feature。

Coding agent 在后续每个 session 运行，使用不同 prompt，要求增量推进。每个 session 从结构化 warm-up 开始：

1. 运行 `pwd` 确认目录。
2. 读取 git log 和 progress file，了解上次做了什么。
3. 读取 feature-list，选择最高优先级未完成 feature。
4. 运行 `init.sh` 启动 dev server，并在实现新功能前跑基本 e2e test。
5. 实现一个 feature。
6. 端到端验证。Anthropic 使用 Puppeteer MCP 做浏览器驱动验证，因为 agent 否则容易在 unit test 通过但实际失败时宣称完成。
7. 用描述性 message commit，并更新 progress file。

这个模式的好处是：agent 被迫在 session 边界进入干净状态，也就是适合 merge 到 main branch 的状态。下一位 agent 不需要先清理上一位留下的问题。

7.3 会话生命周期与干净退出

长运行 harness 需要显式的 session 生命周期：启动、预热、选择有边界的工作、验证、记录证据、干净退出。最后一步很关键。如果一个 session 结束时测试失败、临时文件残留、feature list 没更新、progress note 含糊，下一位 agent 的前几轮就会用来重建状态，而不是推进任务。

因此，干净退出应当成为完成定义的一部分：

- 标准启动路径仍然可用。
- 相关 build、lint、test 或端到端检查已经运行；失败要么修复，要么记录成 blocker。
- 进度工件写清楚改了什么、验证了什么、哪里仍不确定、下一步最好做什么。
- Feature list 或 task list 反映真实状态：没有证据的条目不能标记为 passing。
- 临时调试文件、注释掉的实验代码和过时笔记要删除，或明确隔离。

OpenAI 在 Codex harness 工作中描述了类似的维护循环：agent 生成的系统会复制仓库里已有的模式，所以架构规则和“黄金原则”要写进文档、linter 和周期性清理流程，而不是只依赖偶尔的人类品味 ([OpenAI - Harness Engineering](#))。

Anthropic 的 initializer/coding-agent 模式从 session 角度得出同样的操作结论：下一轮 session 应该能从仓库工件恢复，而不是依赖上一位 agent 的私有记忆。

对更大的项目来说，除了 progress log，还值得维护一份轻量质量文档。Progress log 回答“上一轮发生了什么”；质量文档回答“哪些模块健康、哪些有风险、哪些对 agent 难以理解、哪些缺验证”。这个区分很重要，因为下一位 agent 不只要知道下一个 feature，还要知道代码库哪里正在退化。

7.4 Generator-Evaluator (GAN 启发)

Prithvi Rajasekaran 的后续文章将这个模式扩展到更难的问题：从短 prompt 构建生产质量应用 ([Anthropic - Harness Design for Long-Running Application Development](#))。其动机来自一个观察：当 agent 评估自己的工作时，即使输出普通，也会稳定地偏正面。把做事的 agent 与评判的 agent 分开是强杠杆，因为调一个独立、怀疑性的 evaluator，比让 generator 对自己严厉更容易。

受 GAN 启发，架构包含三个 agent：

- **Planner** 将 1-4 句 prompt 扩展为完整产品 spec，刻意停留在产品/架构层，而不是详细技术设计，以免错误级联，并被鼓励把 AI feature 编进 spec。
- **Generator** 用 React + Vite + FastAPI + SQLite 栈按 feature 实现 spec，并用 git 管理版本。
- **Evaluator** 使用 Playwright MCP 像用户一样点击运行中的应用，测试 UI、API endpoint、数据库状态，然后按产品深度、功能、视觉设计、代码质量 rubric 打分。每个标准都有硬阈值，一个失败就导致 sprint 失败，并给出详细反馈。

两者通过 *sprint contracts* 协调：每个 sprint 前，generator 提出要构建什么以及如何验证成功；evaluator 审查直到双方同意；generator 再按合同构建。通信基于文件：一个 agent 写文件，另一个读取并回应。

成本很高。对于“create a 2D retro game maker” prompt，solo run 花 20 分钟、\$9，产出一个看起来像样但游戏本身不能工作的应用；实体出现在屏幕上，却不响应输入。完整 harness 花 6 小时、\$200，产出带 sprite editor、level editor、AI-assisted level generation 和 playable mode 的可用应用。超过 20 倍成本买到的是能工作的产品，而不是破损 stub。

7.5 自验证是头号杠杆

LangChain 的 Top-30-to-Top-5 案例研究从另一条路得到相同结论 ([LangChain - Improving Deep Agents with Harness Engineering](#))。通过 trace 分析，他们识别出最常见的失败模式：agent 写了方案，重读自己的代码，觉得看起来没问题，然后停止。他们在系统提示中加入结构化指导：Plan、Build with verification in mind、Verify by running tests and comparing output to spec、Fix；并加入 `PreCompletionChecklistMiddleware`，在 agent 退出前强制验证。

这个模式呼应社区流传的“Ralph Wiggum loop”：一个 hook 拦截 agent 的退出尝试，并在干净上下文窗口中重新注入原始 prompt，迫使 agent 继续对照目标工作 ([LangChain - The Anatomy of an Agent Harness](#))。

LangChain 的组合改动，包括上下文 middleware 映射 cwd 和工具、build-verify 指导、loop detection，以及 high-low-high reasoning compute 的“reasoning sandwich”，在不换模型的情况下把分数提高 13.7 点，从 52.8% 到 66.5%。

7.6 Context Reset 与 Compaction

Anthropic 的 harness-design 后续文章明确区分两者 ([Anthropic - Harness Design for Long-Running Application Development](#))。Compaction 是在同一对话中总结早期部分，同一个 agent 带着缩短历史继续。Context reset 则清空上下文，启动一个新 agent，用结构化 handoff 传递上一 agent 的状态和下一步。

两者解决不同问题。Compaction 保持连续性。Reset 用来缓解“context anxiety”：Anthropic 在 Sonnet 4.5 中观察到，agent 在接近它认为的上下文上限时，会过早收尾。Reset 给 agent 一个干净开局；代价是 handoff artifact 必须携带足够状态，让下一 agent 干净恢复。

当 Opus 4.5 基本自行修复 context-anxiety 行为后，Anthropic 能够完全从 harness 中删除 context reset。这是第 12 章 model-harness coupling 的明确例子。

7.7 Managed Agents：解耦 Brain、Hands 与 Session State

OpenReview 综述强调了一个平台化方向，Anthropic 后续称为 managed agents：把模型侧的 **brain**、执行侧的 **hands**，以及持久 **session/event log** 分开 ([OpenReview - Agent Harness Engineering: A Survey](#))。Brain 决定应该发生什么；hands 在可替换环境中运行 shell、编辑文件、浏览和调用外部服务；session log 记录足以重建任一侧的状态。

这个拆分对长运行任务很重要：

- 如果模型上下文耗尽，可以用 event log 和仓库工件启动新的 brain 继续。
- 如果 sandbox 损坏、超时或被攻陷，可以从干净镜像重建 hands。
- 如果操作需要凭据，proxy 和 vault 可以在边界处附加凭据，而不是把 secret 放进 sandbox。
- 如果部署在 agent 运行中发生变化，平台可以在渐进 handoff 中同时保留新旧 worker 版本。

这样看，context reset 只是恢复机制之一。生产级长运行 harness 还需要环境重置、凭据隔离、可恢复 event log，以及 in-flight session 的迁移规则。

7.8 多代理研究系统

对具有并行结构的任务，例如有许多独立线索要探索的研究，第 6 章的 orchestrator-worker 模式适用。Anthropic 的研究功能用 Claude Opus 4 做 lead agent，用 Claude Sonnet 4 做 sub-agents ([Anthropic - How We Built Our Multi-Agent Research System](#))。Lead 分析查询、制定策略、派生并行 sub-agents；每个 sub-agent 搜索并返回浓缩发现；lead 综合；citation agent 再为论断标注来源。

他们经验中出现八条 prompt engineering 原则：

1. **像 agent 一样思考**：在 Console 中用完全相同工具模拟 prompt，观察逐步行为。
2. **教 orchestrator 如何委托**：给 sub-agent 目标、输出格式、工具指导和明确边界；模糊委托会重复或误解。
3. **按查询复杂度缩放努力**：prompt 中明确规则，例如事实查找 1 个 agent / 3-10 次调用；比较任务 2-4 个 sub-agent / 每个 10-15 次调用；复杂研究 10+ sub-agent，防止过度投入。
4. **工具设计与选择很关键**：明确启发式，例如先检查所有可用工具、按用户意图匹配工具、优先专用工具而非通用工具。
5. **让 agent 改进自己**：tool-testing agent 使用有缺陷 MCP 工具、观察失败并重写描述，使后续任务完成时间下降 40%。
6. **先宽后窄**：提示 agent 从短而宽的查询开始，再逐步细化；自然倾向往往相反。
7. **引导思考过程**：extended thinking 可作为可控 scratchpad；interleaved thinking 帮助 sub-agent 在工具调用之间评估质量和细化查询。
8. **并行工具调用改变速度**：并行启动 sub-agents，并让 sub-agent 并行调用多个工具，在复杂查询中可将研究时间最多减少 90%。

7.9 有状态 Agent 的生产可靠性

Anthropic 的研究系统文章记录了 agent 长时间运行后的工程挑战 ([Anthropic - How We Built Our Multi-Agent Research System](#))：

- **错误会复合**：没有 checkpoint-and-resume 基础设施，小系统故障也可能灾难化。Anthropic 结合 AI 适应性（让 agent 知道工具故障并信任它调整）与 retry logic、定期 checkpoint 等确定性 safeguard。
- **调试需要新工具**：agent 每次运行非确定，完整生产 tracing 是主要诊断表面，用于观察决策模式和交互结构，而不必阅读对话内容。
- **部署需要协调**：当许多 agent 正在运行时推出代码变更，需要 *rainbow deployments*，逐步把流量从旧版本切到新版本，并同时保持两者存活。
- **同步执行制造瓶颈**：当前架构中，lead agent 等待 sub-agents 完成后再继续，简化协调但受最慢 sub-agent 阻塞。异步执行可释放更多并行性，但带来结果协调、状态一致性、错误传播挑战。

7.10 持久化执行：Checkpoint、Replay 与恢复

第 7.9 节指出错误会复合，而 Anthropic 的做法是把 AI 适应性与确定性的 checkpoint、retry 相结合。*持久化执行 (durable execution)* 正是将这一思路一般化的系统工程学科。持久化执行引擎把工作流的每一步都持久化到日志，使得进程一旦死掉——机器故障、超时、部署、上下文窗口耗尽——就从最后记录的一步恢复，而不是从零重启。这个想法早于 agent；它正是 Temporal、DBOS 等工作流引擎提供容错的方式 ([Temporal - Durable Execution Meets AI](#))。它直接对应第 7.7 节的受管代理拆分：持久事件日志，正是让全新 brain 在旧 brain 消失后得以恢复的凭借。

对 agent 而言，持久化的单位是 agent 的状态——它的上下文、工具调用结果，以及它在计划中的位置。LangGraph 把这暴露为一个 *checkpointer*，在每个 super-step 保存 graph 状态，从而支持失败后恢复、human-in-the-loop 暂停，乃至时间旅行回到先前状态，并提供可选的持久化模式，在性能与崩溃可能损失的工作量之间权衡 ([LangChain - Durable Execution](#))。

设计上的张力是 *确定性 vs. 模型*。基于 replay 的持久化假设某一步可以被重新执行以复现其效果，但模型调用与工具结果是非确定的——重跑它们会偏离已记录的历史。标准解法是把模型和工具调用当作有副作用的 *activity*，其结果只记录一次，之后从日志中 replay 而非重新计算。这正是 ReWOO（第 6.6 节）与 context reset（第 7.6 节）背后“记录观察，而不重算它”的同一逻辑，如今被做成了一种基础设施保证。

如此看来，持久化执行把第 7.3 节“干净退出、可从产物恢复”的纪律，从一个 agent 必须自觉记住的约定，变成了平台强制的性质。它也约束成本——任务中途崩溃，不必把此前花掉的 token 从头重付一遍——并且它也是第 17 章所需回滚能力的底座：当某次 harness 变更在生产中行为异常时，回滚正是建立在它之上。

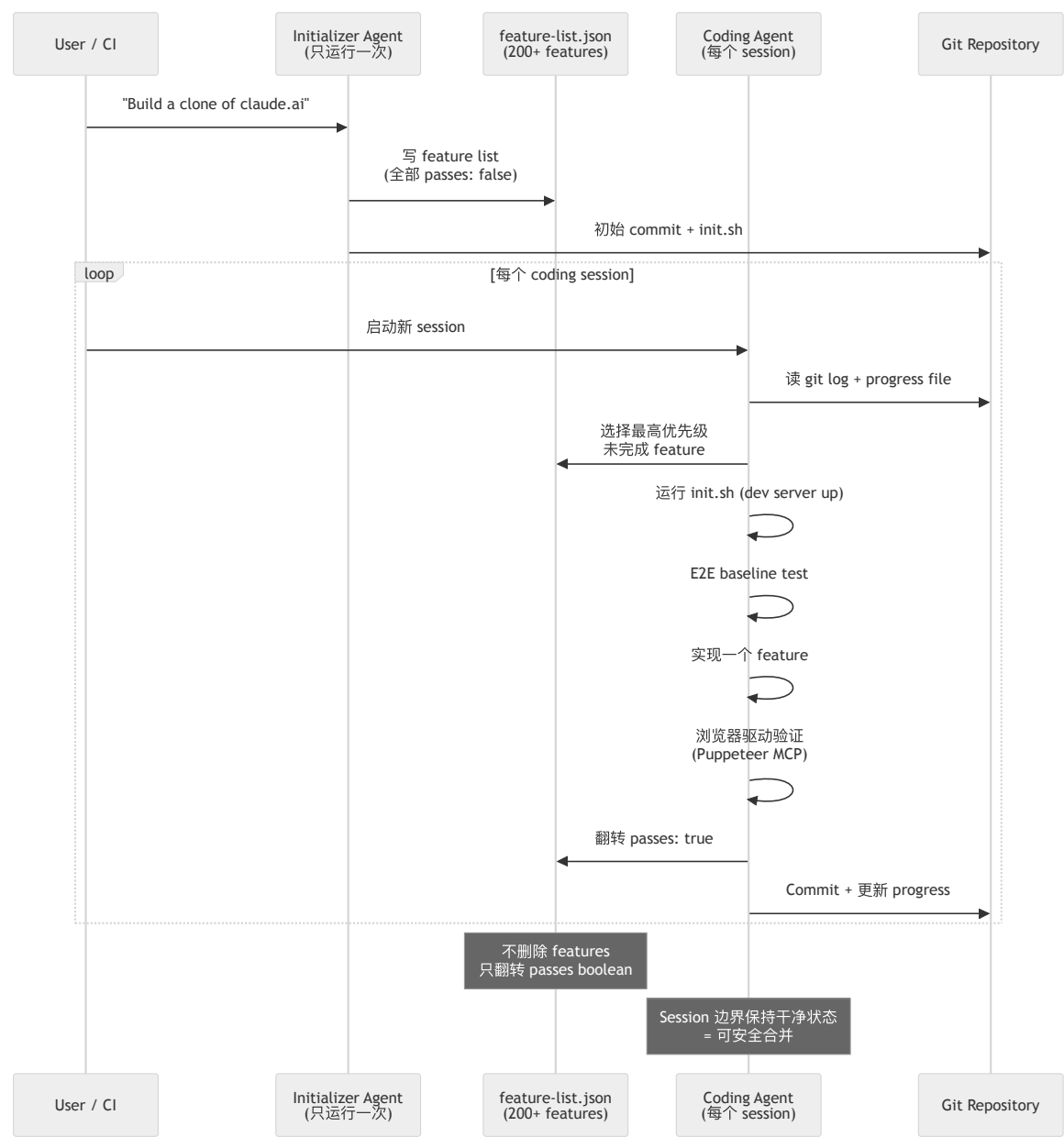
7.11 “长”到底有多长？时间视野指标

本章讲的是超出单个上下文窗口的任务，但“长”值得有个度量。METR 提出了一个：一个模型的 *时间视野 (time horizon)* 是它以 50% 可靠性能完成的任务长度——以人类完成该任务所需时间来衡量。一个“50 分钟时间视野”的模

型，在需要人类约五十分钟的任务上有一半时间能成功。在 2019 至 2025 年的前沿模型上测量，这个视野大约每七个月翻一番 (Kwa et al. - Measuring AI Ability to Complete Long Tasks; METR)。

对本章有两点意义。第一，这个视野是模型加其 harness 的性质，而非模型自身的性质：本章的 handoff、checkpoint、自我验证机制，正是 harness 把有效视野拉长到超出裸模型自身所能维持的手段——这是第 12 章模型-harness 耦合从能力侧看到的样子。第二，该指标重新框定了何时值得构建本章这套机制。随着内在视野增长，一些脚手架变得不再必要——Anthropic 随模型进步先后去掉了 context reset 与 sprint 分解（第 7.6 节、第 12 章）——但有趣的长视野任务前沿也随之外移。harness 的工作会迁移到更难的问题，而非消失（第 18 章）。

图：Initializer Agent -> Feature List -> Coding Agent Sessions



要点

- 交接班问题是根本性的：上下文限制意味着 agent 需要结构化 handoff，而不是只依赖更大窗口。
- Initializer + coding agent 是有用的长周期模式：规划与增量执行分角色。
- 干净退出是完成定义的一部分：每轮 session 都应留下可用启动路径、更新后的状态工件、验证证据，以及不会拖累下一轮的工作区。
- Generator 与 evaluator 分离是强杠杆：agent 对自己输出偏正面，独立 evaluator 更可靠。
- Sprint contracts 协调多 agent 工作：构建前用文件沟通并约定成功标准。
- Context reset 可以缓解 context anxiety：有时带结构化 handoff 的全新开始优于压缩。
- Managed agents 解耦 brain、hands 和状态：模型上下文、沙箱执行、凭据和 event log 应能独立失败并恢复。

- 自验证是头号杠杆：退出前强制验证，在不换模型的情况下提升 13.7 分。
- 持久化执行把可恢复性变成基础设施保证：把每一步持久化到日志，使全新 agent 能在崩溃、上下文耗尽或部署后恢复——把非确定的模型/工具调用当作已记录的副作用，而非要重算的步骤。
- 时间视野度量“多长”：METR 的任务完成视野（模型有 50% 时间能完成的人类任务长度）大约每七个月翻一番——而它是模型加 harness 的性质，这正是本章机制能拉长它的原因。

延伸阅读

- Justin Young et al., *Effective Harnesses for Long-Running Agents*, Anthropic, Nov 2025.
<https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>
- Prithvi Rajasekaran, *Harness Design for Long-Running Application Development*, Anthropic, Mar 2026.
<https://www.anthropic.com/engineering/harness-design-long-running-apps>
- Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026.
<https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
- Jeremy Hadfield et al., *How We Built Our Multi-Agent Research System*, Anthropic, Jun 2025.
<https://www.anthropic.com/engineering/multi-agent-research-system>
- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- OpenAI, *Harness Engineering: Leveraging Codex in an Agent-First World*, Feb 2026.
<https://openai.com/index/harness-engineering/>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>
- Temporal, *Durable Execution Meets AI: Why Temporal Is the Perfect Foundation for AI*, 2025.
<https://temporal.io/blog/durable-execution-meets-ai-why-temporal-is-the-perfect-foundation-for-ai>
- LangChain, *Durable Execution* (LangGraph documentation), 2025.
<https://docs.langchain.com/oss/python/langgraph/durable-execution>
- Thomas Kwa et al., *Measuring AI Ability to Complete Long Tasks*, METR / arXiv, Mar 2025.
<https://arxiv.org/abs/2503.14499>

第 8 章：Loop Engineering（循环工程）

Agent = Model + Harness。第 1 章把 agent loop 介绍为每个 agent 中心的那个循环——组装上下文、模型发出工具调用、harness 执行、把观察追加回去、重复。前面几章工程化的是这个循环的*内部*：模型看到什么、有哪些工具、在哪里运行、如何跨 session 恢复。本章讲的是把这个循环本身当作首要的工作单元去工程化——决定什么启动它、每一趟做什么、谁检查结果、何时停止。2026 年，这个实践有了名字：*loop engineering*，以及一句口号：别再 prompt 那个 agent，去构建那个 prompt 它的系统。

8.1 从 Prompting 到 Looping

1.6 节勾勒了一条脉络——prompt engineering 让位于 context engineering，而后者又位于 harness engineering 之下。Loop engineering 是同一条路上的下一站，也是让这个转变对实践者变得具体的那一站。Addy Osmani 在 2026 年 6 月的文章《Loop Engineering》中给这个模式命了名，提供了它的经典解剖和如今流传的词汇 ([Addy Osmani — Loop Engineering](#))。同一周，Peter Steinberger 把它压缩成一句话，一天之内触达数百万人：你不该再 prompt coding agent 了，你该设计那些 prompt 你的 agent 的 loop ([O'Reilly Radar — Loop Engineering](#))。构建了 Claude Code 的 Boris Cherny 给出了实践者版本的直白说法：他不再一回合一回合地 prompt 模型——他的工作是写驱动它的 loop ([The New Stack — Loop Engineering](#))。

这个重构很小，却是承重的。在 prompt engineering 里，人在循环内，每一步之间按回车、评判每个结果。Loop engineering 把人从这个内部位置移出，并提出一个更难的问题：如果你不在场去决定工作是否够好、下一步做什么，那么什么来决定？本章的一切都是对这个问题的回答。相对第 1 章的机制，这里没有任何新东西——还是那个 agent loop——但重心从模型的回合转向了它周围的控制结构，而那正是 harness 的地盘。Loop engineering 和 harness engineering 是紧密相关工作的两个名字；loop engineering 是运维者的说法，聚焦在那个跨长周期调度、验证并约束 agent 的外层循环上。

8.2 一个 loop 就是带 check 的 task

那本 field guide 的一句话定义是恰当的锚点：一个 loop 就是带 check 的 task，而不带 check 的 task 只是一厢情愿 ([The Agentic Loop — A Practical Field Guide](#))。展开来说，一个良构 loop 的一趟是：观察当前状态、采取一个有界动作、把结果对照一个固定标准做检查、决定继续还是停止。这个结构暴露出四个设计杠杆，而 loop engineering 在很大程度上就是把它们规定好的工作：

- **Trigger（触发器）**——什么启动一趟：一个人给的目标、一个 schedule、一个 webhook，或另一个 agent。
- **Topology（拓扑）**——loop 如何嵌套与交接：单个 agent、一个 maker 加一个 checker，或一个 orchestrator 管一群 worker。
- **Verifier（验证器）**——决定“够好了”的那个固定标准，以及由谁来施加它。
- **Stop rules（停止规则）**——loop 成功、放弃或求助的明确条件。

任一个留空，失败都是可预测的。没有 trigger，loop 只是一次对话。没有 verifier，它会在垃圾上宣布胜利。没有 stop rule，它会永远跑下去——或者跑到账单来为止。本章余下部分依次处理这四个杠杆。

8.3 Trigger 与嵌套的 loop

Trigger 是把一个 agent 从“你去调用它”提升为“它自己会跑”的那个东西。Osmani 的解剖把它叫作 *heartbeat*（心跳）——一个无需人类 prompt 就唤醒 loop 的 schedule 或事件 ([Addy Osmani — Loop Engineering](#))。正是在这里，coding agent 从一个编辑器变成一个运维关切：cron 每晚触发它、webhook 在新 issue 上触发它，或一个监督 agent 把它作为子任务触发。

Andrew Ng 通过观察这些 loop 会嵌套、且各有不同的 owner 和时间尺度，给出了这个拓扑最清晰的地图 ([Andrew Ng — The Batch, 2026 年 6 月](#))：

- **Agentic coding loop** 以分钟计：给定一份 spec 和一些 evals，agent 写代码、测试、迭代，直到满足 spec——回合之间没有人。
- **Developer feedback loop** 以小时计：一个人检视已构建的东西，把 agent 引向下一步该做什么。
- **External feedback loop** 以天计：alpha 测试者、A/B 测试、生产信号。

内层 loop 是 loop engineering 自动化得最激进的那一个；外层 loop 则是人类判断保持不可替代的地方，因为人握有 agent 缺乏的上下文优势——对意图、对产品到底为何而做的了解。Ng 自己的例子：一个 coding agent 无人值守地工作了约一个小时，其间在浏览器里检查了好几次自己做的东西，然后才回来要方向。设计目标就是让每个 loop 在必须上交给出下一层之前，尽可能长地高效运行。

8.4 Verifier 才是瓶颈

四个杠杆里，verifier 是 loop engineering 投入精力最集中的地方，因为它是让无人值守运行变得安全的东西。第 7 章从另一个方向确立了那条头号结论：agent 对自己的工作偏正面，所以把做工作的 agent 和评判它的 agent 分开是一个强杠杆。Loop engineering 把这个观察提升为一条设计法则——maker 不能是 checker ([Loop Engineering Crash Course](#))。一个独立的 reviewer agent，拿到的是 spec 而非 diff，并被要求带着怀疑，能抓住 generator 自圆其说糊弄过去的东西。这就是 7.4 节的 generator-evaluator 拆分和第 6 章的 evaluator-optimizer workflow，从一种技术被提升为决定这个 loop 能不能被放手的那个东西。

社区的口号点出了杠杆的转移：写 verifier 是新的 prompt engineering ([The Agentic Loop — A Practical Field Guide](#))。稀缺、有价值的动作不再是措辞请求，而是把“done”定义得足够精确，好让机器能检查它。由此引出一个有用的区分：

- **Closed loop (闭环)** 预先把验收标准钉成硬的、可检查的通过项——测试全绿、schema 校验通过、截图匹配。它在可预测的预算上运行，放着跑是安全的。
- **Open loop (开环)** 朝一个模糊目标松散地探索。它需要一个更强的 verifier，因为没有它，它不会大声失败——它会成功地、自信地、成百上千次地产出看似合理的垃圾。

Verifier 应尽任务所允许地机械：一个测试、一次类型检查、一次 schema 校验、一个浏览器断言，最后才是对无法确定性检查之物用 LLM-as-judge——这就是第 5 章的“计算型先于推断型”排序，以及第 10 章的 grader 分类。最强的形态是用一个全新的模型，它对工作是如何产出的毫无记忆，因而不会继承 maker 的盲点。

8.5 Stop rule 与三个硬停

一个能自己启动的 loop，也必须能自己停下——且理由不止“成功”。每个良构 loop 都需要明确的停止条件——success、no-op（没剩下什么可做）、ask-for-approval、blocked-or-exhausted（受阻或耗尽）——其中三个是用来兜住失控的、不可商量的硬停 ([The Agentic Loop — A Practical Field Guide](#))：

1. **最大迭代次数**——一个硬上限，比如“测试全绿，或六轮，先到先算”。
2. **无进展检测**——若 N 趟都没产生可测量的变化，就停下，而不是空转。
3. **预算上限**——一个 token 或美元上限，越过它 loop 就停下来问。

这就是从 loop 内部看到的第 17 章的 per-task budget，而它在这里承重的原因是：人已经不在场去注意那种空转。失败模式是具体的：Uber 的一个团队把 agent 花费上限设为每月 \$1,500，此前一个无人值守的设置四个月里烧光了它的年度 AI 预算 ([AI Builder Club — Loop Engineering Guide](#))。Ask-for-approval 这个停是通往第 15 章的桥：把升级建模为一次工具调用，能让 loop 挂起、把一个决定交给人类、并在人回应时从 event log 恢复——同一个持久 approval 模式，如今成了 loop 面对任何有后果之事的指定出口。

8.6 Ralph 谱系

Loop engineering 不是凭空冒出来的；它是这个领域自 2022 年以来一直在爬的那把梯子的当前一级 ([The Agentic Loop — A Practical Field Guide](#))：

- **ReAct** (2022) 确立了 reason-act-observe 的基本循环，也就是第 1 章所说的 agent loop ([Yao et al. — ReAct](#))。
- **AutoGPT** (2023) 让它变得目标驱动、自主——并暴露了这门学科如今所防范的那个失败：一个没有 verifier、没有 stop rule 的 loop，会永远跑下去或跑偏。
- **“Ralph Wiggum” loop** (2025) 加入了 7.5 节所见的关键修法：每次迭代 reset 到干净的 context window，把状态锚在磁盘上的文件里而非膨胀的历史里，并重新注入目标，逼 agent 持续对照它工作。
- **可验证完成命令** (2026)，例如把退出交给一个独立 validator 模型把关的 `/goal`，把 stop rule 变成一等的、机器检查的步骤，而不是 agent 自己的意见。
- **Orchestration** (当前) 让 loop 监督 loop——被调度、以 git 为后盾，在 Ng 的嵌套时间尺度间上下交接工作。

有两条主线把这个谱系和本书其余部分连起来。Ralph 的 reset 正是记忆活在磁盘上、而非上下文里（第 2-3 章）的原因：一个每趟都 reset 的 loop，必须从文件重新加载状态，而那正是那几章所规定的结构化笔记和反复复述。可验证完成正是 event log 之所以重要（第 9 章）的原因：一个状态是追加式 log 的 loop，能停止、恢复、被重放，而这正是让长周期自治可调试的东西。

8.7 一个 loop 由什么构成

Osmani 的解剖列出了一个持久 loop 装配起来的各个部件，而每一个都对应到前面各章已经建好的一项能力 ([Addy Osmani — Loop Engineering](#))：

- **Heartbeat**——触发一趟的 schedule 或事件（8.3 节）。
- **Worktrees**——隔离的工作目录，让并行的趟不相撞，借用第 5 章的 sandbox 隔离。
- **Skills**——可复用、由文件支撑、写一次并按需加载的项目知识，即第 4 章的 `SKILL.md` 模式。

- **Connectors**——触达工作所依赖的真实工具的 MCP server 和插件（第 4 章）。
- **Sub-agents**——作为分开角色的 maker 和 checker（8.4 节、第 3 章）。
- **Spine（脊柱）**——一个在多次运行间存活、跨 reset 携带 loop 记忆的持久状态文件（第 2–3 章，以及 7.6 节的结构化 handoff）。

Loop 的好坏，取决于它所对着跑的那个代码库，这也是为什么实践者描述一个仓库要“loop-ready”需具备的三项属性 ([AI Builder Club — Loop Engineering Guide](#))。它必须**可读（legible）**——一份精简的 AGENTS.md 索引和定制 lint，让 agent 知道代码的形状以及什么不该碰。它必须**可执行（executable）**——一个以接近零 token 成本起来、并容忍并行 worktree 的 dev server。它还必须**可验证（verifiable）**——对核心流程的 end-to-end 测试和浏览器驱动检查，好让 8.4 节的 verifier 有机械可断言之物。这些就是第 5 章的 *ambient affordances*，如今是自治的前提，而不再是锦上添花。

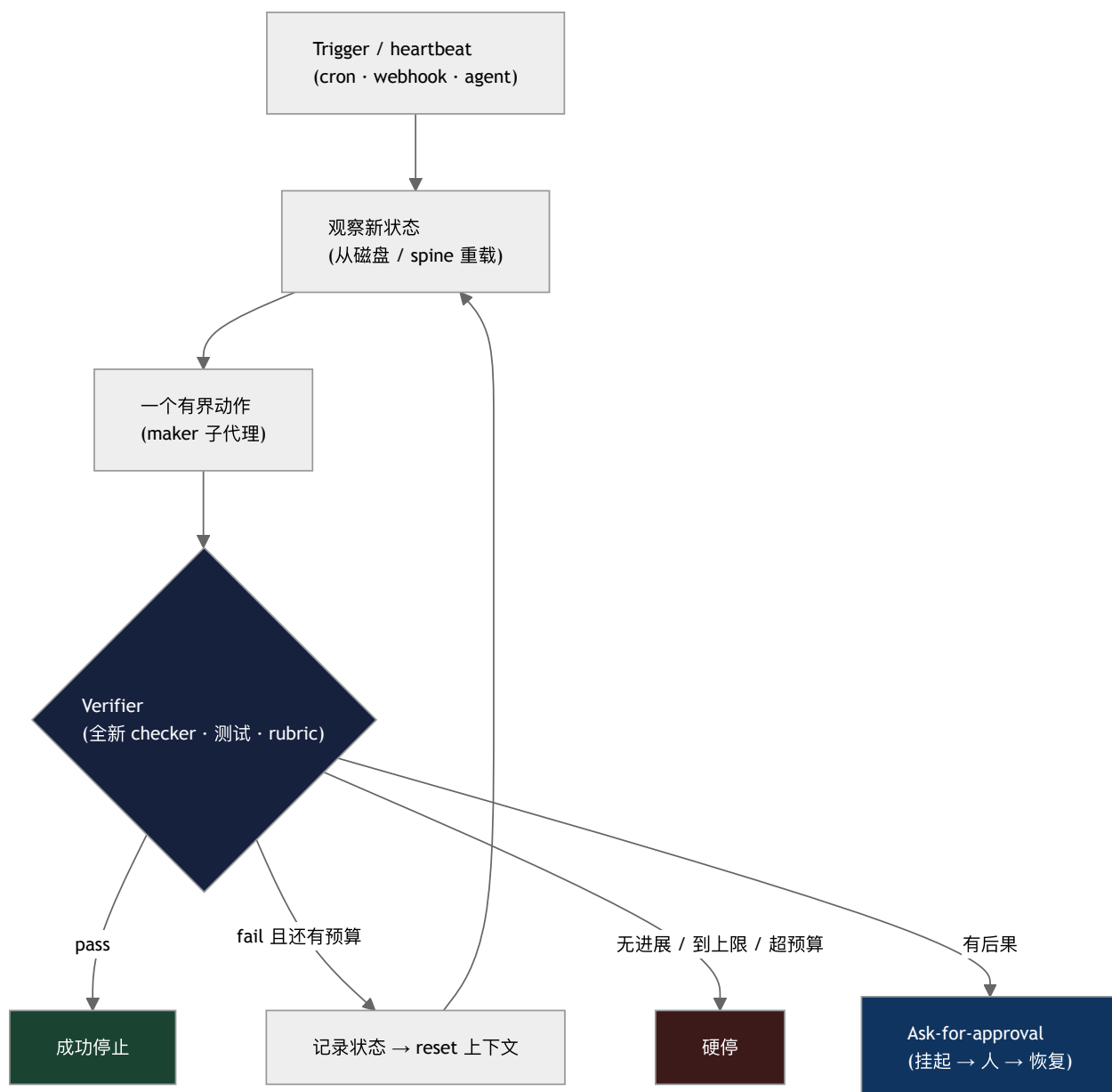
8.8 成熟度阶梯与无人值守的风险

因为 loop 会把它做的一切复利放大，采用应当分级。社区的成熟度阶梯一次只爬一级，且只在当前一级已经产出你本来会亲手做的工作时才往上爬 ([The Agentic Loop — A Practical Field Guide](#)):

0. **Manual（手动）**——你每回合都 prompt。
1. **Triage（分诊）**——loop 把发现写进一个 markdown 文件，什么都不改。
2. **Draft（草稿）**——它在隔离分支上做修复。
3. **Verified PR（已验证 PR）**——一个独立 verifier 在人 review 之前先把关。
4. **Auto-merge（自动合并）**——只留给低风险类别。

这份纪律之所以存在，是因为 loop engineering 并没有消除那个难题；它只是把它挪了位置。速度把成与败一起复利放大：一个无人值守跑着的 loop，也是一个无人值守犯错的 loop，而它 ship 代码可以比人读代码更快，从而累积 *comprehension debt（理解债）*——一个其主人不再完全理解的代码库 ([The New Stack — Loop Engineering](#))。验证与问责仍归人所有，在两个不可约的端点上：定义什么叫“好”的那个意图，以及对 ship 出去之物的所有权 ([Loop Engineering Crash Course](#))。由此得到一条界定范围的规则：只对那些重复、无人值守、被调度、有后果的工作动用结构化 loop。对任何你正交互式盯着看的东西，check 就是你自己的眼睛，一次普通对话是更好的工具——为一次性任务过度工程化一个 loop，本身就是一种失败模式。

图示：被工程化的 loop



本章要点

- **Loop engineering** 是 **harness engineering** 的外层循环视角：别再一回合一回合地 prompt agent，去设计那个 prompt 它的系统，并规定由什么来决定工作何时够好。
- 一个 **loop** 就是带 **check** 的 **task**：观察 → 一个有界动作 → 对照固定标准验证 → 决定继续或停止。不带 **check** 的 task 只是一厢情愿。
- 四个杠杆定义一个 **loop**：trigger、topology、verifier、stop rules。任一个留空，失败都可预测。
- **loop** 会嵌套：agentic coding 以分钟计、developer feedback 以小时计、external feedback 以天计——自动化内层，把人类判断留在外层。
- **verifier** 才是瓶颈：写它是新的 prompt engineering，maker 不能是 checker，弱 check 会以 ship 出自信的垃圾这种方式无声失败。
- **stop rule** 是必需的：最大迭代次数、无进展检测、预算上限——因为没有人会在场去注意失控。
- **Ralph** 谱系是它的血统：ReAct → AutoGPT → 干净上下文 reset → 可验证完成 → orchestration；记忆活在磁盘上，状态活在 event log 里。
- 慢慢爬成熟度阶梯：triage 先于 draft、draft 先于 auto-merge，且只用于重复、无人值守、有后果的工作——无人值守的速度会把错误和理解债一起复利放大。

延伸阅读

- Addy Osmani, *Loop Engineering*, addyosmani.com, 2026 年 6 月。 <https://addyosmani.com/blog/loop-engineering/>
- *Loop Engineering*, O'Reilly Radar, 2026。 <https://www.oreilly.com/radar/loop-engineering/>
- Andrew Ng, *Three Loops for Building 0-to-1 AI Products*, The Batch, 2026 年 6 月。
<https://www.deeplearning.ai/the-batch/>
- *The Anthropic leader who built Claude Code ditched prompting — now he writes loops*, The New Stack, 2026。
<https://thenewstack.io/loop-engineering/>
- *The Agentic Loop: A Practical Field Guide*, DEV Community, 2026。 <https://dev.to/truongpx396/the-agentic-loop-a-practical-field-guide-mnc>
- *Loop Engineering Guide (2026)*, AI Builder Club。 <https://www.aibuilderclub.com/blog/loop-engineering-guide-2026>
- Shunyu Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, arXiv, 2022 年 10 月。
<https://arxiv.org/abs/2210.03629>

第 9 章：生产级 Agent 的十二要素

前几章讨论了单个 harness 技术：上下文管理、工具、沙箱、工作流和长运行 handoff。本章退一步看：HumanLayer 的“12 Factor Agents”更适合作为一份生产 checklist，把这些技术重新连接到普通软件架构。它借用了经典 Twelve-Factor App 的命名风格，但要素本身针对的是 LLM agent——而且它是一份宣言，而不是完整参考架构。

本章里有两个软件概念很关键。状态是继续执行所需的信息：当前步骤、重试次数、审批、用户消息、工具结果，以及目前已经触及的业务对象。事件日志是可以用来重建这些状态的追加式记录。把 agent 建模为对事件的 reducer 后，暂停/恢复、重放、调试和测试就会变成普通软件问题，而不是隐藏在对话里的状态问题。

9.1 作为软件架构的十二要素

这十二条原则来自许多生产部署 (HumanLayer - 12-Factor Agents)：

1. **Natural Language to Tool Calls**：原子模式是把用户自然语言转换成结构化 JSON call，再由确定性代码执行。
2. **Own Your Prompts**：不要把 prompt engineering 外包给 framework 黑箱。把 prompt 当作一等代码，使其可测试、可评估、可调优。
3. **Own Your Context Window**：标准 message format 只是一个选择；把 history 压进单个 XML-tagged event log 也是一种选择。目标是最大信息密度、最小 token。
4. **Tools Are Just Structured Outputs**：工具调用就是模型输出 JSON，命名意图与参数。确定性代码决定如何执行。
5. **Unify Execution State and Business State**：不要把“当前步骤/下一步骤/retry count”与“对话中发生了什么”分开。应从单一 event log 推导执行状态。
6. **Launch / Pause / Resume with Simple APIs**：agent 是程序，应支持标准 lifecycle，包括在工具选择和执行之间暂停。
7. **Contact Humans with Tool Calls**：不要依赖模型选择普通文本还是结构化输出；给它明确的 `request_human_input` 工具，并带 urgency、format、choices 等结构化选项。
8. **Own Your Control Flow**：接管 loop，用于审批中断、总结工具结果、对输出运行 LLM-as-judge、管理记忆、记录 trace、限流、durable sleep。
9. **Compact Errors into Context Window**：让错误可见才能 self-heal；用连续错误计数器在阈值后升级给人类。之所以用 compact 这个动词，是因为不断堆积原始 stack trace 会撑爆 token 预算、加剧 context rot（见《LLM Foundations》第 9 章）——应在 context 中保留最新的或经过摘要的错误，把更早的重复 trace 丢弃或折叠，使错误历史不会挤占 working context。
10. **Small, Focused Agents**：把单个 agent 范围控制在 3-10 步，最多也许 20 步。上下文越大，性能越差。
11. **Trigger from Anywhere**：允许从 Slack、email、SMS、webhook、cron 启动。结合 factor 7，可以形成 *outer loop*：由事件启动的 agent，在关键点联系人类求助。
12. **Make Your Agent a Stateless Reducer**：agent 是对 events 的 fold。纯、可序列化、可重放。只有当每一次 LLM 响应和工具结果都作为 event 被记录下来时，重放才是确定性的：恢复时你对已记录的结果做 fold，而不是重新调用模型、也不是重新执行有副作用的工具。

贯穿这些要素的深层主张是：“好的 agent 至少不是‘给你一个 prompt、一袋工具，循环直到目标完成’这种模式。它们大多只是软件” (HumanLayer - 12-Factor Agents)。这些要素基本是在把软件工程卫生应用到一个有状态、非确定性的组件上。它们不应被当成普遍法则：研究原型、本地 coding assistant、受监管客服 agent 会需要不同权衡。真正有用的方向是：让状态显式、让控制流可检查、把人类交互放在结构化接口后面。

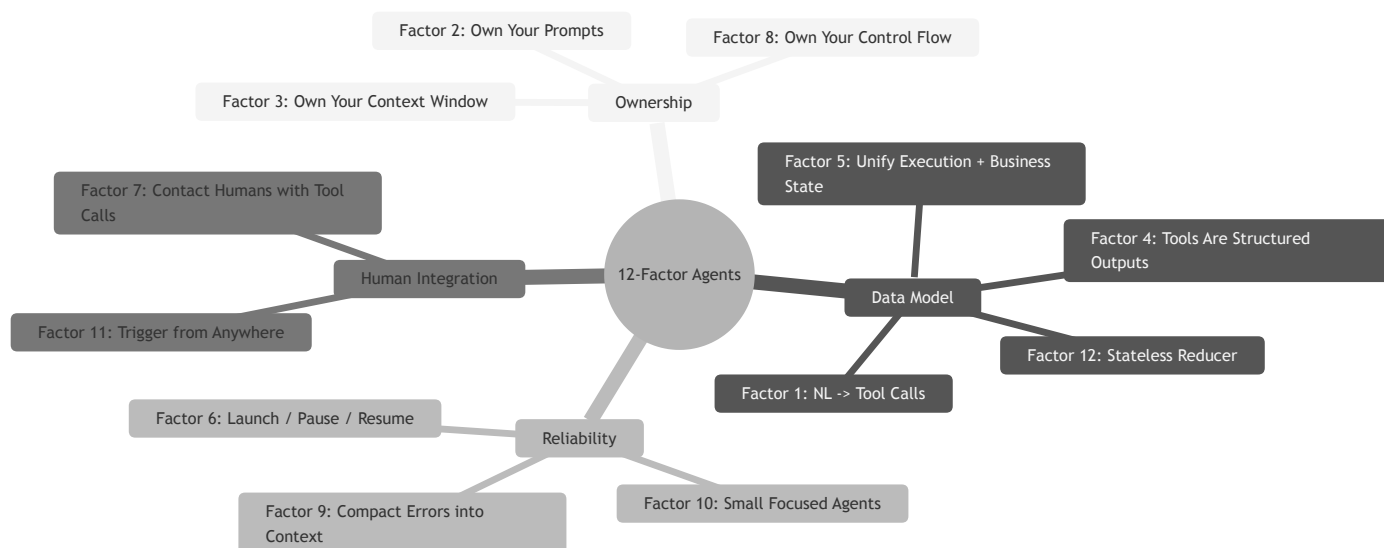
9.2 从 Agent Program 到 Agent Platform

OpenReview 综述认为，生态正在从 agent framework 走向 agent platform (OpenReview - Agent Harness Engineering: A Survey)。Framework 打包的是本地抽象：agents、tools、memory stores 和 loops。Platform 额外提供跨多次运行、多用户的 durable workspace、managed sandbox、identity、billing、observability、evaluation、governance 和 human handoff。

这不会替代十二要素，而是扩大它们的范围。Launch / pause / resume 变成平台 API。Unify execution state and business state 变成带 tenancy 和 migration 语义的 event-log storage。Contact humans with tool calls 变成带 audit 和 permission state 的 handoff interface。Own your control flow 变成明确决定哪些检查同步运行、哪些离线运行、哪些失败值得触发昂贵恢复。

平台边界也改变责任。本地 agent 可以用临时 state files；共享平台需要状态所有权、保留策略、billing attribution、credential scoping 和可重放 audit trail。Harness 不再只是包住一次模型调用的东西，而是围绕许多 agents、许多 environments 和许多人类利益相关者的控制系统。

图：按主题分组的十二要素



要点

- “大多只是软件”：好的 agent 是带有非确定性 LLM 组件的确定性程序，而不是一袋工具循环到完成。
- **Own your prompts**：framework 会隐藏 prompt；prompt 应作为一等代码纳入版本控制。
- **Stateless reducer** 模式：把 agent 看作对 event log 的 fold，使其可序列化、可重放、可测试。
- 平台范围会改变十二要素：lifecycle、state、identity、billing、observability 和 human handoff 会变成共享基础设施问题。
- 小而聚焦的 **agent**：每个 agent 3-20 步；上下文越长，性能越差。
- 用工具联系人类：结构化 `request_human_input` 工具优于依赖模型自由文本选择。
- 压缩错误，不要隐藏错误：可见错误 trace 支持 self-healing，连续错误计数器提供安全升级路径。

延伸阅读

- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>

第 10 章：评估

10.1 为什么需要 Evals

没有 evals，调试就是被动的：等投诉、手动复现、修复，然后祈祷别回归 ([Anthropic - Demystifying Evals for AI Agents](#))。团队无法区分真实回归与噪声，无法自动测试大量场景，也无法衡量改进。采用新模型也会很慢：没有 evals，新模型上线意味着数周手工测试；有 evals 的团队可以在数天内验证优势并调 prompt。

回顾一下：eval 评估的是真正重要的单元——对 agent 来说，这个单元是 loop，而不是单次模型调用（见《LLM Foundations》第 13 章）。Agent eval 运行的是整个 model + harness 系统，并在环境中判断最终状态是否满足任务。这一点很重要，因为 agent 可能通过了中间测试、回答得很流畅、甚至走了一条看似合理的路径，但仍然没有完成用户真正的目标。

Anthropic 将 evals 视为复利型基础设施：成本在前期可见，收益在 agent 生命周期中累积。他们建议尽早开始，哪怕只有 20-50 个简单任务。Agent 早期开发中 effect size 很大，小样本也足以发现方向；成熟 agent 需要更大 eval 才能检测较小效果。

10.2 Evaluation 的结构

eval 的基础词汇——grader (code/model/human)、trace、regression 与 capability、pass@k 与 pass^k、reward hacking——已在《LLM Foundations》第 13 章建立；本章只做简要回顾，并在其上构建 harness 专属的机制。

Anthropic 的词汇 ([Anthropic - Demystifying Evals for AI Agents](#)):

- **Task**: 有定义好的输入和成功标准。
- **Trial**: 对一个 task 的一次尝试。因为输出有随机性，一个 task 会有多次 trial。
- **Grader**: 评分某个性能方面；一个 task 可以有多个 grader，每个包含 assertions。
- **Transcript** (trace、trajectory): trial 的完整记录。
- **Outcome**: trial 结束时的最终环境状态，不同于 agent 的文本回应。例如订票 agent 说“机票已订好”是 response；SQL 数据库是否有对应行才是 outcome。
- **Evaluation harness**: 运行 eval end-to-end 的基础设施，区别于 agent harness。
- **Agent harness** (或 scaffold): 与模型一起被评估的系统。“当我们评估一个 agent 时，评估的是 harness 与模型协同工作。”

10.3 三类 Grader

- **Code-based**: 字符串匹配、二进制测试、静态分析、outcome verification、tool-call verification、transcript analysis。快、便宜、客观、可复现，但对有效变体脆弱。
- **Model-based**: rubric scoring、自然语言断言、pairwise comparison、多 judge 共识。灵活、可扩展、能处理开放任务，但非确定性，需要人类校准。
- **Human**: 领域专家 review、众包判断、A/B testing。最适合校准和主观判断，但昂贵、慢，如果 rubric 弱也会不一致。

Anthropic 建议：能用确定性 grader 就用确定性；必要时用 model-based；human 用于周期性校准。他们还提醒，不要过度评分 agent 采取的路径，而应评估产物。Agent 经常能找到 eval 设计者没想到的有效路径，路径评分会使 eval 脆弱。

10.4 Capability Eval 与 Regression Eval

两类目的不同 ([Anthropic - Demystifying Evals for AI Agents](#)):

- **Capability evals** 问“这个 agent 擅长什么？”它们从低通过率开始，目标是 agent 正在挣扎的任务，给团队一座可爬的山。
- **Regression evals** 问“agent 是否仍能完成过去能完成的事？”它们应接近 100%，用于防止倒退。

Agent 成熟后，通过率高的 capability eval 会 graduate 到 regression suite。曾经衡量“能不能做到”的任务，会变成“是否仍可靠做到”的任务。

10.5 pass@k 与 pass^k

对行为在运行间变化的 agent，有两个斜率相反的指标 ([Anthropic - Demystifying Evals for AI Agents](#)):

- **pass@k**: k 次尝试中至少一次正确的概率。随着 k 增大而上升。
- **pass^k**: k 次 trial 全部成功的概率。随着 k 增大而下降。

这些指标之所以必要，是因为 agent 运行具有随机性。同一个 prompt、模型和 harness，在不同 trial 中可能产生不同的工具顺序、搜索路径或最终答案。因此，单次运行只能提供很弱的证据；重复 trial 才能看出系统只是偶尔成功、稳定可靠，还是碰巧走通了一条脆弱路径。

如果单次成功率 75%， pass^3 约为 42%， pass^{10} 约为 5.6%，而 $\text{pass}@10$ 约为 99.9999%。正确指标取决于产品：当系统可以生成多个候选并选择或展示最佳结果时，一次成功就有价值；面向客户重复执行的 agent 则需要 pass^k 式可靠性。

10.6 八步路线图

Anthropic 将从无 eval 到可信 eval 的路线概括为 ([Anthropic - Demystifying Evals for AI Agents](#)):

1. 尽早开始：从真实失败中收集 20-50 个任务。
1. 从已有手工测试开始：pre-release check 和 bug tracker 中的项目。
2. 写清晰任务和参考解：两个领域专家应得出相同判断；大量 trial 0% 通过率通常意味着 task 坏了，而不一定是 agent 不行。
3. 构建平衡问题集：既包含某行为应出现，也包含不应出现的情况。单侧 eval 会导致单侧优化。
4. 构建稳定 eval harness：隔离 trial，避免共享状态。Anthropic 观察到 Claude 会因查看前一 trial 留下的 git history 而获得不公平优势。
5. 谨慎设计 graders：能确定性就确定性；多组件任务给部分分；校准 LLM-as-judge rubric；允许 “Unknown” 以避免幻觉；防 eval hacking。
6. 阅读 transcripts：失败应看起来公平；当分数不再上升，要判断是 agent 回归，还是 eval 本身不公平。
7. 监控 capability eval 饱和：100% 的 eval 不再提供改进信号。SWE-bench Verified 从 30% 起步，现在接近 80%，小分数提升可能掩盖大能力提升。
8. 开放维护：领域专家和产品团队应贡献 eval task；PM、CS、sales 也可以用 Claude Code 把 eval 作为 PR 提交。

10.7 不同 Agent 类型的真实 Evals

下面这些按 agent 类型给出的典型例子来自 Anthropic 的综述 ([Anthropic - Demystifying Evals for AI Agents](#)):

- **Coding agents**：确定性 grader 很自然，例如代码是否运行、测试是否通过。SWE-bench Verified 基于固定 GitHub issue 跑测试套件；Terminal-Bench 测试端到端任务，例如从源码构建 Linux kernel。
- **Conversational agents**：成功是多维的，例如 ticket resolved（状态检查）、对话少于 10 轮 (transcript constraint)、语气合适 (LLM rubric)。常需要第二个 LLM 模拟用户 (tau-Bench、tau2-Bench)。
- **Research agents**：groundedness check（论断有来源支持）、coverage check（包含关键事实）、source-quality check（权威来源，而非首个检索结果）。需要频繁与人类专家校准。
- **Computer-use agents**：真实或 sandbox 环境，URL/page-state check，后端状态验证（订单是否真的创建，而不仅是出现确认页面）。WebArena 和 OSWorld 是典型例子。

10.8 面向修复的验证反馈

对 coding agent 来说，最有用的 grader 往往同时也是修复信号。只说 “test failed” 的检查能确认 outcome 不好，但给 agent 的抓手很少。更好的失败消息会说明违反的是哪条路径、期望状态是什么、实际状态是什么、下一步该检查哪里。OpenAI 的 Codex harness 指南强调，应把反复出现的 review 意见和架构规则转成 repo-local 检查，让 agent 在还能修复时收到具体反馈 ([OpenAI - Harness Engineering](#))。

端到端验证也应作为完成门槛，而不是象征性的最后一步。Anthropic 的长运行应用 harness 要求 coding agent 启动应用，并通过浏览器驱动路径验证 feature，因为 agent 否则容易在本地测试或视觉检查通过后宣称完成，但真实用户流程仍然坏着 ([Anthropic - Effective Harnesses for Long-Running Agents](#))。第 9.2 节的通用规则在这里直接适用：评估环境状态，而不是评估 agent 的自信。

10.9 阅读 Transcript 是核心技能

反复出现的主题是：在有人阅读 transcripts 前，不要直接相信 eval 分数。Anthropic 提到 Opus 4.5 在 CORE-Bench 上初始得分 42%，但调查发现严格 grader 会惩罚把期望答案 96.124991... 写成 96.12，任务 spec 模糊，还有无法精确复现的随机任务。修复 grader bug 并使用限制更少的 scaffold 后，分数跳到 95% ([Anthropic - Demystifying Evals](#))。类似地，METR 发现 time-horizon benchmark 中有任务要求 agent 优化到某个阈值，但评分要求超过阈值，于是惩罚遵循指令的模型，奖励忽略指令的模型 ([Anthropic - Demystifying Evals](#))。

通用规则是：失败应显得公平。当分数平台期时，要问 eval 是否仍在测它应该测的东西。

10.10 Evals 只是多层体系中的一层

自动 eval 不是完整图景。Anthropic 将其类比安全工程中的 Swiss-cheese model：没有一层能抓住所有问题 ([Anthropic - Demystifying Evals](#))。完整栈包括：

- **自动 evals**：快速迭代、回归检测、模型升级。
- **生产监控**：获得真实世界失败和 ground truth。
- **A/B testing**：在流量足够后验证重大改动。
- **用户反馈**：发现没人预料的问题。
- **人工 transcript review**：建立对失败模式的直觉。
- **系统性人类研究**：校准 LLM grader 或主观输出评分。

10.11 Readiness Validation 与失败归因

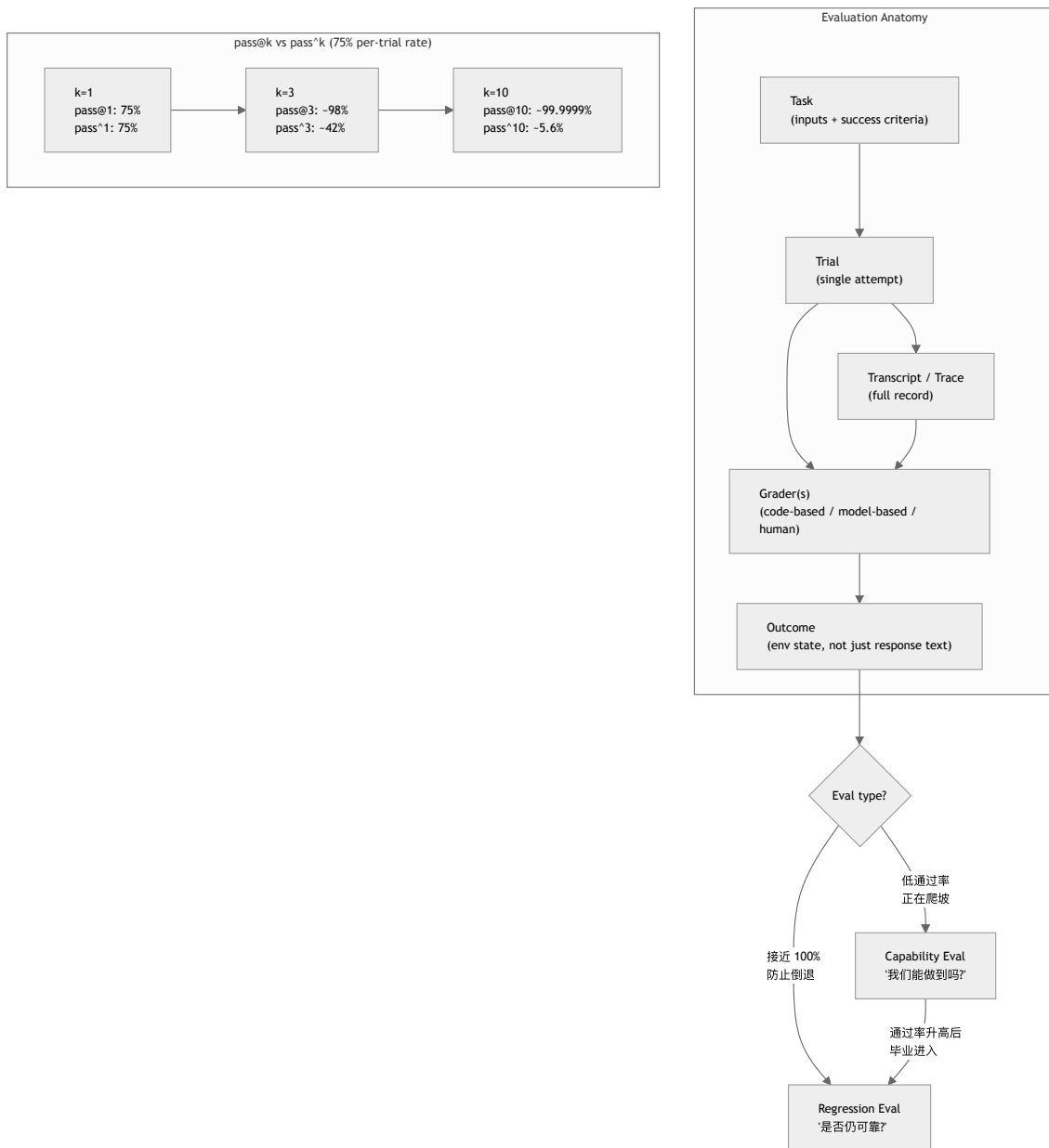
OpenReview 综述把 **Verification** 单独列为一层，是因为 harness 需要的不只是打分。Verification 问的是：某个 model + harness 组合，在特定任务分布、特定环境、预算和治理规则下，是否已经可以投入使用 ([OpenReview - Agent Harness Engineering: A Survey](#))。

这个框架给普通 eval suite 增加了两个实践要求：

- **Readiness validation**：发布门槛应把任务、环境重置规则、工具可用性、上下文策略、预算限制和治理检查绑定在一起。换了 execution substrate 或 tool menu 后，原来的分数不自动可迁移。
- **失败归因**：失败应标注最可能出问题的层：execution、tool interface、context、lifecycle、observability、verification 或 governance。否则团队很容易反复调 prompt，但真正缺陷可能是沙箱不稳定、工具面过大、缺 checkpoint，或 policy hook 太弱。

这也解释了为什么 benchmark 数字很脆弱。基础设施变化、成本优化、工具边界改变，都可能改变同一模型测出来的能力。好的 eval report 因此应和模型名一起记录 harness 配置：环境镜像、资源限制、工具目录、上下文组装策略、retry、grader 和 human-approval 规则。

图：Eval 结构与 pass@k / pass^k



要点

- **Evals** 是复利型基础设施：即使 agent 未成熟，也从 20-50 个真实失败任务开始。
- **Eval** 比单元测试更宽：它在环境中检查 model + harness 系统是否达成任务 outcome。
- **Outcome** 不等于 **response**：测环境状态（数据库行、URL、文件），而不是只测 agent 说了什么。
- 面向修复的反馈能提高自我修正：检查应说明哪里失败、为什么失败，以及什么证据才算修好。
- 三类 **grader** 形成金字塔：code-based 负责速度，model-based 负责细微判断，human 负责校准。
- **pass@k** 与 **pass^k** 服务不同产品：多候选生成可用 pass@k，重复面向客户执行需要 pass^k 式可靠性。
- 阅读 **transcript** 是核心技能：分数平台期可能是 agent 回归，也可能是 eval 不公平；只有 transcript 能区分。
- **Readiness** 绑定配置：eval 结果应带着产生它的 harness 配置一起解释。
- **Evals** 是多层之一：自动 eval + 生产监控 + A/B testing + 用户反馈 + 人工 review。

延伸阅读

- Mikaela Grace et al., *Demystifying Evals for AI Agents*, Anthropic, Jan 2026.
<https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- Gian Segato, *Quantifying Infrastructure Noise in Agentic Coding Evals*, Anthropic, Feb 2026.
<https://www.anthropic.com/engineering/infrastructure-noise>

- Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026. <https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
- Ken Aizawa, *Writing Effective Tools for Agents - with Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/writing-tools-for-agents>
- OpenAI, *Harness Engineering: Leveraging Codex in an Agent-First World*, Feb 2026. <https://openai.com/index/harness-engineering/>
- Justin Young et al., *Effective Harnesses for Long-Running Agents*, Anthropic, Nov 2025. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>

第 11 章：基础设施噪声

阅读 benchmark leaderboard 的实践者需要知道：小分差的不确定性比报告数字看起来更大。Anthropic 的“Quantifying Infrastructure Noise”记录了这种影响的量级 ([Anthropic - Quantifying Infrastructure Noise in Agentic Coding Evals](#))。

静态 benchmark 直接评分模型输出。Agentic coding eval 不同：模型写程序、跑测试、安装依赖、多轮迭代。Runtime 不是被动容器，而是问题求解过程的一部分。资源预算不同的两个 agent，其实并不是在参加同一场考试。

11.1 头号结果

Anthropic 在 Google Kubernetes Engine cluster 上用六种资源配置运行 Terminal-Bench 2.0：同一 Claude 模型、同一 harness、同一任务集，只改变资源 floor 和 ceiling。最充足与最不足资源设置之间相差 6 个百分点 ($p < 0.01$) ([Anthropic - Quantifying Infrastructure Noise](#))。

这超过了许多 leaderboard 顶部模型之间的典型差距。直接含义是：leaderboard 上 2 分领先可能是真实能力差异，也可能只是某次 eval 跑在更强硬件上。

11.2 两个区间

数据呈现两个区间：

- 从 1x 到 3x 单任务资源规格，分数在噪声范围内波动 ($p = 0.40$)，但基础设施错误率单调下降：从严格 enforcement 的 5.8% 到 3x headroom 的 2.1%， $p < 0.001$ 。在 1x 崩溃的任务本来也会失败；额外资源修复了瞬内存峰值导致的 OOM kill，并没有让 eval 本身更简单。
- 3x 以上，分数上升快于基础设施错误下降。从 3x 到 uncapped，infra errors 下降 1.6 个百分点，但成功率上升接近 4 点。额外资源让 agent 能尝试只有在资源宽松时才成立的方法，例如拉大型依赖、跑高内存测试套件、用 heavyweight 工具 brute force。

11.3 对测量意味着什么

严格资源限制会无意中奖励高效策略；宽松限制奖励善用可用资源的 agent。二者都可以是合法测试目标，但如果不说明配置就折成单一分数，解释会变困难。

Anthropic 的 `bn-fit-modify` 例子说明了这一点：在宽松限制下，一些模型默认先安装完整 Python 数据科学栈 (pandas、networkx、scikit-learn)，再写方案代码。在严格限制下，pod 在安装时内存耗尽。其实也存在更轻的策略：只用标准库从头实现数学。一些模型默认走轻策略。资源配置决定哪种默认策略成功 ([Anthropic - Quantifying Infrastructure Noise](#))。

同样效应也出现在 Terminal-Bench 之外，但幅度较小。Anthropic 的 SWE-bench 实验中，5x RAM 在 227 个问题上比 1x 高 1.54 个百分点；比 Terminal-Bench 小，因为 SWE-bench 任务资源密集度更低，但仍非中性 ([Anthropic - Quantifying Infrastructure Noise](#))。

11.4 建议

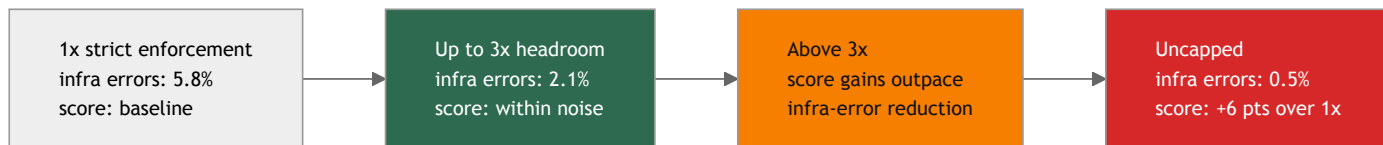
Eval 应分别说明保证分配 (floor) 和硬上限 (ceiling)，不要只给一个固定值。对 Terminal-Bench 来说，任务规格 3x ceiling 是合理默认：它将 infra errors 减少三分之二，同时让分数提升保持在噪声内 ([Anthropic - Quantifying Infrastructure Noise](#))。具体倍数取决于 benchmark 和任务分布，应当报告。

对 leaderboard 读者来说，操作规则是：低于 3 个百分点的差距，在资源配置被文档化并匹配前都应保持怀疑。几分领先可能是真实能力，也可能只是更大的 VM。

图：资源配置与分数 (Terminal-Bench 2.0 汇总)

下表总结 Anthropic 报告的两个区间。只有 1x、3x 和 uncapped 错误率在文章中明确量化；中间行保持定性，以免暗示来源没有给出的精度。

资源水平	Infra Error Rate	分数变化	解释
1x (严格)	5.8%	baseline	OOM kill 掩盖真实失败
3x	2.1%	噪声内	甜点：infra errors 减少 2/3
3x 以上	更低	分数开始比 infra errors 更快上升	Agent 开始利用额外 RAM
Uncapped	0.5%	比 1x +6 pts	资源密集默认策略成功



3x 以上，分数收益超过基础设施错误下降；额外资源启用了策略，而不只是提高稳定性。

要点

- 硬件本身就可带来 6 点差距：Terminal-Bench 2.0 最强与最弱资源配置差距超过常见顶部模型差距。
- 两个区间：1x-3x 修复基础设施不稳定；3x 以上启用资源密集策略。
- **3x ceiling** 是实践默认：降低三分之二 infra errors，并让分数提升保持在噪声范围。
- **Leaderboard** 怀疑阈值：3 个百分点以下差异，需等资源配置文档化并匹配后再解读。
- 资源限制塑造策略：严格限制奖励效率，宽松限制奖励资源利用；二者都有效，但必须区分。

延伸阅读

- Gian Segato, *Quantifying Infrastructure Noise in Agentic Coding Evals*, Anthropic, Feb 2026.
<https://www.anthropic.com/engineering/infrastructure-noise>
- Mikaela Grace et al., *Demystifying Evals for AI Agents*, Anthropic, Jan 2026.
<https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026.
<https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>

第 12 章：基于 Trace 的迭代与 Model-Harness 共同演化

12.1 Traces 是反馈回路

今天的模型很大程度上是黑箱，内部机制难以解释。但输入和输出以文本形式可见，这足以驱动系统改进。LangChain 将 traces 视作 harness 调试的主要表面：每个 agent 动作都被存储，包括延迟、token 数、成本和工具调用 ([LangChain - Improving Deep Agents](#))。

他们的 *Trace Analyzer Skill* 自动化了这个循环：

1. 从 LangSmith 拉取实验 traces。
2. 派生并行错误分析 agents；主 agent 综合发现和建议。
3. 聚合反馈，并对 harness 做目标明确的改动。

这在结构上类似经典机器学习中的 boosting：迭代聚焦于上一轮错误。第 3 步有人类 review 会有帮助，但不严格必要，主要用于防止改动过拟合特定任务而损害泛化。

12.2 运行时可观测性与过程可观测性

基于 trace 的迭代需要两层可观测性。**运行时可观测性**记录系统做了什么：工具调用、命令输出、浏览器动作、延迟、token 使用量、错误、重试和最终环境状态。这是 LangChain 强调的那一层：存储每个 agent 动作，并从 traces 中聚合失败模式。

过程可观测性记录为什么当前工作应该被接受：任务范围、sprint contract、rubric、验证证据、排除项和 handoff note。Anthropic 的 generator-evaluator harness 把这一点显式化：实现前先协商 sprint contract，sprint 失败时 evaluator 按 rubric 给出具体反馈 ([Anthropic - Harness Design for Long-Running Application Development](#))。

两层会互相增强。只有运行时信号，没有过程工件，你知道发生了什么，却不知道是否满足原定范围。只有过程工件，没有运行时信号，又容易变成围绕坏行为写出的漂亮文档。生产级 harness 应让两者都可检查：任务轨迹、验收标准，以及环境确实达到目标状态的证据。

OpenReview 综述补充了一个有用的实现细节：agent trace 应组织成 span tree，而不是平铺日志 ([OpenReview - Agent Harness Engineering: A Survey](#))。至少应覆盖 model calls、tool invocations、retrieval steps、context-assembly operations、latency、token usage、cost、retries、permission decisions 和最终 outcome state。

这套结构如今正在被标准化。OpenTelemetry——已在分布式系统中广泛使用的厂商中立可观测标准——正在定义 *GenAI semantic conventions*：一套面向 LLM 与 agent 遥测的、约定好的 span 与属性名 schema ([OpenTelemetry - Semantic conventions for generative AI spans](#))。这些约定为 harness 产生的 span 命名——一个顶层的 `invoke_agent`、每次模型调用的子 `chat` span、每次工具调用的 `execute_tool` span——并规定了操作时长、token 使用量等标准指标。采纳它带来两点好处。第一，agent trace 与系统其余服务共用同一套可观测栈，因此延迟、成本和错误都能用现有工具查询，而不必自建一套 agent 日志器。第二，该标准直面隐私问题：它是否为是否记录 prompt 与 completion 内容定义了明确模式——不记录、附在 span 上、或存到外部再引用——并建议默认不捕获敏感载荷，这正是第 12.3 节与第 17 章要求的脱敏纪律。（在撰写本文时，GenAI 约定仍标记为 *Development*，因此具体命名会变；价值在方向，而非版本号。）

12.3 从生产 Trace 到 Regression Case

可观测性应反哺验证。暴露真实生产失败的 trace 太有价值，不能只停留在 debug artifact。成熟循环是：

1. 捕获失败或异常的生产轨迹。
2. 脱敏敏感数据，并冻结相关环境或 fixture。
3. 抽取用户意图、工具序列、中间状态和最终 outcome。
4. 为修正后的行为写确定性断言或 model-graded assertion。
5. 把 case 加入 regression suite，并保留原始 trace 作为证据。

这会把 traces 变成 eval task 的来源，也能防止团队只优化合成 benchmark，却错过自己用户真正触发的失败。它与 governance 的边界很重要：trace-to-eval pipeline 必须保留 privacy、provenance 和 permission metadata，否则会生成技术上有用但运营上不安全的测试。

12.4 压力测试 load-bearing 组件

Anthropic 的 harness-design 后续文章增加了互补纪律 ([Anthropic - Harness Design for Long-Running Application Development](#))。Harness 中每个组件都编码了一个关于“模型无法独立完成什么”的假设。随着模型改进，这些假设会过期。推荐做法是：一次移除一个组件，跑 eval，然后观察结果。

当 Opus 4.6 推出并带来更强长上下文检索和长周期 coding 行为后，Anthropic 在一个 harness 版本中移除了 sprint construct。Generator 可以不经 sprint decomposition 连续运行两个多小时。Evaluator 在早期模型上更 load-bearing，

但在 4.6 上变得更情境化：对处在当前模型 solo 能力边缘的任务有用，在能力范围内则可能是不必要开销。团队总结的原则是：“evaluator 不是固定的 yes/no 决策。任务超出当前模型 solo 可靠边界时，它才值得成本。”

12.5 Model-Harness 共同演化

今天的 frontier coding models 往往在其 harness 中一起 post-train ([LangChain - The Anatomy of an Agent Harness](#))。有用 primitive 被发现、加入 harness，并用于训练下一代模型；下一代模型在该 harness 中表现更强。这形成反馈回路，也带来副作用：改变 harness 逻辑可能让模型表现更差，即使改动看起来中性。

Codex 的 `apply_patch` 工具是典型例子。Codex 模型在这种具体 patch 格式上 post-train。OpenCode 作为 Claude Code 的开源替代，为 GPT/Codex 模型专门添加了 `apply_patch` 工具，以模拟 Codex harness；Claude 和其他模型仍使用普通 `edit` 与 `write` 工具 ([HumanLayer - Skill Issue](#))。

12.6 最佳 Harness 不一定是模型训练时的 Harness

对应推论，也是实践中可以大胆迭代的理由：模型训练时的 harness 通常不自动等于某个任务上的最优 harness。Terminal-Bench 2.0 是实践讨论中的常见数据点：HumanLayer 引用 Opus 4.6 在 Claude Code 中排名 33，而同一模型在另一 harness 中排名 5，leaderboard noise 大约为 4 个名次 ([HumanLayer - Skill Issue](#); [LangChain - The Anatomy of an Agent Harness](#))。应把具体名次视为 leaderboard 快照，而不是永久模型事实。

LangChain 的案例研究也实验性地得出同结论。他们用早期 harness 测 Claude Opus 4.6，得分 59.6%，有竞争力但低于调优后的 Codex 配置。原则可以泛化，例如上下文准备和验证；但针对模型再做几轮 harness 迭代，可能弥合差距 ([LangChain - Improving Deep Agents](#))。

实用规则是：换模型时，重新审查 harness。调优仍然 load-bearing 的部分，移除不再承担负载的部分。

12.7 实践要点

LangChain 总结的 harness 迭代原则 ([LangChain - Improving Deep Agents](#)):

1. **替 agent 做 context engineering**：用目录结构、可用工具、编码最佳实践、问题解决策略给模型 onboard 环境。
2. **帮助 agent 自我验证**：模型偏向第一个看似合理方案；应强提示其通过测试验证。
3. **Tracing 作为反馈信号**：同时调试工具和推理；模型走错路可能因为缺工具，也可能因为缺使用工具的指令。
4. **短期检测并修复坏模式**：loop detection 等 guardrail 是会随模型进步而消失的拐杖，但今天有用。
5. **按模型定制 harness**：Claude 与 Codex prompting guide 不同是有原因的；原则泛化，细节不泛化。

HumanLayer 的平行经验 ([HumanLayer - Skill Issue](#)):

有效的是：从简单开始，只在真实失败后添加配置；迭代并丢弃无效内容；通过 repo-level config 在团队内分发经过验证的配置；优化迭代速度，而不是只优化一次成功概率；在知道真正需要什么后修剪能力。

无效的是：预先设计理想 harness；“以防万一”安装几十个 skills 和 MCP servers；每次 session 末尾跑完整测试套件；微调每个 sub-agent 可访问哪些工具。

12.8 关于 AGENTS.md 的误导性数据

一个值得阅读的细节：ETH Zurich 研究测试了多个 repo 中的 138 个 agentfile (agentfile 是 AGENTS.md、CLAUDE.md 这类指令文件的统称)，发现 LLM 生成的 agentfile 会损害性能并增加 20% 成本；人类写的只提升约 4%；agent 处理 context-file 指令时多花 14-22% reasoning tokens；代码库概览和目录列表在该 benchmark 中没有帮助，因为 agent 自己可以发现 repo 结构 ([HumanLayer - Skill Issue](#) citing the ETH Zurich paper)。

HumanLayer 将其解读为支持自己的 AGENTS.md 建议：文件要简短，避免自动生成，用渐进披露而不是前置倒入所有指令，内容应普遍适用，而不是充满条件规则。他们自己的 CLAUDE.md 不到 60 行。

这不应被理解成“不要写仓库指令”。意思是根指令文件应该是路由器，而不是百科全书。OpenAI 的 Codex harness 指南也是同样形状：关键上下文应保留在仓库里，AGENTS.md 保持简短，指向更深入的文档，并尽可能用机械检查强制最重要的规则 ([OpenAI - Harness Engineering](#))。

通用原则是：更多配置不等于更好。每条无关指令都是 agent 必须处理但没有收益的指令；*instruction budget* 与 *token budget* 同样重要。

12.9 可复用 Harness 包与 Skills

一个 harness 模式经过真实使用验证后，不应该只停留在某个仓库里的隐性经验。应该把它打包。实践中的单位可以是 skill、模板包、小型脚手架生成器，或一组 repo 检查。关键是它同时携带指令和可工作的工件：不只是“记得维护状态”，而是包含 progress log 模板、feature list schema、启动脚本和验证命令。

Learn Harness Engineering 课程用 `harness-creator` 展示了这种打包形态：它是一个用于创建、评估和改进五个 harness 子系统的 skill，覆盖指令、状态、验证、范围和会话生命周期 ([Learn Harness Engineering — Skills](#))。这是一个有用的工程边界。可复用 harness 包不应把某个理想 workflow 永久冻结，而应让经过验证的默认做法容易安装、容易检查，并在 trace 证明组件不再 load-bearing 时容易移除。

12.10 Meta-Harness：优化 Harness 本身

一旦有 evals 和 traces，harness 本身就成为优化对象。OpenReview 综述提到 *meta-harness* 方向：不再把 harness 固定住后只比较模型，而是搜索 harness structure、prompting strategy、tool interface 和 control loop 选择 ([OpenReview - Agent Harness Engineering: A Survey](#))。

务实版本不是完全自动化架构搜索，而是有纪律的实验：

- 用同一 task suite A/B test 工具 schema 或 prompt 改动。
- 消融某个组件，例如 planner、context reset、memory layer 或 evaluator。
- 调整 cost controls、retry policy 和 tool-response truncation，观察质量在哪里下降。
- 衡量完整闭环，而不只是模型输出：success rate、pass^k reliability、latency、cost、human escalations 和 security false positives。

这扩展了 load-bearing 的思想。Harness 组件应持续证明自己值得保留。如果某组件只对少数高价值任务提升可靠性，就选择性路由；如果模型升级后它不再有用，就移除。

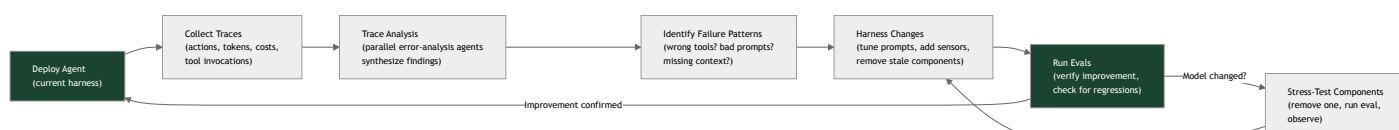
12.11 跨层耦合

ETCLOVG 也是调试地图。Trace 中看起来像模型决策差，可能只是另一层问题的症状 ([OpenReview - Agent Harness Engineering: A Survey](#))：

- 工具选择错误可能来自 **Tool** 层：动作空间太大，或 schema 没有暴露关键 affordance。
- 过早宣称完成可能来自 **Lifecycle** 层：退出条件没有被表示成持久状态机。
- 成本调优后的回归可能来自 **Observability** 与 **Execution** 层：资源限制改变了延迟、timeout 或 benchmark fidelity。
- 安全失败可能来自 **Governance** 层：训练时 alignment、部署配置和运行时 enforcement 之间的 policy、permission、audit hook 不一致。

实际做法是：在 trace review 中先标注层级假设，再改 prompt。好的 harness 迭代会先问：是哪一层让错误行为变得容易、不可见或便宜？然后再决定修复是 prompt、工具重设计、沙箱变更、新指标、更强 grader，还是 governance hook。

图：基于 Trace 的迭代循环



要点

- **Traces** 是主要调试表面：模型内部不可见，但文本 I/O 可见；系统化 trace 分析驱动 harness 改进。
- 可观测性有两层：运行时 trace 说明发生了什么；过程工件说明为什么这项工作应被接受。
- **Trace span** 应结构化：model call、tool call、retrieval、context assembly、permission、cost 与 outcome state 都需要机器可读 telemetry。
- 这套遥测正在出现标准：OpenTelemetry 的 GenAI semantic conventions (`invoke_agent`、`chat`、`execute_tool` span) 让 agent trace 加入普通可观测栈，并把内容捕获模式为隐私固化下来。
- 生产 trace 应转成 **regression case**：真实失败是最高信号 eval 任务，前提是保留 privacy 和 provenance。
- 训练时 harness 不自动最优：leaderboard 快照显示，同一模型在不同 harness 下可能大幅移动。
- 模型变化时压力测试组件：每个 harness 组件都编码了可能过期的假设。
- **Model-harness** 共同演化真实存在：post-training 将 harness 纳入训练回路，改变任一侧都可能破坏耦合。
- 臃肿的 **AGENTS.md** ROI 有限：保持简短、人写，再通过渐进披露进入 repo-local 文档和机械检查。
- 可复用 harness 包保留来之不易的实践：当 traces 证明某些模式有效，就把它们提升为 skills、模板、脚手架和检查。

- **Meta-harness** 把 **eval** 变成设计搜索：prompts、tools、retries、context policies 和 evaluators 都可像系统组件一样消融和优化。
- 层级归因避免只改 **prompt**：先用 ETCLOVG 判断是哪一层让失败成为可能，再改指令。
- 迭代速度胜过前置设计：从简单开始，只在真实失败后添加，并积极修剪。

延伸阅读

- Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026. <https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
- Prithvi Rajasekaran, *Harness Design for Long-Running Application Development*, Anthropic, Mar 2026. <https://www.anthropic.com/engineering/harness-design-long-running-apps>
- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- OpenAI, *Harness Engineering: Leveraging Codex in an Agent-First World*, Feb 2026. <https://openai.com/index/harness-engineering/>
- Walking Labs, *Learn Harness Engineering - Skills*. <https://walkinglabs.github.io/learn-harness-engineering/zh/skills/>
- OpenTelemetry, *Semantic conventions for generative AI spans*. <https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-spans/>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>

第 13 章：System Prompt 与指令架构

前言里的核心图把 *System Prompts* 列为 harness 的第一个组件，但前面各章一直把它当成既定前提。本章直接看这一指令层：它应该装什么、相互冲突的指令如何排优先级、运行时如何组装，以及为什么它值得用和系统其他部分一样的工程纪律来对待。

13.1 System Prompt 是 harness 层，不是一句 prompt

随口说的“prompt”是聊天框里敲进去的一个问题。Agent 的 system prompt 是另一回事：它是随 harness 一起发布的持久程序，框定 agent loop 的每一个回合。HumanLayer 的十二要素宣言把这点直接讲明——*own your prompts*——并主张对生产 agent 而言，prompt 是核心工程逻辑，而不是一个交给框架隐藏默认值的字符串 ([HumanLayer — 12-Factor Agents](#))。把它当应用代码看待，是下文一切的前提。

这个区别重要，是因为 system prompt 做着别的层做不了的工作：确立 agent 的角色与目标、声明可用工具及何时使用、编码 agent 不可违反的 policy、设定输出 contract。其中任何一项含糊时，失败不会表现为语法错误，而表现为漂移、过度积极、拒绝，或在错误时机用错工具——这些都很难精确归因（第 10 章、第 12 章）。

13.2 指令层级 (Instruction Hierarchy)

Agent 同时从多个来源收到指令：平台的 system prompt、开发者的配置、用户的请求，以及——关键地——藏在工具结果和检索文档里的文本。它们的权威并不相等，必须明确告诉模型这一点。OpenAI 把这形式化为 *instruction hierarchy*：可以训练模型把 system 级指令视为高于 user 级指令，两者又都高于工具输出里遇到的内容，从而让低优先级指令无法覆盖高优先级指令 ([OpenAI — The Instruction Hierarchy](#))。

对 harness engineer 来说，这正是第 5 章从安全角度画出的同一条边界，只是换到指令一侧来看。Prompt injection 恰恰是层级的失效：网页或文件里的文本 (*lethal trifecta*，第 5 章) 试图把自己提升到指令级。层级给出两道互补的防线：

- **依靠模型训练出的优先级**：把真正的 policy 放在 system 位置，绝不放在用户可编辑或工具提供的位置。
- **从结构上再加固**：把不可信片段标注为 data，因为训练出的优先级只是一种倾向，不是保证。

实用规则是：一条指令的权威应当来自 *harness 把它放在哪里*，而不是它措辞多强硬。重要的约束属于 system 层；写进检索文档里的约束只是建议，agent 可以忽略——也应该被要求忽略。

13.3 什么该进 system prompt

并非所有为真的东西都需要写出来；一个塞得过满的 system prompt 会在任务还没开始前就花掉第 2 章的 attention 预算。一个有用的划分：

- **属于 system prompt**：agent 的角色与目标、它必须始终遵守的高价值规则、工具描述自身表达不了的工使用 policy（何时不要用某工具、工具之间的关系）、输出 contract。
- **属于工具描述，而非 system prompt**：每个具体工具的机制（第 4 章）。把工具细节复制进 system prompt 会制造两份会逐渐漂移的真相来源。
- **属于检索上下文，而非 system prompt**：会变化、体量大、或只是偶尔需要的事实。它们应通过 just-in-time retrieval（第 2 章）到达，而不是烤进一个静态前缀。

13.4 动态组装与稳定前缀约束

大多数真实 agent 并不发布一个固定字符串。System prompt 是按调用 *组装* 的：一段 base policy、当前工具集、项目特定指令，可能还有一个检索到的 skill（第 4 章）。这个组装和第 2 章的 KV-cache 经济学相撞：cache 只在前缀逐字节相同时复用，所以任何在调用之间会变的东西，都应放在所有不变的东西之后。

这把指令架构变成一个布局问题。稳定、可复用的材料——base policy、常驻工具目录——放在最前。易变材料——当前任务、刚检索的事实、时间戳——放在最后。一个在顶部插入当前时间或请求 ID 的 system prompt，会在每个回合悄悄废掉前缀缓存，为零行为收益付出延迟和成本代价。

13.5 把 prompt 当版本化代码

因为 system prompt 是承重的，对它的改动就是对系统的改动，带着和代码改动一样的回归风险。第 10 章的纪律原封不动地适用：一次 prompt 编辑应该在改动前后各跑一遍 eval 套件，比较 pass rate、失败类别、成本和延迟。第 12 章的 model-harness 耦合更使这点尖锐——为某个模型调好的 prompt 可能在下一个模型上回退，所以 prompt 要和它被验证时所针对的模型一起版本化。

具体地，指令层值得拥有：版本控制、一份与 eval 结果挂钩的 changelog，以及一个负责人。“Own your prompts”正是这个意思——prompt 是一个有历史、被维护的产物，而不是某人在生产环境里现场改的字符串 ([HumanLayer — 12-Factor Agents](#))。

13.6 合适的“高度”

写 system prompt 最难的判断是它的具体程度。Anthropic 把这描述为找到合适的 *altitude* (高度): 太低, prompt 会变成一堆脆弱的硬编码 if-then 规则, 碰到第一个没预料到的情况就崩, 而且无止境地膨胀; 太高, 指引又含糊到模型没有可据以行动的具体信号 (Anthropic — Effective Context Engineering for AI Agents)。目标是: 具体到能可靠塑造行为, 又一般到能迁移到 agent 实际会遇到的各种情况。

高度也是过度工程藏身的地方。每一条为单次坏 trace 而加的特例规则都是一份人质: 它收窄行为、消耗上下文, 还可能和下个月加的规则冲突。往往更好的修法不是在 system prompt 里再加一句, 而是改工具、加一个 sensor (第 5 章), 或加一个把行为钉死的 eval case (第 10 章)。Transcript 会告诉你该选哪个 (第 12 章)。

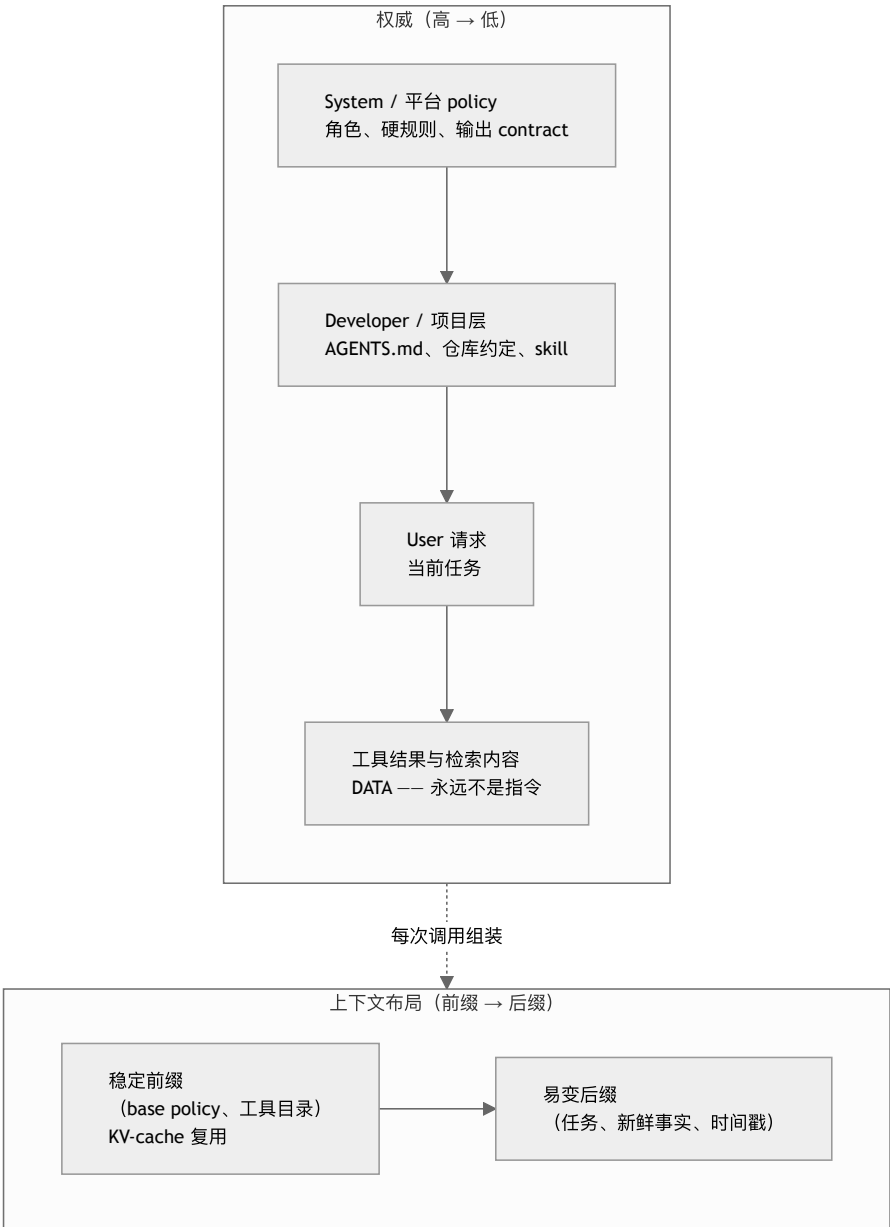
13.7 跨三层 harness 的分层指令

指令层不是铁板一块; 它镜像第 1 章的 inner/outer harness 结构。一个 coding agent 通常组合:

- 实验室发布的 **builder harness** system prompt,
- 团队加上的 **user harness** 项目指令—— AGENTS.md 文件、仓库约定、review 规则 (第 12 章) ——以及
- 为特定任务按需加载的 **skill** (第 4 章)。

它们在运行时组合成模型看到的有效指令集。第 12 章的教训在此延续: 不要假设项目指令越多越好。关于臃肿 AGENTS.md 文件的证据是混杂的, 一个过度规定的项目层可能和它所依附的 builder harness 互相打架。指令架构是要去测量的, 而不是去最大化的。

图：指令栈



权威自上而下流动，由位置而非措辞强制；布局自前缀向后缀流动，受缓存经济学支配。两条轴相互独立，都必须设计。

要点

- **System prompt 是应用代码**：一个持久、承重的 harness 层，不是随口字符串——拥有它、版本化它、用 eval 检验它的改动。
- **权威来自位置，而非强调**：instruction hierarchy 让 system 指令高于用户输入和工具内容；prompt injection 是层级失效，所以要把不可信片段标注为 data。
- **把各自该在的东西分开**：角色与 policy 进 system prompt，机制进工具描述，会变的事实进 just-in-time retrieval。
- **为缓存而组装**：稳定指令在前，易变材料在后，否则前缀缓存会悄悄失效。
- **瞄准合适的高度**：具体到能引导，一般到能迁移；克制住用一条新硬编码规则去补每个 trace 的冲动。

延伸阅读

- Eric Wallace et al., *The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions*, OpenAI, Apr 2024. <https://arxiv.org/abs/2404.13208>
- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Simon Willison, *The lethal trifecta for AI agents*, Jun 2025. <https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/>

第 14 章：模型选择、路由与推理模型

$Agent = Model + Harness$ 。前面各章都在做 harness，而把“模型”当成一个固定的单一选择。实际上模型这一侧本身就是一个设计面：某一步该跑哪个模型、用更便宜的是否也行，以及——自从推理模型出现以来——在行动前该让模型思考多久。本章讲 harness 拥有的、面向模型的决策。至于这些模型内部在做什么，姊妹卷 *LLM Foundations* 是参考；这里关心的是运维层面。

14.1 模型选择是 harness 决策

第 6 章的第一原则——做能奏效的最简单的事——对模型和对控制流一样适用。最大、最强的模型很少是 agent loop 每一步的正确默认。大多数 loop 里是混合作：一个真正需要前沿模型的困难规划或综合步骤，被一圈简单步骤——给输入分类、格式化结果、抽取字段——包围着，而后者用一个更小、更快、更便宜的模型一样能搞定 (*Anthropic — Building Effective Agents*)。

把模型当成一个全局设置，就把这块收益留在了桌上。Harness 可以按步选择，而选择的单位是“调用”，不是“应用”。这和第 3 章的 model-routing 提法是同一个直觉的推广：花在不改变结果之处的能力就是被浪费的能力，而在生产中这种浪费体现为每个请求上的延迟和金钱。

14.2 路由：让模型强度匹配任务难度

工作流意义上的 routing（第 6 章）把输入派发到专门分支。Model routing 是同一思路用在模型选择上：给请求的难度分类，把简单的送给弱而便宜的模型、把困难的送给强而昂贵的模型。难点在分类器——只有当“决定走哪条路”远比它带来的节省更便宜时，路由才划算。

RouteLLM 表明这可以学出来：一个在偏好数据上训练的 router 把 query 送往强或弱模型，在标准 benchmark 上以一小部分成本恢复了强模型的大部分质量 (RouteLLM)。harness 层的要点不是某个具体 router，而是这个杠杆的形状：在昂贵调用前放一个小而快的决策，可以推动 cost-quality 前沿——前提是路由决策本身可靠且便宜。一个被错误路由、又静默失败的困难任务，可能比它省下的那个模型贵得多。

14.3 级联与回退

路由是事先决定。*级联 (cascade)* 则推迟决定：先试便宜模型，若 verifier 满意就接受其答案，只在不满意时升级到更强的模型。FrugalGPT 演示了这种级联：把大多数 query 送给便宜模型、只把困难的残差升级，从而在大幅降本的同时匹配前沿模型的准确率 (FrugalGPT)。

级联的好坏完全取决于它的升级信号——那个判断“这个答案不够好”的 verifier。这就是本书各处一样的验证机制：基于代码的检查、测试、schema 校验，或基于模型的 grader（第 5 章、第 10 章）。没有可信信号，级联只是在产出同一个错误答案之前多加一段延迟。与之相关的生产模式是 fallback（回退）：当主模型出错、超时或被限流时，切换到备用模型，让 agent 优雅降级而不是卡死——这是一个可靠性问题，归属第 7 章的 checkpoint/resume 纪律。

14.4 推理模型与 test-time compute

推理模型经过 post-training——通常是在可检查的问题上做 reinforcement learning——学会在给出答案前生成一长串内部推理，花额外的 inference token 在困难问题上做得更好。DeepSeek-R1 记录了强推理行为可以主要通过在可验证奖励上做 RL 来诱发 (DeepSeek-R1)，而一条研究线表明 test-time compute——让模型在推理时想得更多——在某些问题上可能比换更大的模型更划算 (Snell et al. — Scaling LLM Test-Time Compute)。机制见姊妹卷 (*LLM Foundations*, 第 7–8 章)；这里的后果是：“模型思考多久”已成为一个 harness 可控的轴，独立于“用哪个模型”。

这是和第 11 章里任何东西都不同的一条 scaling 轴。那里，更多硬件带来更多 agent 动作。这里，更多 inference 预算在单次调用内、在任何工具被触碰之前，买到更多思考。两者都花钱，且不可互换。

14.5 驾驭推理模型：不要和训练对着干

推理模型改变了若干 harness 假设，而失败模式就是去和模型已有的行为对着干：

- **不要去手动 prompt 它天生就会的推理。** 对一个被训练成会推理的模型强塞“think step by step”，可能浪费 token 或与其训练行为冲突。遵循供应商对该类模型的指引。
- **为不可见的 token 做预算。** 一个推理模型可能在它第一个可见字之前吐出几千个隐藏推理 token。那是真实的延迟和成本。模型若暴露 reasoning-effort 控制，就把它当一等 harness 参数，必要时透出给用户。
- **不要把 trace 当解释信任。** 推理文本可能被隐藏、被摘要，或与模型实际计算不忠实。把任何暴露出的推理当 debug 辅助，而非验证——这正是 eval 章对模型自我报告所持的谨慎（第 10 章）。
- **推理不是 grounding。** 更长的内部独白仍然是无依据的生成。它能在 harness 提供的证据上推理得更仔细，却造不出从未给过它的事实。工具、检索和验证仍是必须（第 4 章、第 10 章）。

14.6 路由到推理：额外 token 何时划算

推理模型在数学、代码、规划和多步分析上帮助最大——这些活的 loop 里坐着一个可检查的困难子问题。它在简单抽取、格式化、分类上帮助最小，那里它更慢更贵却没有质量提升。这使推理成为和其他任何东西一样的路由目标：把困难、可验证的步骤送给推理模型，把周围简单步骤留在快速的非推理模型上 (§14.2)。

第 6 章的小代理模式在此自然组合。一个确定性 DAG 可以把推理模型调用放在恰好需要斟酌的那个节点——一个两侧是便宜确定性工作的“reasoning sandwich”——而不是在开放式 loop 的每个回合都为斟酌付费。

14.7 模型升级是 harness 事件

因为前沿模型越来越是带着它们的 harness 一起 post-training 的，模型和 harness 是耦合的（第 12 章）。因此换模型——升级、换更便宜的供应商、换量化变体、换不同推理类别——是对系统的改动，不是即插即换。一个在公开 benchmark 上分更高的模型，仍可能在你的 workflow 上回退，因为它对工具描述的遵循不同、verbosity 不同，或在你想要快答时却去推理。

纪律就是第 10 章那一条：每次模型改动都是回归事件。改动前后各跑一遍 eval 套件，既看 pass rate 也看失败类别和成本，并把针对旧模型验证过的 prompt 和工具与它一起版本化。Readiness validation 是针对每个 model-plus-harness 配置的，不是针对每个模型的。

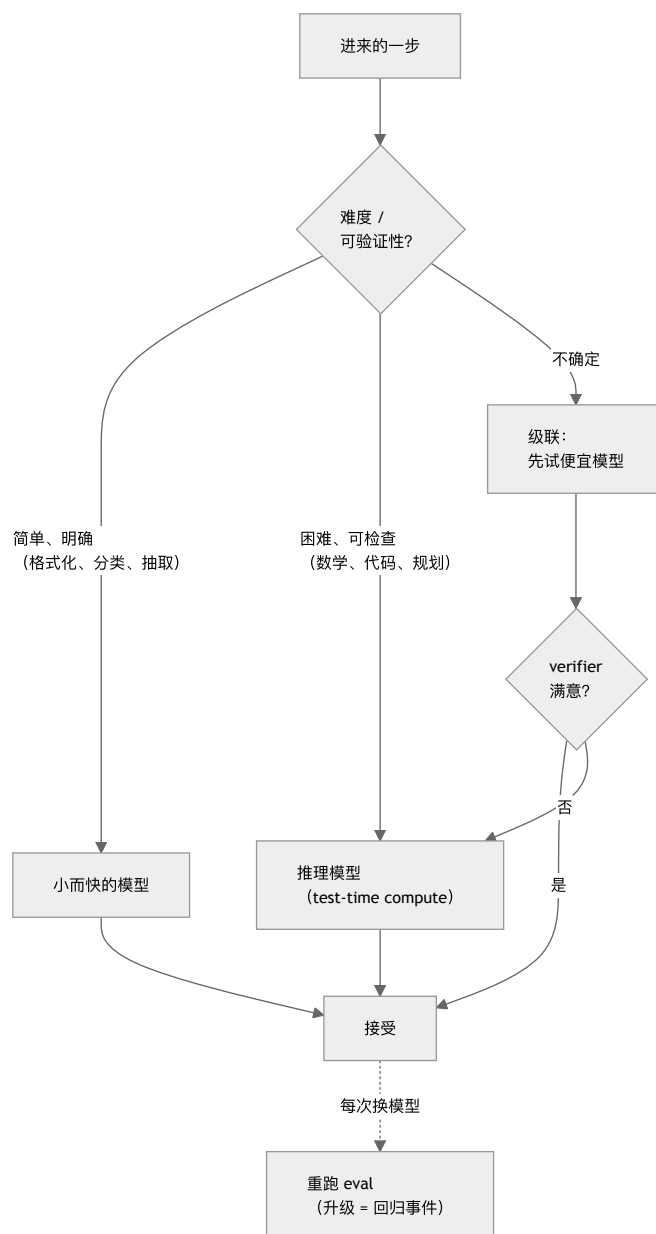
14.8 AI 网关：把路由做成基础设施

第 14.2-14.3 节把路由、cascade 和 fallback 当作逻辑。在生产中，这套逻辑越来越多地落在一个专门的基础设施组件里——AI 网关 (AI gateway, 亦称 LLM gateway 或 LLM proxy)。网关坐落在 harness 与各模型供应商之间，对外呈现一个通常兼容 OpenAI 的统一端点，同时吸收各种横切关切：模型路由与 fallback、负载均衡、重试、按 key 的预算与限流、缓存、以及请求/响应日志。开源例子包括 LiteLLM，它把 100 多个供应商收在一个接口之后，带成本追踪与负载均衡 (LiteLLM)；以及 Portkey，它还把 guardrails 与 PII 脱敏折进网关层 (Portkey - AI Gateway)。

它之所以属于 harness 讨论，是因为它把若干 harness 责任外置到了同一个地方。第 14.3 节的 fallback 模式变成一条网关配置，而非每个 agent 各自的代码。第 17 章的成本预算与按 span 归因自然挂在网关上，因为它看得见每一次调用。而第 14.7 节“换模型即回归事件”的纪律，在换模型只是稳定接口后面一处网关设置时，更容易执行——你可以在一部分流量上灰度一个新模型，而不动 agent 代码。

代价也很明显。网关是热路径上的一个新依赖，因此它自身也是施加第 5.11 节熔断器与超时的地方——网关宕机就是它背后每个 agent 都宕机。而集中化路由并不消除对可靠路由决策的需要（第 14.2 节）；它只是给那个决策一个安身之处。网关是管道，与第 4.3 节称 MCP 为管道是同一意义：它让好的路由便于运营，但不会把糟糕的路由策略变好。

图：模型选择决策



简单步骤往下路由，困难可检查的步骤往上路由；不确定时级联，由 verifier 决定升级。对模型的任何改动都是整个配置的回归事件。

要点

- 模型选择是按步、不是按应用：只在改变结果之处花前沿能力；把简单步骤路由到小而快的模型。
- 路由事先决定，级联事后决定：学出来的 router 能移动 cost-quality 前沿 ([RouteLLM](#))；verifier 把关的级联以更低成本匹配强模型准确率 ([FrugalGPT](#)) —— 两者都依赖一个便宜、可靠的决策信号。
- 推理模型加了一条新轴：test-time compute 在困难、可检查的任务上用 inference token 换质量 —— 区别于选更大的模型或买更多硬件。
- 不要和训练对着干：不手动 prompt 天生的推理、为隐藏 token 做预算、不把 trace 当解释、记住推理不是 grounding。
- 模型升级是 **harness** 事件：每次换模型都重跑 eval；readiness 是针对每个 model-plus-harness 配置的。
- AI 网关给路由一个安身之处：LiteLLM、Portkey 等 proxy 把 fallback、预算、缓存和日志收在一个接口之后 —— 便利的基础设施，但仍需要可靠的路由决策与它自己的熔断器。

延伸阅读

- Isaac Ong et al., *RouteLLM: Learning to Route LLMs with Preference Data*, 2024. <https://arxiv.org/abs/2406.18665>

- Lingjiao Chen, Matei Zaharia, and James Zou, *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*, 2023. <https://arxiv.org/abs/2305.05176>
- Charlie Snell et al., *Scaling LLM Test-Time Compute Optimally Can Be More Effective Than Scaling Model Parameters*, 2024. <https://arxiv.org/abs/2408.03314>
- DeepSeek-AI, *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*, 2025. <https://arxiv.org/abs/2501.12948>
- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
- *LLM Foundations for Harness Engineering* (姊妹卷), 第 5–8 章。../llm-foundations-zh/
- LiteLLM (BerriAI), *Python SDK and Proxy Server (AI Gateway)*. <https://github.com/BerriAI/litellm>
- Portkey, *AI Gateway*. <https://github.com/Portkey-AI/gateway>

第 15 章：人-Agent 交互

本书大部分在讲模型与世界之间的机器：context、工具、sandbox、状态、评估。但对任何做要紧工作的 agent，人也在这个 loop 里——批准动作、任务中途引导、review 结果、决定何时信任输出。通往这个人的接口，是一个和第 4 章的 agent-computer interface 一样真实的 harness 层，它是被工程化的，不是附带产生的。第 5 章介绍了它的一角——permission fatigue。本章把人的接口当成一个整体来处理。

15.1 人在 loop 之内

Agent-computer interface 那一章主张：投入到“agent 如何使用工具”的工程，应该和投入到“人如何使用屏幕”的一样多。人-agent 接口是它的镜像：投入到“人如何监督 agent”的工程，应该和投入到“agent 如何行动”的一样多。

Anthropic 的三条实现原则已经指向这里——保持简单、通过展示 agent 的规划步骤来优先做到透明、精心打造接口 (Anthropic — Building Effective Agents)。透明不是 UX 上的点缀；它是让监督根本成为可能的东西。

这个前提比 agent 更古老。Horvitz 1999 年的 *mixed-initiative* (混合主动) 接口原则就已框定了核心问题：一个代用户行事的系统，必须决定何时自主行动、何时让步，必须管理打断用户的代价，并且必须记住和使用交互的上下文 (Horvitz — Principles of Mixed-Initiative User Interfaces)。这些问题中的每一个，都在一个能跑 shell 的 agent 身上更尖锐地重现。

15.2 两个失败模式：permission fatigue 与盲目信任

人的接口会朝两个相反方向失败，好的设计必须同时避开两者。

- **Permission fatigue (许可疲劳)** 是问得太多的失败。当 agent 对每个动作都请求批准时，人会习惯化、不读就点“allow”，这比不问更糟，因为它制造了监督的表象却没有实质 (Anthropic — Beyond Permission Prompts, 第 5 章)。Sandbox 存在的意义，正是让边界内低风险的动作根本不需要询问。
- **盲目信任** 是问得太少的失败。一个流畅自信的输出诱使人盖橡皮图章。当 agent 以一种貌似合理的方式出错——幻觉出的引用、微妙地坏掉的重构——一个不暴露证据的 review 界面会让错误溜过去。

这是同一个旋钮的两端。把它拧向更少询问，冒盲目信任的险；拧向更多询问，冒疲劳的险。解法不是一个全局设置，而是与风险成比例的交互：让人被询问的程度，校准到动作的可逆性和影响半径上 (§15.3)。

15.3 混合主动：何时问、何时做

核心设计问题是逐动作的：agent 该直接做，还是先问？一个可行的默认是让答案随后果缩放：

- **静默执行**：对可逆、低风险的动作——读文件、跑测试、改 scratch 工作区 (第 5 章)——在 sandbox 内静默做。在这里询问只会训练人停止阅读。
- **执行并报告**：对要紧但可观察、可逆的动作——留下清晰 trace 供人事后 review，而不是阻塞等批准。
- **先问**：对不可逆或高影响半径的动作：删数据、发外部消息、花钱、动生产。这正是 *lethal trifecta* 边界 (第 5 章)——agent 即将把私有数据、不可信输入和外部触达结合起来的那一刻，正是人该在 loop 里的那一刻。

这张图就是本书在展望里反复回到 capability-control 权衡：更多自治是更多能力，也是更多控制负担。接口正是这个权衡被逐动作落地的地方。

15.4 把批准建模为一次工具调用

HumanLayer 的十二要素宣言做了最干净的结构化动作：用工具调用联系人类。与其把人类批准当成一个特殊的控制流异常，不如把它建模为 agent 可以调用的又一个工具——`request_human_approval(action, context)`——其结果像任何别的 observation 一样回到 loop 里 (HumanLayer — 12-Factor Agents)。

这把人类交互和架构其余部分统一了起来。批准请求是 event log (第 9 章) 里的一个事件，所以它是持久、可重放、可审计的。它和第 7 章的长运行模式组合：agent 可以请求批准、挂起，并在数小时后人类回应时恢复，因为它的状态活在 log 里，而不在一个一直挂着的连接里。它还让人类成为 trace (第 12 章) 里的一等参与者，而不是带外的打断。

15.5 设计 review 界面

当人确实去 review 时，决策的质量被接口展示的内容所限。一个只呈现 agent 结论的 review 界面诱发盲目信任；一个呈现结论背后证据的界面则让真正的判断成为可能。对 coding agent，这意味着 diff、跑过的测试、执行的命令——而不只是“done”。对 research agent，这意味着来源，而不只是摘要。

这是同一份 span telemetry——驱动调试的那份 (第 12 章)——面向监督的用法：一个好的 trace 同时也是一个好的 review 产物。人-AI 交互研究的指引在此直接适用——讲清系统能做什么、讲清这次做得多好、支持对错误输出高效地纠正与否决 (Amershi et al. — Guidelines for Human-AI Interaction)。一个让自己的工作便于验证、便于否决的 agent，才是人真正能监督的 agent。

15.6 引导与打断

监督不只在事前和事后，也在事中。一个看着 agent 走上错路的人，需要在不杀掉 session、不丢掉全部状态的前提下重定向它。引导 (steering) 是一个 harness 能力：把一条新指令注入正在运行的 agent，并让它在下一回合被纳入。

机制连回前面各章。因为 agent loop 每回合都重新组装 context (第 1 章)，一条引导消息只是被加进下一次迭代的 context——而按第 3 章的 recitation 逻辑，把纠正放在 context 靠近末尾处，能让它留在模型最可靠的 attention 跨度里。可打断性也依赖干净的 checkpoint (第 7 章)：一个能被暂停和恢复的 agent 才能被引导；一个把全部状态攥在单次不可打断调用里的 agent 不能。

15.7 校准信任：透明与不确定性

人接口的目标是 *校准过的信任*：人对 agent 的信任，恰好等于它在这个任务上配得到的程度。过度信任产生未经 review 的错误；信任不足产生疲劳和被浪费的人力。两个 harness 杠杆移动校准：

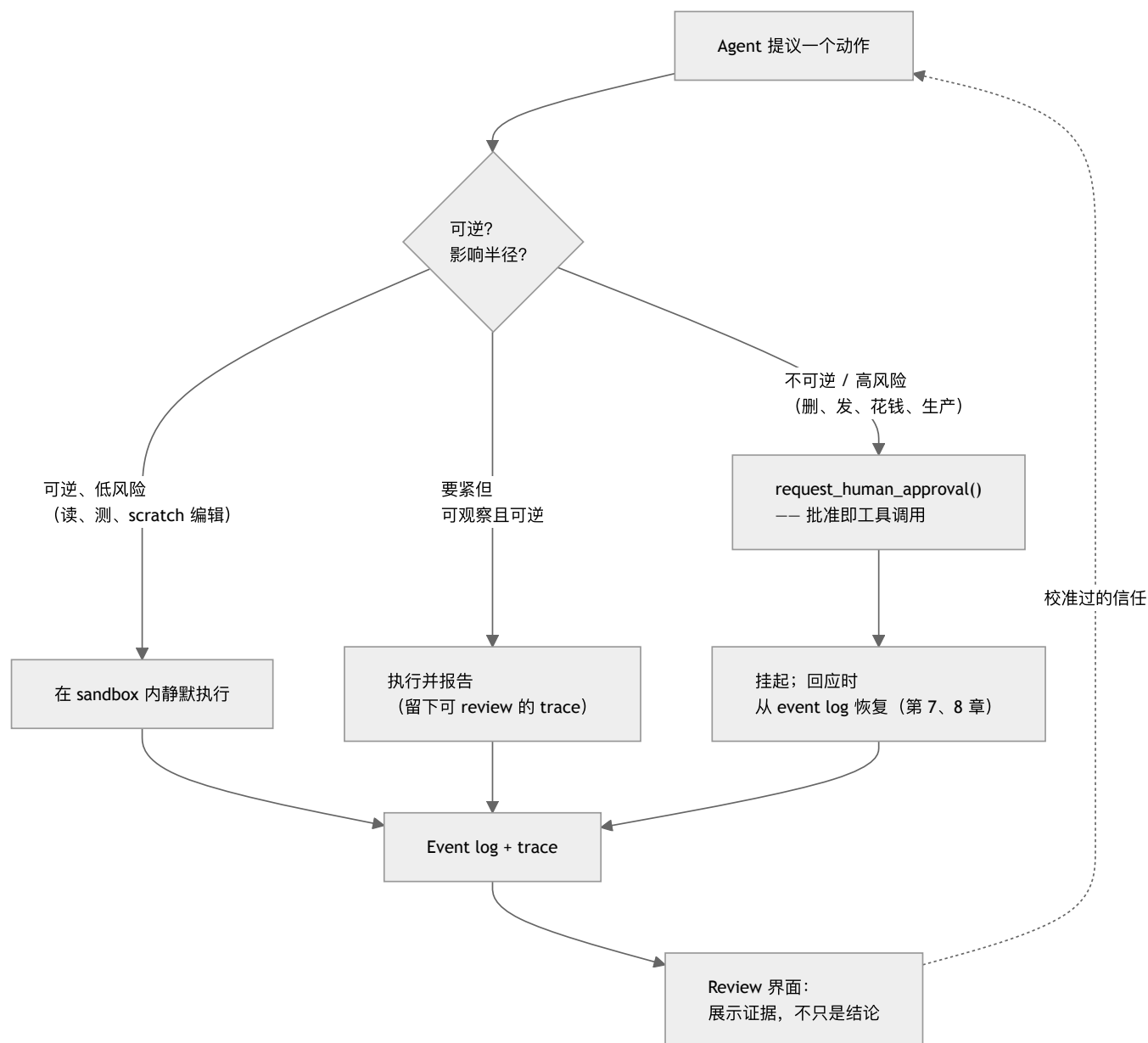
- **透明**——展示规划步骤、工具调用和证据——让人的信任跟随 agent 的实际推理，而不是它的流畅度 ([Anthropic — Building Effective Agents](#))。
- **不确定性信号**——讲清 agent 何时没把握——把人的注意力导向需要的地方。这取决于 agent 的校准是否可信，而姊妹卷警告校准本身就不可靠 (*LLM Foundations*, 第 10 章)；自报信心是弱信号，所以 harness 应优先把不确定性扎根在验证里 (测试过了吗？被引用的来源存在吗？)，而不是模型自己的含糊其辞。

15.8 监督许多 agent

随着 agent 增多，人的角色从做这份工作，转为监督做这份工作的 agent——再转为*同时*监督许多 agent。这正是这个领域在走向的杠杆，它改变了接口需求。一个人监督十个 agent，不可能读完每条 trace；接口必须浮现出需要注意的东西：哪些 agent 卡在批准上、哪些没把握、哪些产出的结果没过验证。

这把监督提升为一个 fleet 级关切，而[展望](#)把它标记为开放问题——人类批准接口和规模化的 harness 一致性都还没解决。仍然成立的原则是：让监督*便宜且*有意义。便宜，使一个人能监督许多 agent 而不被淹没；有意义，使监督是真实的而非橡皮图章。本章的每一项技术——与风险成比例的询问、批准即工具调用、富证据的 review 界面、引导、校准过的透明——都服务于这一个目标。

图：与风险成比例的交互



旋钮从静默执行到强制批准；按可逆性和影响半径把每个动作放在它上面。批准是一次工具调用，review 展示证据——同时避开 permission fatigue 和盲目信任。

要点

- 人的接口是一个 **harness** 层：投入到“人如何监督 agent”的工程，应和投入到“agent 如何行动”的一样多。
- 避开两个失败模式：permission fatigue（问太多训练出橡皮图章）和盲目信任（问太少放过貌似合理的错误）是同一旋钮的两端。
- 让交互与风险成比例：可逆低风险静默执行、可观察的执行并报告、不可逆或高影响半径的先问——即 lethal-trifecta 边界。
- 把批准建模为工具调用：它就变得持久、可重放、可审计，并与长运行的挂起/恢复组合。
- 展示证据，而非只是结论：一个工作便于验证、便于否决的 agent，才是人真正能监督的——无论是一个还是一队。

延伸阅读

- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024.
<https://www.anthropic.com/engineering/building-effective-agents>
- Dex Horthy, *12-Factor Agents*（要素 7：用工具调用联系人类），HumanLayer, Apr 2025.
<https://www.humanlayer.dev/blog/12-factor-agents>

- David Dworken and Oliver Weller-Davies, *Beyond Permission Prompts: Making Claude Code More Secure and Autonomous*, Anthropic, Oct 2025. <https://www.anthropic.com/engineering/claude-code-sandboxing>
- Saleema Amershi et al., *Guidelines for Human-AI Interaction*, CHI 2019. <https://www.microsoft.com/en-us/research/publication/guidelines-for-human-ai-interaction/>
- Eric Horvitz, *Principles of Mixed-Initiative User Interfaces*, CHI 1999. <https://www.microsoft.com/en-us/research/publication/principles-of-mixed-initiative-user-interfaces/>

第 16 章：Computer-Use 与多模态 Agent

第 4 章用定义良好的工具搭出 agent-computer interface：带 schema 的函数、MCP server、code API。有一类正在壮大的 agent 工作方式不同。它们不调用干净的 API，而是像人一样操作软件——看着屏幕移动光标、往字段里打字、点按钮。这些 *computer-use agent*，以及它们依赖的多模态感知，带来了 API-工具那几章没有的 harness 问题，而它们正是当下最有野心的通用 agent 所在的前沿。

16.1 越过 API：操作软件的 agent

动机是触达范围。大多数软件没有对 agent 友好的 API；它有一个为人造的 GUI。一个能看屏幕并据此行动的 agent，能用人能用的一切——老旧桌面应用、没有 API 的网站、永远不会有人去包成 MCP 的内部工具。Anthropic 的 *computer-use* 模型和 OpenAI 的 *Computer-Using Agent* 都走这条路：模型接收截图、对其推理、并发出像“在 (x, y) 点击”或“输入这个字符串”这样的底层动作 ([Anthropic — Computer use](#)；[OpenAI — Computer-Using Agent](#))。

这是一个和第 1 章里那个截然不同的 agent loop。observation 是一张图，而不是工具结果；action 是一个 UI 手势，而不是函数调用；环境是整个操作系统或浏览器，而不是一套精挑的工具集。本书关于工具、context、安全说过的一切仍然适用，但每一项都带上了视觉的、更底层的性格。

16.2 环境就是工具面

在 ACI 那章，harness engineer 选择工具，能让它们少而高 affordance（第 4 章）。一个 *computer-use agent* 从环境继承它的动作面：屏幕上每个按钮、菜单、字段都是潜在目标。第 4 章里那种谨慎的工具策展，不再以同样方式可用，因为“工具”就是应用恰好在视觉上暴露出来的任何东西。

这反转了一个关键杠杆。Harness 不再能通过提供更少工具来缩小动作空间；它必须帮 agent 准确感知动作空间，并约束 agent 被允许在哪里行动。设计工作从工具选择转向感知与划界——这正是接下来几节的主题。

16.3 屏幕的几种编码

屏幕可以用几种方式呈现给模型，而这个选择是一个带直接后果的 harness 决策——正是姊妹卷讲“图片变成 token”时所说的多模态要点 (*LLM Foundations*，第 2 章)。主要编码：

- **截图像素**：保留视觉布局和样式，但 token 很重，且可能让小字模糊。模型必须用视觉去定位元素。
- **DOM 或 HTML**（网页）：保留精确文本和结构，却丢掉了真正可见的东西——隐藏的、屏幕外的、被视觉遮挡的元素看起来都一样。
- **Accessibility tree**（可达性树）：暴露一个为辅助技术构建的结构化、语义化视图：role、label、state，通常远比裸 DOM 紧凑。
- **OCR**：从像素里恢复文本，却丢掉了结构和空间关系。

没有一种是完整的。截图显示人所见，却不显示底层结构；DOM dump 显示结构，却不显示显著性。生产系统常常组合编码——用截图看布局，加 accessibility tree 或 DOM 拿精确目标——更稳健，但花更多 context。这和检索（第 2 章）是同一个精度对预算的张力，只是现在落在视觉域里。

16.4 Grounding：从“看见”到“点击”

computer-use agent 独有的最难问题是 *visual grounding*（视觉接地）：把一个意图（“点击 Submit 按钮”）翻译成一个具体动作（在特定坐标点击）。模型可以正确判断某个按钮该被按，却仍发出错误的像素位置。Grounding 错误是一种在 API 工具里没有对应物的失败模式——在 API 里，点名一个函数是精确的。

一个常见的 harness 技术是 *Set-of-Mark prompting*：在截图上给候选可交互元素叠加编号标记，让模型选择一个离散标签（“点击元素 7”），而不是产出裸坐标 ([Yang et al. — Set-of-Mark Prompting](#))。这把一个易错的连续 grounding 问题，转成一个更可靠的离散选择，代价是 harness 必须提供一个元素检测步骤。相关的 SeeAct 工作线表明，即使强视觉模型，也需要这类 grounding 脚手架才能在真实网页上可靠行动——感知和动作是可分离的，而它们之间的缝隙正是 harness 体现价值的地方 ([Zheng et al. — GPT-4V is a Generalist Web Agent](#))。

16.5 动作空间及其失败模式

Computer-use 动作是底层且有状态的，函数调用则不然。一次点击依赖屏幕处于模型以为的那个状态；一个加载慢了的页面、一个弹出的 modal、一个移位的布局，都可能让模型已经决定好的动作失效。因此 loop 必须在每个动作后重新观察——屏幕是新的 ground truth——并容忍 observation 和 action 会失去同步。

这让第 7 章的头号杠杆 self-verification 更加核心。行动后，agent 应在继续之前检查屏幕是否如预期改变。它也让恢复更难：撤销一个 GUI 动作很少像逆转一次 API 调用那样干净，所以第 15 章的与风险成比例的交互更要紧——一个即将确认不可逆对话框的 *computer-use agent*，正是人类 checkpoint 该在的地方。

16.6 延迟、成本与像素的 token 重量

Computer-use loop 的每个回合都往 context 里送一张图，而图在 token 上很贵——单张高分辨率截图可能和一页文字一样贵（*LLM Foundations*，第 2 章）。一个人要点三十下的任务，就是三十张截图穿过 context window，这和第 2 章的有限上下文预算、以及 agentic 负载的 prefill 偏斜，相处得很糟。

Harness 杠杆还是那些熟悉的，只是更尖锐。把截图缩到任务能容忍的程度——但不能再多，因为缩小可能抹掉 agent 正需要读的那段文字。信息抽取后不要把旧截图留在 context 里；保留一条关于先前屏幕显示了什么的紧凑笔记，而不是像素（第 3 章“把没用的东西挡在外面”的纪律，对图片尤其适用）。能用结构化编码（accessibility tree）就别用像素，把昂贵的截图留给布局真正要紧的时候。

16.7 安全：全书最宽的攻击面

一个 computer-use agent 是 *lethal trifecta*（第 5 章）最浓缩的形态。它读取不可信内容（屏幕上任何网页或文档）、常常能访问私有数据（浏览器或文件系统里打开的任何东西）、并能对外通信（它能导航、提交表单、发消息）。这里的 prompt injection 不是工具结果里假设的文本；它是渲染在 agent 正盯着的网页上的恶意指令，而模型可能把它读成命令。

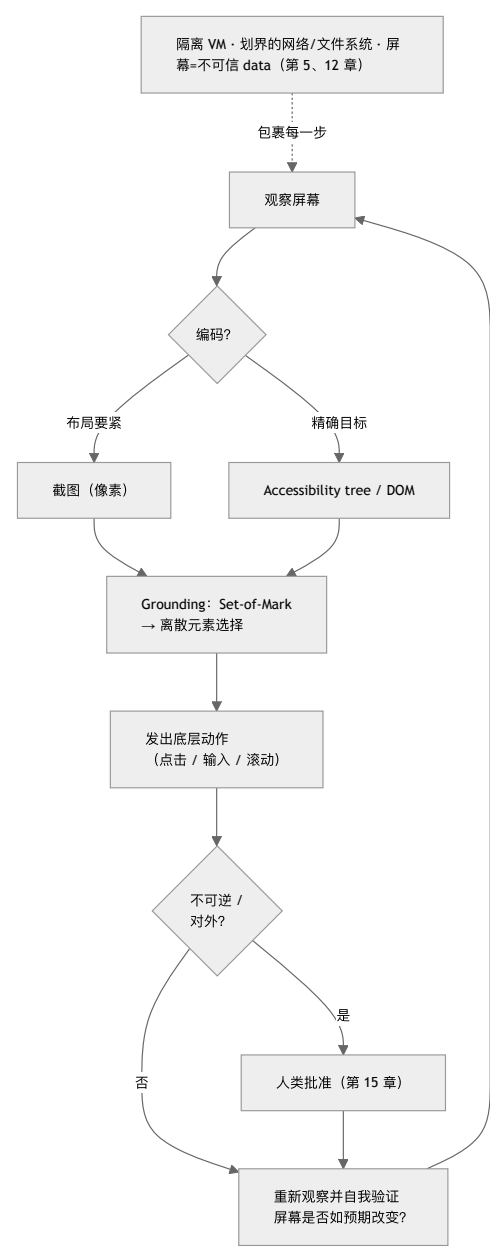
防线就是第 5 章的 sandboxing 和 governance，只是因为动作面如此之宽而更严。把 agent 跑在隔离环境里——一个专用 VM 或容器，而不是用户的主 session。划定网络和文件系统访问。在不可逆或对外动作前要求人类批准（第 15 章）。并把屏幕上的一切都当不可信 data，绝不当指令——当“data”是一整张渲染好的网页时，第 13 章的 instruction-hierarchy 纪律是承重的。

16.8 评估 computer-use agent

因为环境是一整个 OS 或浏览器，评估必须是环境式的，而非基于文本——第 10 章的 *outcome* 区分在这里不可回避，因为唯一要紧的是任务是否真的在环境中被完成。为此而造的真实 benchmark 提供了样板：WebArena 提供一个可自托管、真实的 web 环境，带基于执行的成功检查 (*WebArena*)；OSWorld 把这个思路扩展到一个真实桌面操作系统上的开放式任务 (*OSWorld*)。

这些 benchmark 的教训印证了全书：computer-use agent 在真实任务上远未饱和，而“模型能感知屏幕”和“agent 可靠完成任务”之间的差距，正是 harness 差距——grounding 脚手架、重新观察、恢复、划界、验证。一如既往，正确的 eval 是工作负载 eval（第 10 章）：一个公开 computer-use benchmark 是背景信号；agent 能否可靠地驱动 *你的* 应用，才是发布信号。

图：Computer-Use 循环



感知 → grounding → 行动 → 重新观察，包在一个 sandbox 里。Grounding 把“看见”变成“点击”；重新观察应对有状态的屏幕；sandbox 容纳全书最宽的攻击面。

要点

- **Computer-use agent 用干净 API 换触达**：它们像人一样操作 GUI，因此能用没有 agent API 的软件——代价是更难的 loop。
- **环境就是动作面**：工具策展让位于感知与划界；harness 帮 agent 准确看见动作空间，并约束它能在哪里行动。
- **屏幕编码是 harness 选择**：像素、DOM、accessibility tree、OCR 各自保留不同的东西；组合更稳健但花 context。
- **Grounding 是独有的失败模式**：把意图翻译成正确点击很易错；像 Set-of-Mark 这样的离散技术比裸坐标更可靠。
- **最宽攻击面，最严 sandbox**：computer-use agent 是浓缩的 lethal trifecta——隔离它、划界它、给不可逆动作设闸，并把整个屏幕当不可信 data。

延伸阅读

- Anthropic, *Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku*, Oct 2024. <https://www.anthropic.com/news/3-5-models-and-computer-use>
- OpenAI, *Computer-Using Agent (Operator)*, Jan 2025. <https://openai.com/index/computer-using-agent/>
- Jianwei Yang et al., *Set-of-Mark Prompting Unleashes Extraordinary Visual Grounding in GPT-4V*, 2023. <https://arxiv.org/abs/2310.11441>

- Boyuan Zheng et al., *GPT-4V(ision) is a Generalist Web Agent, if Grounded (SeeAct)*, 2024. <https://arxiv.org/abs/2401.01614>
- Shuyan Zhou et al., *WebArena: A Realistic Web Environment for Building Autonomous Agents*, 2023. <https://arxiv.org/abs/2307.13854>
- Tianbao Xie et al., *OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments*, 2024. <https://arxiv.org/abs/2404.07972>

第 17 章：成本、隐私与生产运维

到目前为止各章搭出了一个能工作的 agent。把它放到生产、服务真实用户，会多出三个被 agent loop 本身掩盖的关切：每次运行花多少钱、它如何处理敏感数据、以及一旦有人依赖它之后 harness 如何被安全地改动。没有一个光鲜，而每一个都是“能跑的 demo”悄悄无法变成“能用的产品”的地方。第 5 章讲了安全威胁模型，第 11 章讲了测量噪声；本章讲围绕它们的运维经济学、数据治理和发布纪律。

17.1 成本：给 agent 做预算

Agent 是运行模型最贵的方式，因为一个用户请求会扩展成许多模型调用、工具往返，以及——有了推理模型之后——很长的内部 token 流（第 14 章）。更糟的是，一个任务的成本事先未知：开放式 loop 可能跑三个回合，也可能跑三十个。这正是本书反复回到“做能奏效的最简单的事”（第 6 章）的运维理由——每个 agentic 回合都是一笔反复发生的费用，而不是一次性的。

因此生产 agent 需要一个明确的预算，就像第 7 章需要 checkpoint 一样。具体地：

- **每任务上限**：对 token、工具调用或墙钟成本设上限，超过就停下来问，而不是无限循环。无界 loop 就是一张失控的账单。
- **已经确立的成本杠杆**：把简单步骤路由到便宜模型，把前沿和推理模型留给确实需要的步骤（第 14 章）；缓存稳定前缀，让重复的 context 不必每回合重付（第 2 章）；返回 token 高效的工具结果，并用 code execution 避免把大结果倒进 context（第 4 章）。
- **要不要建的决定**：最便宜的 agent 是不跑的那个。许多任务用一个 workflow 或单次调用服务更好，而成本分析是这个选择的一部分（第 6 章）。

第 14 章的级联和路由模式既是质量控制，也同样是成本控制；FrugalGPT 的核心结果是，朴素的“总是调最好的模型”策略上的大部分花费，在不损失准确率的前提下是可以省掉的 (FrugalGPT)。

缓存在这里值得再看一眼，因为它有两种不同的形态。第 2 章的 KV/前缀缓存廉价地服务相同前缀；而语义缓存 (semantic cache) 服务相似请求：它把进来的查询嵌入成向量，当某个先前查询足够接近时就返回已存响应，这一做法由 GPTCache 推广开来 (Fu Bang - GPTCache)。对高频重复查询，它能消掉整次模型调用，而不只是让调用变便宜——但它引入了前缀缓存所没有的正确性风险：一次错误命中会为一个只是看起来像缓存项的问题返回一个微妙错误的答案。因此语义缓存是一个需要调优的组件——相似度阈值在节省与该风险之间权衡——并且应像任何其他 harness 变更一样接受同样的 eval 纪律（第 17.5 节）。两种缓存，连同上面的按 key 预算，都天然地在 AI 网关（第 14.8 节）处执行：它坐落在每一次调用的路径上，是唯一可以跨 agent 统一施加成本控制的地方。

17.2 成本归因与 trace

预算只有在成本被测量时才可执行，而测量它的地方是 trace。第 12 章的 span telemetry——一棵由模型调用、工具调用、检索构成的 span 树——同时也是成本账本：给每个 span 附上 token 数和美元成本，trace 就不只告诉你 agent 做了什么，还告诉你每一步花了多少。这把“agent 很贵”变成“这个工具每次调用返回 8000-token 的结果”，那是一个工程任务，而不是一句抱怨。

成本归因也喂进和失败同一个迭代 loop。一个既贵又低价值的步骤，是换更便宜模型、收紧工具结果或干脆移除的候选——把第 12 章的 trace 驱动迭代用在成本轴而非正确性轴上。正如第 11 章所警告，成本和资源配置也与被测行为相互作用，所以成本应该和能力并列报告，而不是分开。

17.3 隐私与数据治理

Agent 每次运行都触碰数据——读文件、查数据库、检索文档、向模型供应商和工具发请求。每一处都是敏感数据可能泄露的地方，而 harness 拥有这些控制。威胁在第 5 章的 lethal trifecta 形态下最尖锐：一个同时拥有私有数据访问、暴露于不可信内容、并有对外通信路径的 agent，可以被诱导去外泄那些数据 (Simon Willison — The lethal trifecta)。隐私不是和安全分开的关切；它是安全的数据处理那一面。

Harness 层的控制：

- **最小化进入 context 的东西**。Agent 永远看不到的数据不会泄露。检索任务所需的切片，而不是整条记录（第 2 章），并优先用 handle 而非裸敏感内容。
- **在边界处脱敏**。在内容跨进模型 context 或跨进 trace 之前，剥掉 secrets 和 PII——trace 是持久的，而且常被送往第三方 observability 工具，所以一条未脱敏的 trace 本身就是一次泄露（第 12 章）。
- **在检索时尊重权限**。在检索之前按用户被允许看到什么来过滤，而不是在生成之后；agent 绝不能变成读取其用户读不到的数据的途径（第 5 章、第 12 章）。
- **管控出口**。Trifecta 里对外通信那条腿，是 harness 能切断的那条：约束 agent 能把数据发到哪里，并对对外发送要求批准（第 15 章）。

17.4 多租户与隔离

当一个 agent 平台服务许多用户或组织时，隔离成为一个正确性属性，而不只是锦上添花。两个失败模式要紧。**状态串味**——一个租户的 context、记忆或缓存前缀泄进另一个租户的 session——是灾难性的，而且在加入记忆（第 3 章）和前缀缓存（第 2 章）却没做租户划界时很容易引入。**权限串味**——agent 用一个租户的凭据代另一个租户行事——是第 5 章的 delegated-auth 关切的多租户版本。

纪律遵循第 7 章的 managed-agent 架构：每租户的持久工作区、划界到单一租户资源的 sandbox、以及划界且绝不跨 brain/hands 边界共享的凭据。Event log（第 9 章）应携带租户身份，使每个动作可归因，而这也是审计（第 5 章）在共享系统里有意义的前提。

17.5 发布 harness 改动

Harness 是软件，而改动有人依赖的软件需要发布纪律——但 harness 改动异常地有风险，因为它的影响是统计性的，不是确定性的。一次 prompt 编辑、一个新工具、一次换模型、一次检索调整，都可能在没人测过的输入上改变行为，而回归可能是几个百分点的案例，而不是一次崩溃（第 10 章、第 12 章）。

随之而来的发布实践：

- **每次改动都用 eval 把关。** 改动前后各跑套件，比较 pass rate、失败类别、成本和延迟——第 10 章的回归纪律就是发布闸门。
- **逐步放量。** 因为 eval 套件永远覆盖不了完整输入分布，把 harness 改动当成任何有风险的部署：先发布给一小部分流量，盯着生产 trace 和 outcome 指标，并随时准备回滚。Canary 是便宜地接住 eval 漏掉的案例的方式。
- **把整个配置一起版本化。** 第 12 章的 model-harness 耦合意味着发布的单位是**配置**——模型、prompt、工具、检索策略——而不是其中任一部分。Readiness validation（第 10 章）是针对每个配置的，回滚也必须恢复整个配置，而不只是 prompt。

17.6 监控、SLO 与事件响应

Eval 在发布前跑；监控在发布后跑。两者互补：eval 测量一个精挑的分布，生产告诉你真实的那个。第 12 章的 span telemetry 是监控底座，而运维上的补充就是任何生产系统都需要、只是为 agent 特化的那些：

- **Outcome 指标，而不只是 uptime。** 一个“API 有响应”的 liveness 检查，对任务是否在成功一无所知。把任务成功率、升级/批准率、每任务成本、延迟分位（p50/p95/p99，如第 6 章）当一等信号来跟踪。
- **对漂移告警。** 上升的失败率、攀升的每任务成本、下降的 self-verification 通过率，即便什么都没崩，也是一次事件——这些是 agent 特有的失败形状。
- **生产失败变成回归案例。** 第 12 章的 trace-to-eval loop 同时也是事件响应 loop：一次生产失败被脱敏、变成一个可复现的 eval case、并被永久工程化掉，使同一事件不能再悄悄复发。

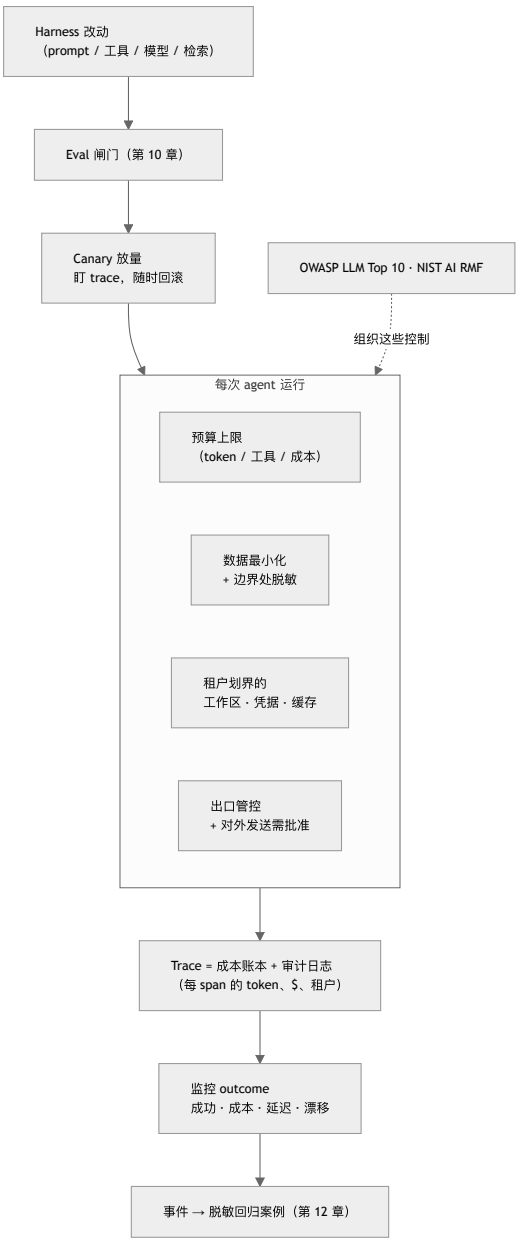
17.7 治理框架

这些关切不只是内部工程；对许多部署而言，它们是外部要求。四个参考框架帮助组织这项工作、无需重复造轮子，覆盖从自愿指南到有约束力的法律。

- **OWASP Top 10 for LLM Applications** 列举了常见风险类别——prompt injection、敏感信息泄露、过度自主（excessive agency）、供应链风险——并干净地映射到第 5 章和本章的控制（OWASP - Top 10 for LLM Applications）。
- **NIST AI Risk Management Framework** 提供一个更高层的结构——govern、map、measure、manage——给需要围绕 agent 部署做可审计风险流程的组织（NIST - AI Risk Management Framework）。
- **ISO/IEC 42001:2023**，首个 AI 管理体系标准，是 ISO 27001 的 AI 对应物：它规定组织如何为其 AI 建立、运行并持续改进一套管理体系，且可认证（ISO/IEC 42001:2023）。
- **欧盟《AI 法案》（EU AI Act）**（Regulation (EU) 2024/1689）是首部全面的 AI 法律。它按风险层级对系统分类，并对高风险用途施加有约束力的义务——风险管理、数据治理、透明度、人类监督、日志记录——相关条款在 2026-2027 年间分阶段生效（EU AI Act - Regulation (EU) 2024/1689）。注意两者互补但不同：ISO 42001 认证是良好流程的证据，而非 AI 法案合规的自动证明。

这些框架都不替代本书里的工程；它们组织这些工程，并让它对审计方、客户和监管者可读。**展望**把跨层治理一致性标记为开放问题，恰恰因为 policy、审计和运行时强制仍然活在分离的层里——这些框架是当下让它们保持对齐的最好脚手架。

图：生产外壳



成本、隐私和租户隔离在每次运行处强制；trace 既是成本账本也是审计日志；每次 harness 改动都要过 eval 闸门和 canary 才到生产。

要点

- **Agent 需要明确预算**：开放式 loop 是失控账单；给每任务的 token/工具/成本设上限，并用路由、级联、缓存和 token 高效工具控制花费（FrugalGPT）。
- **Trace 是成本账本**：附上每 span 的 token 和美元，“agent 很贵”就变成一个具体、可修的工程任务。
- **隐私是 lethal trifecta 的数据那一面**：最小化进入 context 的东西、在边界脱敏（trace 也会泄露）、按用户权限过滤检索、管控出口。
- **多租户要求隔离**：按租户划界工作区、凭据、记忆和前缀缓存，并在 event log 里携带租户身份——状态串味和权限串味都是灾难性的。
- **两种缓存，两种风险画像**：前缀缓存廉价地服务相同上下文；语义缓存服务相似查询、能省掉整次调用，但一次错误命中会返回错误答案——调好它的阈值，并像任何 harness 变更一样对它做 eval。
- **发布 harness 改动是一次统计性部署**：用 eval 把关、逐步 canary、把整个配置一起版本化、把生产失败变成回归案例。
- **治理框架横跨指南到法律**：OWASP LLM Top 10 和 NIST AI RMF 组织这些控制；ISO/IEC 42001 认证流程；欧盟《AI 法案》让高风险用途的义务具有约束力。

延伸阅读

- Lingjiao Chen, Matei Zaharia, and James Zou, *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*, 2023. <https://arxiv.org/abs/2305.05176>
- Simon Willison, *The lethal trifecta for AI agents*, Jun 2025. <https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/>
- OWASP, *Top 10 for Large Language Model Applications*, 2025. <https://genai.owasp.org/llm-top-10/>
- NIST, *AI Risk Management Framework (AI RMF 1.0)*, 2023. <https://www.nist.gov/itl/ai-risk-management-framework>
- Gian Segato, *Quantifying Infrastructure Noise in Agentic Coding Evals*, Anthropic, Feb 2026. <https://www.anthropic.com/engineering/infrastructure-noise>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Fu Bang, *GPTCache: An Open-Source Semantic Cache for LLM Applications*, NLP-OSS @ EMNLP 2023. <https://github.com/zilliztech/GPTCache>
- *ISO/IEC 42001:2023 - Information technology - Artificial intelligence - Management system*, ISO, 2023. <https://www.iso.org/standard/42001>
- *Regulation (EU) 2024/1689 (Artificial Intelligence Act)*, European Union, Jun 2024. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng>

第 18 章：展望

18.1 这个领域仍然年轻

本书使用的许多词汇，例如 `initializer agents`、`context firewalls`、`sprint contracts`、`reasoning sandwiches`、`ambient affordances`、`computational vs. inferential controls`，都是在最近十二到十八个月里进入主流 `agent-engineering` 讨论的。部分底层思想出现得更早，但共享语言很新。多数来源文章发表于 2025 和 2026 年。这个领域变化快于任何单本书能记录的速度——而这个速度有一个数字：METR 发现，前沿模型的任务完成时间视野 (*time horizon*) (它们有一半时间能完成的人类任务长度) 大约每七个月翻一番 (第 7.11 节) ([Kwa et al. - Measuring AI Ability to Complete Long Tasks](#))。一本讲 harness 必须提供什么的书，某种程度上也是一本讲移动靶的书。

LangChain 对演进轨迹的描述很诚实：随着模型改进，今天 harness 中的一些东西会被模型吸收。模型会在规划、自验证、长周期连贯性上更原生地变强，因此需要更少上下文注入。但有趣的 harness 组合空间不会随模型变强而收缩；它会移动 ([LangChain - The Anatomy of an Agent Harness](#); [Anthropic - Harness Design for Long-Running Application Development](#))。

18.2 开放问题

文献中反复出现几类开放问题：

- **面向功能正确性的 behavior harness**。Maintainability 和 architecture-fitness harness 有几十年现成工具。Behavior harness，即应用功能行为是否满足用户意图，没有这样的成熟工具。今天多数团队依赖 AI 生成测试，普遍共识是这还不够好 ([Thoughtworks - Harness Engineering](#))。
- **规模化 harness 的一致性**。当 guides 和 sensors 增多，它们如何保持一致？sensor 从不触发意味着质量好，还是检测不足？今天还没有类似 code coverage 或 mutation testing 的 harness coverage 指标。
- **跨层 governance 一致性**。OpenReview 综述强调，policy、permission prompt、audit log、constitutional instruction 和 runtime hook 往往分布在不同层，可能彼此干扰而不是自然组合。可迁移的 policy 与 audit language 仍然缺位 ([OpenReview - Agent Harness Engineering: A Survey](#))。
- **标准化 readiness reporting**。如果模型分数依赖 execution environment、tool surface、context policy、retry 规则和治理门槛，benchmark 报告就需要一份 harness bill of materials。今天还没有广泛采用的配置披露格式 ([OpenReview - Agent Harness Engineering: A Survey](#))。
- **Cost-quality-speed 三难**。更强沙箱、更丰富可观测性、更深验证和更严格治理，通常会增加成本和延迟。成熟 harness 需要明确哪些检查同步运行、哪些离线运行、哪些风险值得昂贵控制 ([OpenReview - Agent Harness Engineering: A Survey](#))。
- **Capability-control 权衡**。更多工具、记忆、自治和网络触达会提高任务覆盖，但也提高选择错误、prompt-injection surface、provenance 风险和审计负担。能力与控制是一条设计轴，而不是两个独立问题。
- **超越同步编排的多 agent 协调**。Anthropic 的研究系统同步运行 sub-agents；异步协调会释放更多并行性，但带来结果协调、状态一致性、错误传播挑战 ([Anthropic - How We Built Our Multi-Agent Research System](#))。
- **Harness 层的持续学习**。让 agent 在多个 session 中积累代码库或领域知识，而不是每次从零开始的 memory primitive，仍是活跃研究方向 ([LangChain - Improving Deep Agents with Harness Engineering](#))。
- **Human-in-the-loop 审批**。harness 应该在何处暂停以等待人工决策，以及如何在淹没操作者的前提下呈现足够的上下文来支撑该决策，目前还没有定型的界面。审批门过粗会被随手放行；过细则会抵消自治本身的意义。
- **Just-in-time tool assembly**。根据任务动态组合合适工具与上下文，而不是预配置一切，是 LangChain 等探索的方向 ([LangChain - Improving Deep Agents with Harness Engineering](#))。
- **Trace 作为文档**。LangChain 观察到“在软件中，代码记录 app；在 AI 中，traces 记录 app”，这暗示了一种新文档模型，但领域尚未完全解决 ([LangChain - The Anatomy of an Agent Harness](#))。
- **端到端供应链治理**。工具完整性只是其中一块。Agent 还依赖 MCP servers、外部 packages、datasets、retrieval sources 和生成出的依赖名。覆盖整条链路的 provenance 仍然不足 ([OpenReview - Agent Harness Engineering: A Survey](#))。

18.3 长期建议

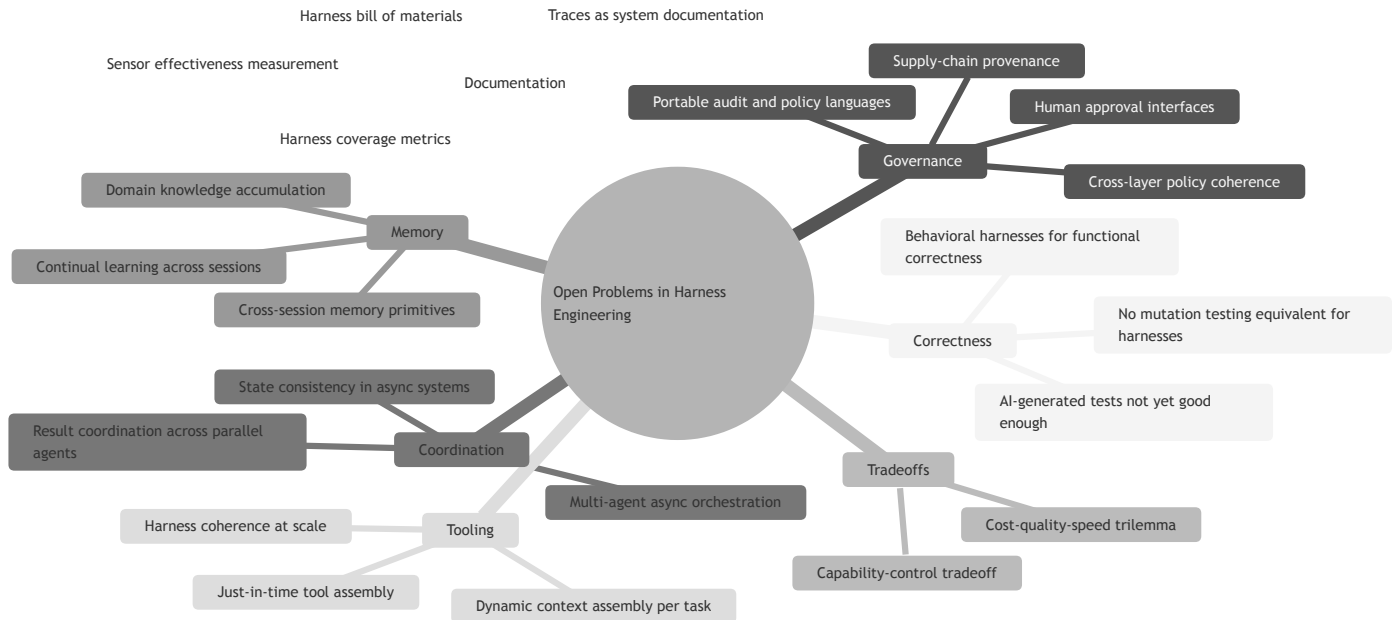
几条原则几乎贯穿所有文章：

- **把上下文当有限资源**。寻找最小高信号 token 集合，以产生期望结果。
- **做最简单可行事**。Agent 昂贵；workflow 往往足够；许多任务二者都不需要。
- **阅读 transcripts**。其他一切都从这里流出。
- **迭代 load-bearing 的东西**。模型变化时压力测试组件。移除不再有负载的部分，调优仍有负载的部分。

- 按模型定制 **harness**，但把原则用于整个领域。具体 prompt 和工具会随模型变化；工作形状，例如上下文工程、工具设计、评估、沙箱、自验证，不会消失。

这个领域还没有成熟到可以拥有一本 textbook。本文件是在它仍然会快速过时的前提下，尝试写一本。希望引用能让读者在对话继续演进时回到原文。

图：开放问题 Mindmap



要点

- 共享词汇仍很年轻：许多术语在 2025-2026 年才变得常见，尽管底层思想更早。
- 模型吸收 **harness**，但 **harness** 会移动：模型原生能力增强后，有趣 harness 工作会转向更难问题，而不是消失。
- 开放问题已经横跨完整 **ETCLOVG** 栈：behavioral correctness、规模化一致性、governance 一致性、readiness reporting、cost-quality-speed、capability-control、异步协调、持续学习、just-in-time tool assembly、traces-as-documentation。
- 五条长期原则跨场景成立：有限上下文、最简单可行、读 transcript、迭代 load-bearing、按模型定制。
- Harness engineering** 是长期工作：不是模型变强后丢弃的脚手架，而是围绕越来越强核心构建有效系统的持续工艺。

延伸阅读

- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- Prithvi Rajasekaran, *Harness Design for Long-Running Application Development*, Anthropic, Mar 2026. <https://www.anthropic.com/engineering/harness-design-long-running-apps>
- Birgitta Böckeler, *Harness Engineering for Coding Agent Users*, Thoughtworks / martinfowler.com, Apr 2026. <https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html>
- Jeremy Hadfield et al., *How We Built Our Multi-Agent Research System*, Anthropic, Jun 2025. <https://www.anthropic.com/engineering/multi-agent-research-system>
- Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026. <https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
- Awesome Harness Engineering* reading list: <https://github.com/walkinglabs/awesome-harness-engineering>
- Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>
- Thomas Kwa et al., *Measuring AI Ability to Complete Long Tasks*, METR / arXiv, Mar 2025. <https://arxiv.org/abs/2503.14499>

参考文献

Agent Harness：实践者教材的完整书目。

基础（前置）

- *LLM Foundations for Harness Engineering* — 本书所依赖的前置 stage-1 卷。../llm-foundations-zh/
-

基础

- OpenAI, *Harness Engineering: Leveraging Codex in an Agent-First World*, Feb 2026.
<https://openai.com/index/harness-engineering/>
 - *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>
 - Justin Young et al., *Effective Harnesses for Long-Running Agents*, Anthropic, Nov 2025.
<https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>
 - Prithvi Rajasekaran, *Harness Design for Long-Running Application Development*, Anthropic, Mar 2026.
<https://www.anthropic.com/engineering/harness-design-long-running-apps>
 - Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026.
<https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
 - Birgitta Böckeler, *Harness Engineering for Coding Agent Users*, Thoughtworks / martinfowler.com, Apr 2026.
<https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html>
 - Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024.
<https://www.anthropic.com/engineering/building-effective-agents>
 - Shunyu Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, arXiv, Oct 2022.
<https://arxiv.org/abs/2210.03629>
 - Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026.
<https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
-

上下文、记忆与工作状态

- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Anthropic, Sep 2025.
<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
 - Yichao 'Peak' Ji, *Context Engineering for AI Agents: Lessons from Building Manus*, Manus, Jul 2025.
<https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>
 - Charles Packer et al., *MemGPT: Towards LLMs as Operating Systems*, arXiv, Oct 2023.
<https://arxiv.org/abs/2310.08560>
 - Prateek Chhikara et al., *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*, arXiv, Apr 2025. <https://arxiv.org/abs/2504.19413>
 - Kevin Lin et al., *Sleep-time Compute: Beyond Inference Scaling at Test-time*, arXiv, Apr 2025.
<https://arxiv.org/abs/2504.13171>
-

工具与约束

- Ken Aizawa, *Writing Effective Tools for Agents — with Agents*, Anthropic, Sep 2025.
<https://www.anthropic.com/engineering/writing-tools-for-agents>
- Adam Jones and Conor Kelly, *Code Execution with MCP: Building More Efficient Agents*, Anthropic, Nov 2025.
<https://www.anthropic.com/engineering/code-execution-with-mcp>
- David Dworken and Oliver Weller-Davies, *Beyond Permission Prompts: Making Claude Code More Secure and Autonomous*, Anthropic, Oct 2025.
<https://www.anthropic.com/engineering/claude-code-sandboxing>

- Simon Willison, *The lethal trifecta for AI agents: private data, untrusted content, and external communication*, Jun 2025. <https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/>
 - *Agent2Agent (A2A) Protocol*, Google / Linux Foundation, 2025. <https://github.com/a2aproject/A2A>
 - Linux Foundation, *Linux Foundation Launches the Agent2Agent Protocol Project*, Jun 2025. <https://www.linuxfoundation.org/press/linux-foundation-launches-the-agent2agent-protocol-project-to-enable-secure-intelligent-communication-between-ai-agents>
-

规格与工作流设计

- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
-

推理与规划模式

(另见“基础”中的 *ReAct*, 即推理-行动的基础模式。)

- Noah Shinn et al., *Reflexion: Language Agents with Verbal Reinforcement Learning*, arXiv, Mar 2023. <https://arxiv.org/abs/2303.11366>
 - Aman Madaan et al., *Self-Refine: Iterative Refinement with Self-Feedback*, arXiv, Mar 2023. <https://arxiv.org/abs/2303.17651>
 - Zhibin Gou et al., *CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing*, arXiv, May 2023. <https://arxiv.org/abs/2305.11738>
 - Shunyu Yao et al., *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*, arXiv, May 2023. <https://arxiv.org/abs/2305.10601>
 - Binfeng Xu et al., *ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models*, arXiv, May 2023. <https://arxiv.org/abs/2305.18323>
 - Andy Zhou et al., *Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models*, arXiv, Oct 2023. <https://arxiv.org/abs/2310.04406>
-

多代理系统

- Mert Cemri et al., *Why Do Multi-Agent LLM Systems Fail?* (提出 MAST 失败分类法), UC Berkeley / arXiv, Mar 2025. <https://arxiv.org/abs/2503.13657>
-

可靠性与运营安全

- Temporal, *Durable Execution Meets AI: Why Temporal Is the Perfect Foundation for AI*, 2025. <https://temporal.io/blog/durable-execution-meets-ai-why-temporal-is-the-perfect-foundation-for-ai>
 - LangChain, *Durable Execution* (LangGraph documentation), 2025. <https://docs.langchain.com/oss/python/langgraph/durable-execution>
 - Martin Fowler, *CircuitBreaker*, martinowler.com, Mar 2014. <https://martinfowler.com/bliki/CircuitBreaker.html>
 - Thinkst, *Canarytokens* (free tripwire tokens). <https://canarytokens.org/>
-

Agent 能力度量

- Thomas Kwa et al., *Measuring AI Ability to Complete Long Tasks*, METR / arXiv, Mar 2025. <https://arxiv.org/abs/2503.14499>
-

Loop Engineering (循环工程)

- Addy Osmani, *Loop Engineering*, addyosmani.com, Jun 2026. <https://addyosmani.com/blog/loop-engineering/>
- *Loop Engineering*, O'Reilly Radar, 2026. <https://www.oreilly.com/radar/loop-engineering/>

- Andrew Ng, *Three Loops for Building 0-to-1 AI Products*, The Batch, Jun 2026. <https://www.deeplearning.ai/the-batch/>
 - *The Anthropic leader who built Claude Code ditched prompting — now he writes loops*, The New Stack, 2026. <https://thenewstack.io/loop-engineering/>
 - *The Agentic Loop: A Practical Field Guide*, DEV Community, 2026. <https://dev.to/truongpx396/the-agentic-loop-a-practical-field-guide-mnc>
 - *Loop Engineering Guide (2026)*, AI Builder Club. <https://www.aibuilderclub.com/blog/loop-engineering-guide-2026>
 - *Loop Engineering Crash Course*, The AI Agent Factory (Panaversity). <https://agentfactory.panaversity.org/docs/loop-engineering-crash-course>
-

评估与可观测性

- Mikaela Grace et al., *Demystifying Evals for AI Agents*, Anthropic, Jan 2026. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
 - Gian Segato, *Quantifying Infrastructure Noise in Agentic Coding Evals*, Anthropic, Feb 2026. <https://www.anthropic.com/engineering/infrastructure-noise>
 - Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026. <https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
 - OpenTelemetry, *Semantic conventions for generative AI spans*. <https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-spans/>
-

运行时、Harness 与参考实现

- Harrison Chase, *Agent Frameworks, Runtimes, and Harnesses*, Oh My!, LangChain, Oct 2025. <https://blog.langchain.com/agent-frameworks-runtimes-and-harnesses-oh-my/>
 - Jeremy Hadfield et al., *How We Built Our Multi-Agent Research System*, Anthropic, Jun 2025. <https://www.anthropic.com/engineering/multi-agent-research-system>
-

指令与 Prompting

- Eric Wallace et al., *The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions*, OpenAI, Apr 2024. <https://arxiv.org/abs/2404.13208>
-

模型选择、路由与推理

- Isaac Ong et al., *RouteLLM: Learning to Route LLMs with Preference Data*, 2024. <https://arxiv.org/abs/2406.18665>
 - Lingjiao Chen, Matei Zaharia, and James Zou, *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*, 2023. <https://arxiv.org/abs/2305.05176>
 - Charlie Snell et al., *Scaling LLM Test-Time Compute Optimally Can Be More Effective Than Scaling Model Parameters*, 2024. <https://arxiv.org/abs/2408.03314>
 - DeepSeek-AI, *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*, 2025. <https://arxiv.org/abs/2501.12948>
 - LiteLLM (BerriAI), *Python SDK and Proxy Server (AI Gateway)*. <https://github.com/BerriAI/litellm>
 - Portkey, *AI Gateway*. <https://github.com/Portkey-AI/gateway>
-

人-Agent 交互

- Saleema Amershi et al., *Guidelines for Human-AI Interaction*, CHI 2019. <https://www.microsoft.com/en-us/research/publication/guidelines-for-human-ai-interaction/>
- Eric Horvitz, *Principles of Mixed-Initiative User Interfaces*, CHI 1999. <https://www.microsoft.com/en-us/research/publication/principles-of-mixed-initiative-user-interfaces/>

Computer-Use 与多模态 Agent

- Anthropic, *Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku*, Oct 2024. <https://www.anthropic.com/news/3-5-models-and-computer-use>
 - OpenAI, *Computer-Using Agent (Operator)*, Jan 2025. <https://openai.com/index/computer-using-agent/>
 - Jianwei Yang et al., *Set-of-Mark Prompting Unleashes Extraordinary Visual Grounding in GPT-4V*, 2023. <https://arxiv.org/abs/2310.11441>
 - Boyuan Zheng et al., *GPT-4V(ision) is a Generalist Web Agent, if Grounded (SeeAct)*, 2024. <https://arxiv.org/abs/2401.01614>
 - Shuyan Zhou et al., *WebArena: A Realistic Web Environment for Building Autonomous Agents*, 2023. <https://arxiv.org/abs/2307.13854>
 - Tianbao Xie et al., *OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments*, 2024. <https://arxiv.org/abs/2404.07972>
-

成本、隐私与治理框架

- OWASP, *Top 10 for Large Language Model Applications*, 2025. <https://genai.owasp.org/llm-top-10/>
 - NIST, *AI Risk Management Framework (AI RMF 1.0)*, 2023. <https://www.nist.gov/itl/ai-risk-management-framework>
 - *ISO/IEC 42001:2023 — Information technology — Artificial intelligence — Management system*, ISO, 2023. <https://www.iso.org/standard/42001>
 - *Regulation (EU) 2024/1689 (Artificial Intelligence Act)*, European Union, Jun 2024. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng>
 - Fu Bang, *GPTCache: An Open-Source Semantic Cache for LLM Applications*, NLP-OSS @ EMNLP 2023. <https://github.com/zilliztech/GPTCache>
-

姊妹卷

- *LLM Foundations for Harness Engineering* (本书的模型内部姊妹卷)。../llm-foundations-zh/
-

阅读列表

- *Awesome Harness Engineering*, walkinglabs. <https://github.com/walkinglabs/awesome-harness-engineering>
- *Learn Harness Engineering*, Walking Labs. <https://walkinglabs.github.io/learn-harness-engineering/zh/>
- *Learn Harness Engineering — Skills*, Walking Labs. <https://walkinglabs.github.io/learn-harness-engineering/zh/skills/>

术语表

本书所用术语的简明定义。括注的章节是该术语被深入讨论的位置;某条定义的来源,可顺着对应章节的行内引用和 [参考文献](#) 查找。

核心概念

Agent(代理) — 语言模型,加上让它能真正做事的那套系统:浏览代码库、运行代码、调用工具、从错误中恢复、支撑多步骤任务。由公式 $Agent = Model + Harness$ 概括(第 1 章)。

Agent harness — 围绕模型工程化出来的一切:系统提示、工具及其描述、内置基础设施、子代理编排、控制流、hooks/中间件,以及评估基础设施(第 1 章)。

Agent loop(agent 循环) — 核心执行循环:组装上下文 → 模型发出工具调用或最终答案 → harness 执行调用 → 把结果追加进上下文 → 重复直到完成(第 1 章)。

Augmented LLM(增强型 LLM) — 配备了检索、工具和记忆的模型,能生成自己的查询、选择工具、决定保留什么。是 agent 的基本构件(第 1、6 章)。

ACI(agent-computer interface,代理-计算机接口) — agent 与其工具之间的设计界面,类比 HCI:agent 如何使用工具,值得投入与"人类如何使用界面"同等的工程(第 4 章)。

Builder harness — AI 实验室作为 coding agent 产品一部分而提供的系统提示和工具;harness 三层同心圆中的中间层(第 1 章)。

User harness — 团队为适配自己代码库而在 coding agent 之上添加的 AGENTS.md、hooks、skills 和 review agents;最外层(第 1 章)。

Harness engineering — 迭代模型周边的整个系统(而不只是单个提示),让每个观察到的失败被永久工程化掉(第 1 章)。

Binding-constraint thesis(约束瓶颈命题) — 对长周期 agent 来说,可靠性常常受 harness 层限制,包括 execution、tools、context、lifecycle、observability、verification 和 governance,而不只是受模型能力限制(第 1、11 章)。

ETCLOVG — Agent harness engineering 的七层分类:Execution environment、Tool interface、Context、Lifecycle、Observability、Verification、Governance(第 1 章)。

Context engineering(上下文工程) — 在推理时策划上下文窗口中最小的一组高信号 token;比 harness engineering 低一层的实践(第 1、2 章)。

MCP(Model Context Protocol,模型上下文协议) — 一个开放的客户端-服务器标准,用于把工具、资源和提示暴露给 agent,使任何兼容客户端都能发现并调用它们,无需定制集成(第 4 章)。

A2A(Agent-to-Agent protocol) — 用于不透明 agentic applications 之间委托的协议边界;它与主要向单个 agent runtime 暴露工具和上下文的 MCP 互补(第 4 章)。

Protocol boundary(协议边界) — 工具或 agent 标准跨越的集成线:model-to-function、agent-to-external-capability、agent-to-agent、agent-to-repo/environment(第 4 章)。

Tool call(工具调用) — 模型发出的结构化输出(通常是 JSON),指明工具名和参数。由确定性的 harness 代码决定如何处理(见《LLM Foundations》第 12 章)(第 4、8 章)。

结构化输出(structured output) — 被约束成机器可读形状的模式输出,通常是 JSON 或 XML,便于软件可靠解析。工具调用是它在 agent 系统中的典型形式(见《LLM Foundations》第 12 章)(第 1、4、8 章)。

上下文与记忆

Context window(上下文窗口) — 模型在一次推理调用中能关注的有限 token 跨度(见《LLM Foundations》第 9 章)。在 harness 中,它是每个系统提示、工具结果和历史轮次都要争抢的预算(第 2 章)。

Context rot — 随着上下文变长,模型准确回忆和使用信息的能力下降(见《LLM Foundations》第 9 章)。其 harness 视角:它是首要的运行约束,在本书中以 attention budget 来刻画(第 2 章)。

Attention budget(注意力预算) — 把上下文视为有限资源、每个新增 token 都在花费它的视角(第 2 章)。

KV-cache — 对已处理 token 的 key/value 张量的缓存(见《LLM Foundations》第 9 章)。相同的上下文前缀可由它服务,把首 token 延迟和成本降低约十倍;在 harness 中,前缀稳定性成为一个生产成本杠杆(第 2 章)。

Prefill / decode(预填充/解码) — prefill 是处理输入提示,decode 是生成输出 token(见《LLM Foundations》第 9 章)。Agentic 工作负载严重偏向 prefill(输入输出比约 100:1)(第 2 章)。

Lost-in-the-middle — 模型对长上下文中段信息的关注,不如对开头和结尾可靠的倾向(见《LLM Foundations》第 9 章)(第 2、3 章)。

Compaction(压缩) — 在对话接近上下文上限时将其总结,并用该总结重新开启一个新窗口。有损(见《LLM Foundations》第 9 章)(第 3 章)。

Context reset(上下文重置) — 完全清空上下文,用结构化 handoff 启动一个全新 agent——区别于原地压缩(第 7 章)。

Just-in-time retrieval(即时检索) — 通过轻量引用(文件路径、查询、链接)按需把数据加载进上下文,而不是预先 embed 一切(第 2 章)。

Recitation(反复复述) — 反复把目标或 todo list 重写到上下文尾部,使其留在模型最近的注意范围内(第 3 章)。

结构化笔记(agentic memory) — 让 agent 把进度笔记写到磁盘,以便上下文重置后重新加载(第 3 章)。

MemGPT — 一种记忆架构,把上下文窗口当作 OS 式的“主存”、把外部存储当作“磁盘”,让模型通过函数调用把信息换入换出(虚拟上下文管理)(第 3 章)。

Mem0 — 一层记忆:跨会话动态抽取、整合并检索显著事实,并有可选的图变体捕获实体关系(第 3 章)。

Sleep-time compute(睡眠期计算) — 在请求之间离线处理上下文——预判可能查询并预计算推断——以削减后续查询所需的计算(第 3 章)。

子代理与 workflow

Sub-agent(子代理) — 在自己的上下文窗口内处理聚焦任务、只向父代理返回浓缩摘要的专门 agent(第 3 章)。

Context firewall(上下文防火墙) — 子代理模式的一个性质:父代理永远看不到子代理的中间噪声,只接收其浓缩结果(第 3 章)。

Orchestrator-workers — 一种 workflow:中心 LLM 动态分解任务、委托给 worker LLM,并综合结果(第 6 章)。

Evaluator-optimizer — 一种 workflow:一个 LLM 生成、另一个 LLM 批评,循环往复直到满足评价标准(第 6 章)。

Micro-agent(小代理) — 嵌入在确定性 workflow 中的小而聚焦的 agent(约 3-20 步),而非开放式“loop until done”的 agent(第 6、8 章)。

Multi-agent topology(多代理拓扑) — 多代理系统的协调形态:orchestrator-worker、hierarchical、blackboard/shared-memory,或 debate/voting(第 3、6 章)。

MAST(多代理系统失败分类法) — 一套经验性的多代理失败分类,含 14 种失败模式,分三大类:规格问题、代理间错位、任务验证(第 3 章)。

推理 / 自我纠错模式 — 单个 agent 用 token 换可靠性的思考模式:Reflexion(自我批评记忆)、Self-Refine(批评并修订)、CRITIC(工具落地的批评)、Tree of Thoughts 与 LATS(分支搜索)、ReWOO(先规划再执行)(第 6 章)。

工具与沙箱

Namespacing(命名空间) — 把相关工具放在共同前缀下(`asana_*`、`browser_*`),防止命名冲突并支持按组 masking(第 4 章)。

Action masking(动作掩码) — 在上下文中保持完整工具集稳定,同时根据当前状态约束哪些动作可被选择(第 2 章)。

Progressive disclosure(渐进披露) — 只在需要时加载工具定义、文件或指令,而不是一次性全部前置(第 4 章)。

Skill — 由文件支撑的可复用能力(通常是一个 `SKILL.md` 加上配套代码),agent 可按需加载(第 4 章)。

代码执行(作为元工具) — 把工具呈现为 agent 通过写代码来调用的代码 API,而不是直接调用——大幅降低 token 成本(第 4 章)。

Shell — Bash、zsh 这类命令行接口。在 agent 系统中,shell 访问很强大,因为它让 agent 可以运行测试、检查文件、安装包,并临时组合工具(第 1、4、5 章)。

文件系统(filesystem) — agent 可以读写的目录和文件。它既是工作区,也是持久记忆,还是 agent 与人类协作的界面(第 1、2、5 章)。

Sandbox(沙箱) — 带有文件系统和网络边界的隔离环境,agent 可在其中自由行动而无需逐动作审批(第 5 章)。

Sandbox liveness(沙箱活性) — 沙箱作为授权区域的作用:agent 可在配置边界内行动而无需逐动作审批(第 5 章)。

Governance(治理) — 管理身份、权限策略、scoped credentials、人类审批、审计日志和跨层安全问责的 harness 机制(第 5、17 章)。

Delegated auth(委托授权) — Agent 通过 scoped credentials 或 proxy-authorized identity 行动,而不是继承用户完整环境权限的模式(第 5 章)。

Supply-chain provenance(供应链来源证据) — 关于 agent 所依赖的 tools、packages、datasets、MCP servers 和 retrieval sources 的来源与完整性证据(第 5、17 章)。

Hook / middleware(中间件) — 由 harness 在生命周期事件(启动、工具调用后、停止)自动执行的脚本或检查点,确定性地强制规则(第 5 章)。

Feedforward / feedback(前馈/反馈) — 前馈控制(guides)在 agent 行动前引导它;反馈控制(sensors)在它行动后观察并帮助自我修正(第 5 章)。

Computational / inferential control(计算型/推断型控制) — 计算型控制(linter、类型检查器)确定且快;推断型控制(AI review、LLM-as-judge)能处理细微判断,但更慢、非确定(第 5 章)。

Ambient affordances(环境可供性) — 环境本身的属性(强类型、清晰模块边界、有立场的框架),使代码库对 agent 更易理解和处理(第 5 章)。

CI(持续集成) — 围绕代码变更自动运行的检查,通常包括测试、linter、构建和部署关卡。在 harness 设计中,这类检查会成为反馈型 sensor(第 5、9 章)。

Linter / type checker(linter / 类型检查器) — 在运行前发现风格、语法、结构或类型错误的确定性工具。它们是外层 harness 中常见的计算型 sensor(第 5 章)。

Prompt injection(提示注入) — 一种攻击:藏在 agent 所读内容(网页、文件、工具结果)中的指令被模型当作命令执行(见《LLM Foundations》第 8、12 章)(第 5 章)。

Lethal trifecta(致命三要素) — 同一 agent 同时具备:访问私有数据、接触不可信内容、向外通信能力——这三者的危险组合(第 5 章)。

Circuit breaker(熔断器) — 一个可靠性包装:在失败达到阈值后跳闸,让后续对失败工具、服务或子代理的调用快速失败,而不是挂起或重试成风暴(第 5 章)。

Kill switch(终止开关) — 由人或策略触发的停止,立即且独立于 agent 自身控制流地终止一个 agent 或 fleet;它存在于 harness 中,因为被操纵的 agent 不能被指望停下自己(第 5 章)。

Canary token(金丝雀令牌) — 一份被种下的假机密(未使用的 key、诱饵文件、陷阱 URL),其被访问或外泄会触发高信号警报,表明 agent 已被操纵——对 lethal-trifecta 外泄路径的检测(第 5 章)。

Action budget(动作预算) — 对工具调用、token、墙钟时间或花费设的硬性上限,达到后循环停止并上报,而非失控奔跑(第 5、8、17 章)。

评估

Eval harness(评估 harness) — 端到端运行评估的基础设施;区别于被评估的 agent harness(第 10 章)。

Readiness validation(就绪验证) — 验证某个具体 model + harness 配置是否适合特定任务分布、环境、预算和治理规则(第 10 章)。

Failure attribution(失败归因) — 在选择修复方式前,先把 agent 失败标注到最可能的问题层:execution、tool interface、context、lifecycle、observability、verification 或 governance(第 9、11 章)。

Task / trial(任务/试验) — task 有定义好的输入和成功标准;trial 是对它的一次尝试(第 10 章)。

Grader — 为试验某个方面评分的组件:code-based、model-based 或 human(第 10 章)。

Transcript(trace、trajectory) — 一次试验的完整记录:每条消息、工具调用和结果(第 9、11 章)。

Outcome(结果状态) — 试验结束时的最终环境状态,区别于 agent 的文本回应(第 10 章)。

Capability eval / regression eval — capability eval 衡量 agent 新近能做什么(通过率低、正在爬升);regression eval 保护它已能可靠做到的事(接近 100%)(第 10 章)。

pass@k / pass^k — pass@k 是 k 次尝试中至少一次成功的概率(随 k 上升);pass^k 是 k 次试验全部成功的概率(随 k 下降)(见《LLM Foundations》第 13 章)(第 10 章)。

Infrastructure noise(基础设施噪声) — 由运行时资源配置(而非模型能力)造成的 benchmark 分数波动(第 11 章)。

长运行代理与领域

交接班问题(shift-change problem) — 由于上下文窗口有限,后续 agent session 到来时对之前的 session 毫无记忆这一挑战(第 7 章)。

Initializer agent — 只运行一次、为后续 coding agent session 搭好项目(init 脚本、进度日志、feature list)的 agent(第 7 章)。

Managed agent — 平台管理的 agent 架构,把模型侧 brain、执行侧 hands 和持久 session/event log 分开,使它们能独立失败、重置或迁移(第 7 章)。

Brain / hands split — Managed-agent 中决策上下文(brain)与可替换执行环境(hands)的分离(第 7 章)。

Sprint contract — generator 与 evaluator 两个 agent 之间基于文件的约定,在每个构建 sprint 前敲定要构建什么、如何验证成功(第 7 章)。

Event log(事件日志) — 对消息、工具调用、结果、审批和错误的追加式记录。执行状态可以从推导出来,因此 agent 更容易重放和调试(第 9 章)。

Agent platform — 超出本地 framework 的基础设施:跨多次运行和多用户的 durable workspaces、managed sandboxes、identity、billing、observability、evaluation、governance 和 human handoff(第 8、17 章)。

Checkpoint / resume(检查点/恢复) — 一种可靠性模式:agent 定期保存足够状态,以便在失败或上下文重置后继续工作而不丢进度(第 7、8 章)。

Durable execution(持久化执行) — 一种基础设施保证:把每个工作流步骤持久化,使崩溃或被中断的 agent 从最后记录的一步恢复;非确定的模型/工具调用被记录并重放,而非重算(第 7 章)。

Time horizon(时间视野) — METR 的能力指标:模型以 50% 可靠性能完成的人类任务长度;前沿值大约每七个月翻一番(第 7、18 章)。

Stateless reducer(无状态归约器) — 把 agent 建模为对 event log 的纯 fold,使其可序列化、可重放、可测试(第 9 章)。

Model-harness co-evolution(模型与 harness 共同演化) — frontier 模型在其 harness 一起参与的情况下 post-train 所形成的耦合,因此改变任一侧都可能损害性能(第 12 章)。

Span telemetry — 以 span tree 表示的结构化 trace 数据,覆盖 model calls、tool calls、retrieval、context assembly、permissions、costs 和 outcomes(第 12 章)。

OpenTelemetry GenAI semantic conventions — 一套新兴的、面向 LLM 与 agent 遥测的标准 span 与属性名 schema(`invoke_agent`、`chat`、`execute_tool` span),让 agent trace 加入普通可观测栈(第 12 章)。

Trace-to-eval loop — 把真实生产失败转换成脱敏、可复现、带 outcome assertion 的 regression case(第 12 章)。

Meta-harness — 把 harness 设计本身当作优化对象:用 eval feedback 消融或搜索 prompts、tools、retries、context policies、evaluators 和 control loops(第 12 章)。

Cost-quality-speed trilemma(成本-质量-速度三难) — 更强 execution environment、observability、verification 和 governance 会提高可靠性,但也增加成本和延迟(第 18 章)。

Capability-control tradeoff(能力-控制权衡) — 更多权限、工具、记忆和自治会提升能力,同时扩大控制、provenance 和审计问题(第 18 章)。

Ralph Wiggum loop — 一个 hook,拦截 agent 的退出尝试,并在干净的上下文窗口中重新注入原始 prompt,迫使它继续对照目标工作(第 7、8 章)。

Loop engineering(循环工程) — 把 agent loop 本身当作设计单元:规定环绕模型的 trigger、topology、verifier 和 stop rule,使它能无人值守地运行。这是外层控制循环的运维者视角(第 8 章)。

Trigger(触发器 / heartbeat) — 无需人类 prompt 就启动一趟 loop 的东西:一个 schedule、一个 webhook,或另一个 agent(第 8 章)。

Verifier(验证器 / maker-checker) — 决定“够好了”的固定标准,由一个不同于产出工作的 agent 来施加,使 maker 不能批改自己的作业;是 loop 设计的瓶颈(第 8 章)。

Stop rule(停止规则) — 结束一个 loop 的明确条件——success、no-op、ask-for-approval——外加兜住失控的三个硬停:最大迭代次数、无进展检测、预算上限(第 8 章)。

Closed vs. open loop(闭环与开环) — 闭环预先钉死硬的、可检查的验收标准,放着跑是安全的;开环朝模糊目标探索,需要一个更强的 verifier,否则会 ship 出自信的垃圾(第 8 章)。

指令与模型选择

Instruction hierarchy(指令层级) — 指令按来源带有不同权威——system 高于 developer 高于 user 高于工具/检索内容——使低优先级指令无法覆盖高优先级指令。Prompt injection 就是这一层级的失效(第 13 章)。

Right altitude(合适的高度) — System prompt 的目标具体程度:具体到能可靠引导行为,一般到能跨情况迁移,既不论为脆弱的硬编码规则,也不流于含糊指引(第 13 章)。

Model routing(模型路由) — 给请求的难度分类,把简单的派给便宜的弱模型、把困难的派给昂贵的强模型。只有当路由决策远比它带来的节省更便宜时才划算(第 14 章)。

LLM cascade(级联) — 先试便宜模型,只在 verifier 否决便宜答案时才升级到更强的模型。升级信号可靠时,能以更低成本匹配强模型准确率(第 14 章)。

Fallback(回退) — 当主模型出错、超时或被限时切换到备用模型,使 agent 优雅降级(第 14 章)。

AI gateway(AI 网关) — 位于 harness 与各模型供应商之间的基础设施组件,对外呈现一个统一接口覆盖多个模型,并承载路由、fallback、预算、缓存和日志(如 LiteLLM、Portkey)(第 14、17 章)。

Reasoning model(推理模型) — 经过 post-training(通常是在可验证奖励上做 RL),学会在回答前生成很长内部推理、用 inference token 换困难任务上更好表现的模型(第 14 章;LLM Foundations 第 7–8 章)。

Test-time compute — 在回答时花更多 inference token、时间和金钱以在困难问题上做得更好——区别于更大模型或更多硬件的一条 scaling 轴(第 14 章)。

人类交互

Permission fatigue(许可疲劳) — 当 agent 过于频繁请求批准时监督的退化,把人训练成不读就盖橡皮图章(第 5、14 章)。

Mixed-initiative(混合主动) — 一种交互风格:系统逐动作决定是自主行动还是让步给人,并管理打断的代价(第 15 章)。

Approval as a tool call(批准即工具调用) — 把人类批准建模为 agent 调用的一个工具,使请求成为持久、可重放、可审计、并与挂起/恢复组合的事件(第 15 章)。

Steering(引导) — 把一条新指令注入正在运行的 agent,使其在下一回合被纳入,从而在不丢 session 状态的前提下重定向(第 15 章)。

Calibrated trust(校准过的信任) — 人接口的目标:人对 agent 的信任恰好等于它在给定任务上配得到的程度,通过透明和扎根于验证的不确定性、而非流畅度来实现(第 15 章)。

Computer-Use Agent

Computer-use agent — 通过 GUI 操作软件的 agent——查看截图并发出光标、键盘和导航动作——而不是调用定义好的 API(第 16 章)。

Visual grounding(视觉接地) — 把意图(“点击 Submit”)翻译成具体动作(在特定坐标点击);一种在 API 工具里没有对应物的错误模式(第 16 章)。

Set-of-Mark prompting — 在候选可交互元素上叠加编号标记,让模型选择离散标签而不是产出裸坐标,提升 grounding 可靠性(第 16 章)。

Accessibility tree(可达性树) — UI 的结构化语义表示(role、label、state),为辅助技术构建;常比裸像素或 DOM 更紧凑、更精确的屏幕编码(第 16 章)。

成本与运维

Per-task budget(每任务预算) — 对单次 agent 运行的 token、工具调用或成本设的明确上限,超过后 agent 停下来问,而不是无限循环(第 17 章)。

Cost attribution(成本归因) — 给 trace 的每个 span 附上 token 和美元成本,把“agent 很贵”变成一个具体、可修的工程发现(第 17 章)。

Multi-tenancy / 租户隔离 — 用一个平台服务许多用户或组织,同时防止状态串味(context/记忆/缓存跨租户泄露)和权限串味(用错误租户的凭据行事)(第 17 章)。

Canary rollout(金丝雀放量) — 把 harness 改动发布给一小部分流量,在全量部署前盯住生产 trace 和 outcome 指标,接住 eval 套件漏掉的案例(第 17 章)。

Semantic cache(语义缓存) — 一种缓存:通过嵌入查询、在相似度超过阈值时返回已存响应,来服务 *相似*(而非仅相同)的请求;能省掉整次模型调用,但有错误命中的风险(第 17 章)。

AI 管理体系(ISO/IEC 42001) — 首个用于治理组织 AI 的可认证标准:如何建立、运行并持续改进一套 AI 管理体系——ISO 27001 的 AI 对应物(第 17 章)。

欧盟 AI 法案(EU AI Act) — Regulation (EU) 2024/1689,首部全面的 AI 法律;它按风险层级对系统分类,并对高风险用途施加有约束力的义务(第 17 章)。